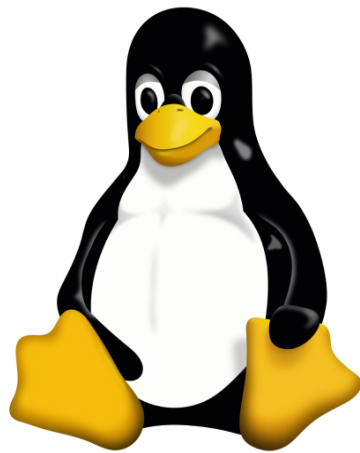


THE LINUX



COMMANDS HANDBOOK

Flavio Copes

Table of Contents

Preface

Introduction to Linux and shells

man

ls

cd

pwd

mkdir

rmdir

mv

cp

open

touch

find

ln

gzip

gunzip

tar

alias

cat

less

tail

wc

grep

sort

uniq

diff

echo

chown

chmod

umask

du

df

basename

dirname

ps

top

kill

killall

jobs

bg

fg

type

which

nohup

xargs

vim

emacs

nano

whoami

who

su

sudo

passwd

ping

traceroute

clear

history

export

crontab

uname

env

printenv

Conclusion

Preface

The Linux Commands Handbook follows the 80/20 rule: learn in 20% of the time the 80% of a topic.

I find this approach gives a well-rounded overview.

This book does not try to cover everything under the sun related to Linux and its commands. It focuses on the small core commands that you will use the 80% or 90% of the time, trying to simplify the usage of the more complex ones.

All those commands work on Linux, macOS, WSL, and anywhere you have a UNIX environment.

I hope the contents of this book will help you achieve what you want: **get comfortable with Linux**.

This book is written by Flavio. I **publish programming tutorials** every day on my website flaviocopes.com.

You can reach me on Twitter [@flaviocopes](https://twitter.com/flaviocopes).

Enjoy!

Introduction to Linux and shells

Linux is an operating system, like macOS or Windows.

It is also the most popular Open Source and free, as in freedom, operating system.

It powers the vast majority of the servers that compose the Internet. It's the base upon which everything is built upon. But not just that. Android is based on (a modified version of) Linux.

The Linux "core" (called *kernel*) was born in 1991 in Finland, and it went a really long way from its humble beginnings. It went on to be the kernel of the GNU Operating System, creating the duo GNU/Linux.

There's one thing about Linux that corporations like Microsoft and Apple, or Google, will never be able to offer: the freedom to do whatever you want with your computer.

They're actually going in the opposite direction, building walled gardens, especially on the mobile side.

Linux is the ultimate freedom.

It is developed by volunteers, some paid by companies that rely on it, some independently, but there's no single commercial company that can dictate what goes into Linux, or the project priorities.

Linux can also be used as your day to day computer. I use macOS because I really enjoy the applications, the design and I also used to be an iOS and Mac apps developer, but before using it I used Linux as my main computer Operating System.

No one can dictate which apps you can run, or "call home" with apps that track you, your position, and more.

Linux is also special because there's not just "one Linux", like it happens on Windows or macOS. Instead, we have **distributions**.

A "distro" is made by a company or organization and packages the Linux core with additional programs and tooling.

For example you have Debian, Red Hat, and Ubuntu, probably the most popular.

Many, many more exist. You can create your own distribution, too. But most likely you'll use a popular one, one that has lots of users and a community of people around it, so you can do what you need to do without losing too much time reinventing the wheel and figuring out answers to common problems.

Some desktop computers and laptops ship with Linux preinstalled. Or you can install it on your Windows-based computer, or on a Mac.

But you don't need to disrupt your existing computer just to get an idea of how Linux works.

I don't have a Linux computer.

If you use a Mac you need to know that under the hood macOS is a UNIX Operating System, and it shares a lot of the same ideas and software that a GNU/Linux system uses, because GNU/Linux is a free alternative to UNIX.

[UNIX](#) is an umbrella term that groups many operating systems used in big corporations and institutions, starting from the 70's

The macOS terminal gives you access to the same exact commands I'll describe in the rest of this handbook.

Microsoft has an official [Windows Subsystem for Linux](#) which you can (and should!) install on Windows. This will give you the ability to run Linux in a very easy way on your PC.

But the vast majority of the time you will run a Linux computer in the cloud via a VPS (Virtual Private Server) like DigitalOcean.

A shell is a command interpreter that exposes to the user an interface to work with the underlying operating system.

It allows you to execute operations using text and commands, and it provides users advanced features like being able to create scripts.

This is important: shells let you perform things in a more optimized way than a GUI (Graphical User Interface) could ever possibly let you do. Command line tools can offer many different configuration options without being too complex to use.

There are many different kind of shells. This post focuses on Unix shells, the ones that you will find commonly on Linux and macOS computers.

Many different kind of shells were created for those systems over time, and a few of them dominate the space: Bash, Csh, Zsh, Fish and many more!

All shells originate from the Bourne Shell, called `sh`. "Bourne" because its creator was Steve Bourne.

Bash means *Bourne-again shell*. `sh` was proprietary and not open source, and Bash was created in 1989 to create a free alternative for the GNU project and the Free Software Foundation. Since projects had to pay to use the Bourne shell, Bash became very popular.

If you use a Mac, try opening your Mac terminal. That by default is running ZSH. (or, pre-Catalina, Bash)

You can set up your system to run any kind of shell, for example I use the Fish shell.

Each single shell has its own unique features and advanced usage, but they all share a common functionality: they can let you execute programs, and they can be programmed.

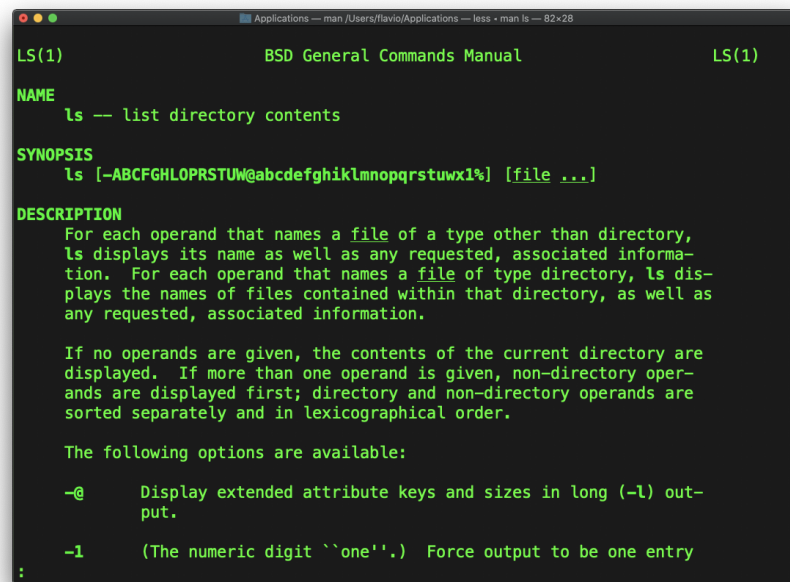
In the rest of this handbook we'll see in detail the most common commands you will use.

man

The first command I want to introduce is a command that will help you understand all the other commands.

Every time I don't know how to use a command, I type

`man <command>` to get the manual:



```
LS(1)                                BSD General Commands Manual                                LS(1)

NAME
  ls — list directory contents

SYNOPSIS
  ls [-ABCFGHLOPRSTUW@abcdeghiklmnopqrstuwx1] [file ...]

DESCRIPTION
  For each operand that names a file of a type other than directory,
  ls displays its name as well as any requested, associated information.
  For each operand that names a file of type directory, ls displays
  the names of files contained within that directory, as well as any
  requested, associated information.

  If no operands are given, the contents of the current directory are
  displayed. If more than one operand is given, non-directory operands
  are displayed first; directory and non-directory operands are sorted
  separately and in lexicographical order.

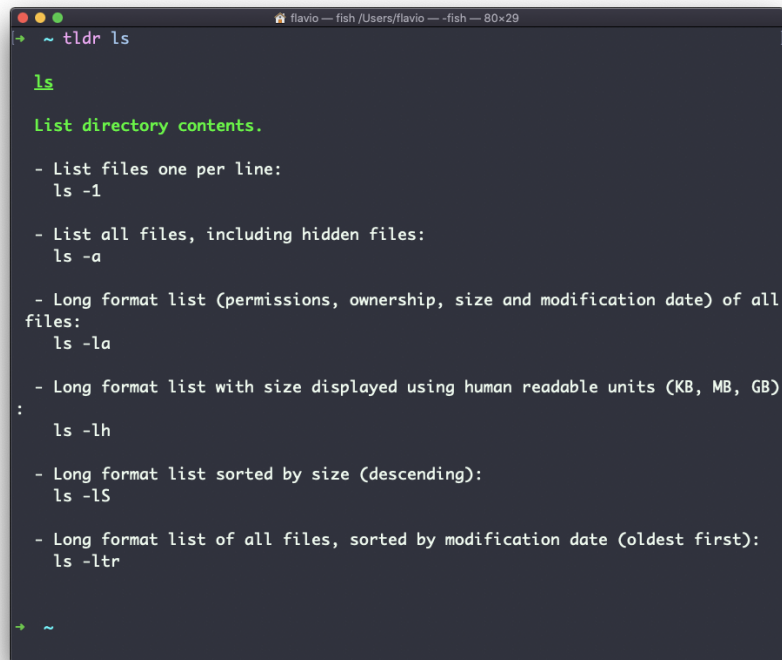
  The following options are available:

  -@      Display extended attribute keys and sizes in long (-l) output.
  -1      (The numeric digit `one'.) Force output to be one entry
  :
```

This is a man (from *manual*) page. Man pages are an essential tool to learn, as a developer. They contain so much information that sometimes it's almost too much.

The above screenshot is just 1 of 14 screens of explanation for the `ls` command.

Most of the times when I'm in need to learn a command quickly I use this site called **tldr pages**: <https://tldr.sh/>. It's a command you can install, then you run it like this: `tldr <command>`, which gives you a very quick overview of a command, with some handy examples of common usage scenarios:

A terminal window with a dark background and light text. The title bar at the top shows a home icon, the name 'flavio', and the path '— fish /Users/flavio — -fish — 80x29'. The prompt is '~ tldr ls'. The output shows 'ls' in green, followed by 'List directory contents.' in green. Then a list of options: '- List files one per line: ls -l', '- List all files, including hidden files: ls -a', '- Long format list (permissions, ownership, size and modification date) of all files: ls -la', '- Long format list with size displayed using human readable units (KB, MB, GB): ls -lh', '- Long format list sorted by size (descending): ls -ls', and '- Long format list of all files, sorted by modification date (oldest first): ls -ltr'. The prompt at the bottom is '~'.

```
~ tldr ls

ls

List directory contents.

- List files one per line:
  ls -l

- List all files, including hidden files:
  ls -a

- Long format list (permissions, ownership, size and modification date) of all
files:
  ls -la

- Long format list with size displayed using human readable units (KB, MB, GB)
:
  ls -lh

- Long format list sorted by size (descending):
  ls -ls

- Long format list of all files, sorted by modification date (oldest first):
  ls -ltr

~
```

This is not a substitute for `man` , but a handy tool to avoid losing yourself in the huge amount of information present in a man page. Then you can use the man page to explore all the different options and parameters you can use on a command.

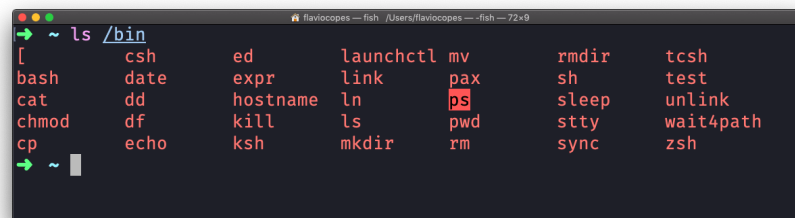
ls

Inside a folder you can list all the files that the folder contains using the `ls` command:

```
ls
```

If you add a folder name or path, it will print that folder contents:

```
ls /bin
```



`ls` accepts a lot of options. One of my favorite options combinations is `-al`. Try it:

```
ls -al /bin
```



```
➔ ~ ls -al /bin
total 5120
drwxr-xr-x@ 37 root wheel 1184 Feb 4 10:05 .
drwxr-xr-x 30 root wheel 960 Feb 8 15:32 ..
-rwxr-xr-x 1 root wheel 22704 Jan 16 02:21 [
-r-xr-xr-x 1 root wheel 618416 Jan 16 02:21 bash
-rwxr-xr-x 1 root wheel 23648 Jan 16 02:21 cat
-rwxr-xr-x 1 root wheel 34144 Jan 16 02:21 chmod
-rwxr-xr-x 1 root wheel 29024 Jan 16 02:21 cp
-rwxr-xr-x 1 root wheel 379952 Jan 16 02:21 csh
-rwxr-xr-x 1 root wheel 28608 Jan 16 02:21 date
-rwxr-xr-x 1 root wheel 32000 Jan 16 02:21 dd
-rwxr-xr-x 1 root wheel 23392 Jan 16 02:21 df
-rwxr-xr-x 1 root wheel 18128 Jan 16 02:21 echo
-rwxr-xr-x 1 root wheel 54080 Jan 16 02:21 ed
-rwxr-xr-x 1 root wheel 23104 Jan 16 02:21 expr
-rwxr-xr-x 1 root wheel 18288 Jan 16 02:21 hostname
-rwxr-xr-x 1 root wheel 18688 Jan 16 02:21 kill
-r-xr-xr-x 1 root wheel 1282864 Jan 16 02:21 ksh
-rwxr-xr-x 1 root wheel 121296 Jan 16 02:21 launchctl
```

compared to the plain `ls`, this returns much more information.

You have, from left to right:

- the file permissions (and if your system supports ACLs, you get an ACL flag as well)
- the number of links to that file
- the owner of the file
- the group of the file
- the file size in bytes
- the file modified datetime
- the file name

This set of data is generated by the `-l` option. The `-la` option instead also shows the hidden files.

Hidden files are files that start with a dot (`.`).

cd

Once you have a folder, you can move into it using the `cd` command. `cd` means **change directory**. You invoke it specifying a folder to move into. You can specify a folder name, or an entire path.

Example:

```
mkdir fruits
cd fruits
```

Now you are into the `fruits` folder.

You can use the `..` special path to indicate the parent folder:

```
cd .. #back to the home folder
```

The `#` character indicates the start of the comment, which lasts for the entire line after it's found.

You can use it to form a path:

```
mkdir fruits
mkdir cars
cd fruits
cd ../cars
```

There is another special path indicator which is `.`, and indicates the **current** folder.

You can also use absolute paths, which start from the root folder `/`:

```
cd /etc
```

This command works on Linux, macOS, WSL, and anywhere you have a UNIX environment

pwd

Whenever you feel lost in the filesystem, call the `pwd` command to know where you are:

```
pwd
```

It will print the current folder path.

mkdir

You create folders using the `mkdir` command:

```
mkdir fruits
```

You can create multiple folders with one command:

```
mkdir dogs cars
```

You can also create multiple nested folders by adding the `-p` option:

```
mkdir -p fruits/apples
```

Options in UNIX commands commonly take this form. You add them right after the command name, and they change how the command behaves. You can often combine multiple options, too.

You can find which options a command supports by typing `man <commandname>`. Try now with `man mkdir` for example (press the `q` key to esc the man page). Man pages are the amazing built-in help for UNIX.

rmmdir

Just as you can create a folder using `mkdir` , you can delete a folder using `rmdir` :

```
mkdir fruits  
rmdir fruits
```

You can also delete multiple folders at once:

```
mkdir fruits cars  
rmdir fruits cars
```

The folder you delete must be empty.

To delete folders with files in them, we'll use the more generic `rm` command which deletes files and folders, using the `-rf` options:

```
rm -rf fruits cars
```

Be careful as this command does not ask for confirmation and it will immediately remove anything you ask it to remove.

There is no **bin** when removing files from the command line, and recovering lost files can be hard.

mv

Once you have a file, you can move it around using the `mv` command. You specify the file current path, and its new path:

```
touch test
mv pear new_pear
```

The `pear` file is now moved to `new_pear`. This is how you **rename** files and folders.

If the last parameter is a folder, the file located at the first parameter path is going to be moved into that folder. In this case, you can specify a list of files and they will all be moved in the folder path identified by the last parameter:

```
touch pear
touch apple
mkdir fruits
mv pear apple fruits #pear and apple moved to the f
```

cp

You can copy a file using the `cp` command:

```
touch test  
cp apple another_apple
```

To copy folders you need to add the `-r` option to recursively copy the whole folder contents:

```
mkdir fruits  
cp -r fruits cars
```


open

The `open` command lets you open a file using this syntax:

```
open <filename>
```

You can also open a directory, which on macOS opens the Finder app with the current directory open:

```
open <directory name>
```

I use it all the time to open the current directory:

```
open .
```

The special `.` symbol points to the current directory, as `..` points to the parent directory

The same command can also be used to run an application:

```
open <application name>
```

touch

You can create an empty file using the `touch` command:

```
touch apple
```

If the file already exists, it opens the file in write mode, and the timestamp of the file is updated.

find

The `find` command can be used to find files or folders matching a particular search pattern. It searches recursively.

Let's learn it by example.

Find all the files under the current tree that have the `.js` extension and print the relative path of each file matching:

```
find . -name '*.js'
```

It's important to use quotes around special characters like `*` to avoid the shell interpreting them.

Find directories under the current tree matching the name "src":

```
find . -type d -name src
```

Use `-type f` to search only files, or `-type l` to only search symbolic links.

`-name` is case sensitive. use `-iname` to perform a case-insensitive search.

You can search under multiple root trees:

```
find folder1 folder2 -name filename.txt
```

Find directories under the current tree matching the name "node_modules" or 'public':

```
find . -type d -name node_modules -or -name public
```

You can also exclude a path, using `-not -path` :

```
find . -type d -name '*.md' -not -path 'node_modules'
```

You can search files that have more than 100 characters (bytes) in them:

```
find . -type f -size +100c
```

Search files bigger than 100KB but smaller than 1MB:

```
find . -type f -size +100k -size -1M
```

Search files edited more than 3 days ago

```
find . -type f -mtime +3
```

Search files edited in the last 24 hours

```
find . -type f -mtime -1
```

You can delete all the files matching a search by adding the `-delete` option. This deletes all the files edited in the last 24 hours:

```
find . -type f -mtime -1 -delete
```

You can execute a command on each result of the search. In this example we run `cat` to print the file content:

```
find . -type f -exec cat {} \;
```

notice the terminating `\; . {}` is filled with the file name at execution time.

ln

The `ln` command is part of the Linux file system commands.

It's used to create links. What is a link? It's like a pointer to another file. A file that points to another file. You might be familiar with Windows shortcuts. They're similar.

We have 2 types of links: **hard links** and **soft links**.

Hard links

Hard links are rarely used. They have a few limitations: you can't link to directories, and you can't link to external filesystems (disks).

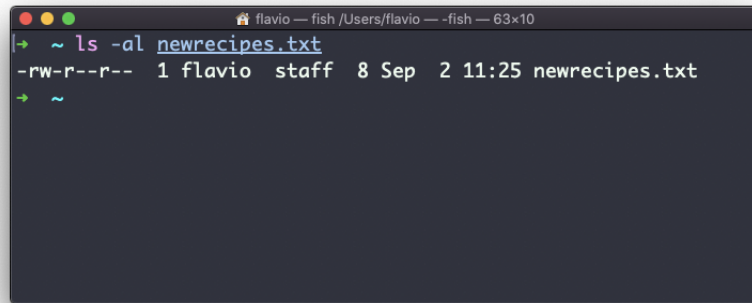
A hard link is created using

```
ln <original> <link>
```

For example, say you have a file called `recipes.txt`. You can create a hard link to it using:

```
ln recipes.txt newrecipes.txt
```

The new hard link you created is indistinguishable from a regular file:



```
flavio — fish /Users/flavio — -fish — 63x10
~ ls -al newrecipes.txt
-rw-r--r-- 1 flavio staff 8 Sep 2 11:25 newrecipes.txt
~
```

Now any time you edit any of those files, the content will be updated for both.

If you delete the original file, the link will still contain the original file content, as that's not removed until there is one hard link pointing to it.



```
flavio — fish /Users/flavio — -fish — 49x9
~ ln recipes.txt newrecipes.txt
~ cat newrecipes.txt
recipes
~ rm recipes.txt
~ cat newrecipes.txt
recipes
~
```

Soft links

Soft links are different. They are more powerful as you can link to other filesystems and to directories, but when the original is removed, the link will be broken.

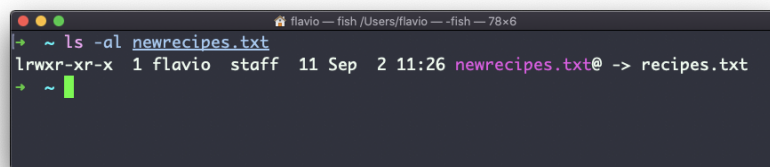
You create soft links using the `-s` option of `ln` :

```
ln -s <original> <link>
```

For example, say you have a file called `recipes.txt`. You can create a soft link to it using:

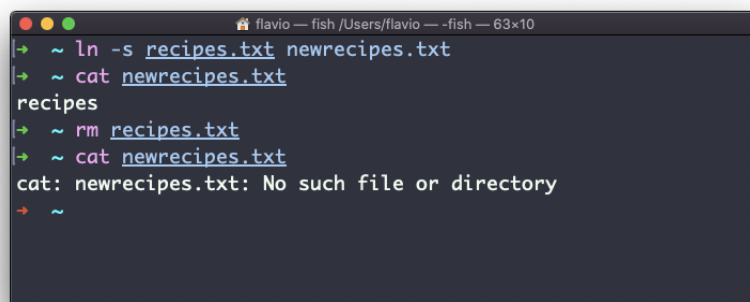
```
ln -s recipes.txt newrecipes.txt
```

In this case you can see there's a special `l` flag when you list the file using `ls -al`, and the file name has a `@` at the end, and it's colored differently if you have colors enabled:



```
flavio — fish /Users/flavio — -fish — 78x6
~ % ls -al newrecipes.txt
lrwxr-xr-x  1 flavio  staff   11 Sep  2 11:26 newrecipes.txt@ -> recipes.txt
~ %
```

Now if you delete the original file, the links will be broken, and the shell will tell you "No such file or directory" if you try to access it:



```
flavio — fish /Users/flavio — -fish — 63x10
~ % ln -s recipes.txt newrecipes.txt
~ % cat newrecipes.txt
recipes
~ % rm recipes.txt
~ % cat newrecipes.txt
cat: newrecipes.txt: No such file or directory
~ %
```


gzip

You can compress a file using the gzip compression protocol named [LZ77](#) using the `gzip` command.

Here's the simplest usage:

```
gzip filename
```

This will compress the file, and append a `.gz` extension to it. The original file is deleted. To prevent this, you can use the `-c` option and use output redirection to write the output to the `filename.gz` file:

```
gzip -c filename > filename.gz
```

The `-c` option specifies that output will go to the standard output stream, leaving the original file intact

Or you can use the `-k` option:

```
gzip -k filename
```

There are various levels of compression. The more the compression, the longer it will take to compress (and decompress). Levels range from 1 (fastest, worst compression) to 9 (slowest, better compression), and the default is 6.

You can choose a specific level with the `-<NUMBER>` option:

```
gzip -1 filename
```

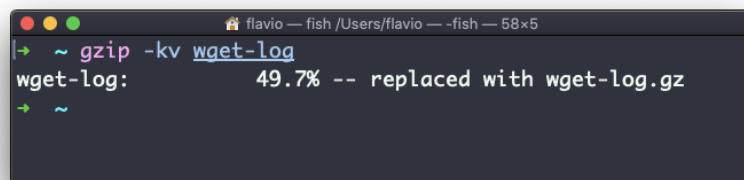
You can compress multiple files by listing them:

```
gzip filename1 filename2
```

You can compress all the files in a directory, recursively, using the `-r` option:

```
gzip -r a_folder
```

The `-v` option prints the compression percentage information. Here's an example of it being used along with the `-k` (keep) option:

A terminal window with a dark background and light text. The title bar shows 'flavio — fish /Users/flavio — -fish — 58x5'. The prompt is '~'. The command entered is 'gzip -kv wget-log'. The output is 'wget-log: 49.7% -- replaced with wget-log.gz'. The prompt is '~' again.

```
flavio — fish /Users/flavio — -fish — 58x5
~ gzip -kv wget-log
wget-log: 49.7% -- replaced with wget-log.gz
~
```

`gzip` can also be used to decompress a file, using the `-d` option:

```
gzip -d filename.gz
```

gunzip

The `gunzip` command is basically equivalent to the `gzip` command, except the `-d` option is always enabled by default.

The command can be invoked in this way:

```
gunzip filename.gz
```

This will gunzip and will remove the `.gz` extension, putting the result in the `filename` file. If that file exists, it will overwrite that.

You can extract to a different filename using output redirection using the `-c` option:

```
gunzip -c filename.gz > anotherfilename
```

tar

The `tar` command is used to create an archive, grouping multiple files in a single file.

Its name comes from the past and means *tape archive*. Back when archives were stored on tapes.

This command creates an archive named `archive.tar` with the content of `file1` and `file2` :

```
tar -cf archive.tar file1 file2
```

The `c` option stands for *create*. The `f` option is used to write to file the archive.

To extract files from an archive in the current folder, use:

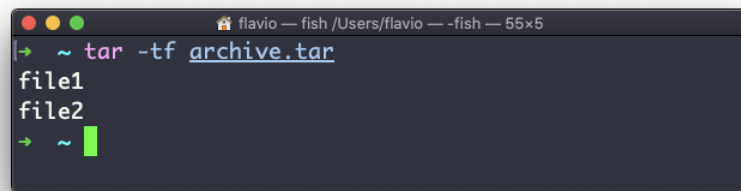
```
tar -xf archive.tar
```

the `x` option stands for *extract*

and to extract them to a specific directory, use:

```
tar -xf archive.tar -C directory
```

You can also just list the files contained in an archive:

A terminal window with a dark background and light text. The title bar shows 'flavio — fish /Users/flavio — -fish — 55x5'. The prompt is '~'. The command entered is 'tar -tf archive.tar'. The output is 'file1' and 'file2' on separate lines. The prompt is now '~ ' with a green cursor.

```
flavio — fish /Users/flavio — -fish — 55x5
~ tar -tf archive.tar
file1
file2
~
```

`tar` is often used to create a **compressed archive**, gzipping the archive.

This is done using the `z` option:

```
tar -czf archive.tar.gz file1 file2
```

This is just like creating a tar archive, and then running `gzip` on it.

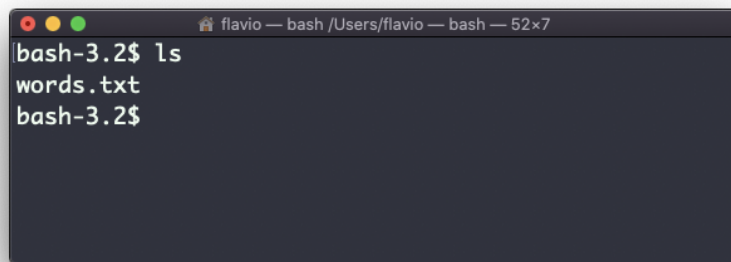
To unarchive a gzipped archive, you can use `gunzip`, or `gzip -d`, and then unarchive it, but `tar -xf` will recognize it's a gzipped archive, and do it for you:

```
tar -xf archive.tar.gz
```

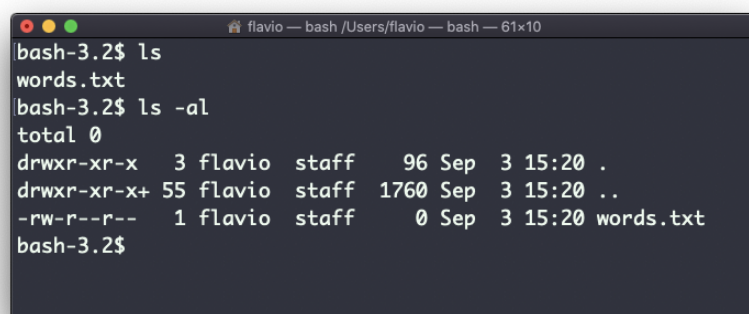
alias

It's common to always run a program with a set of options you like using.

For example, take the `ls` command. By default it prints very little information:

A terminal window titled 'flavio — bash /Users/flavio — bash — 52x7'. The prompt is 'bash-3.2\$'. The user enters 'ls'. The output is 'words.txt'. The prompt returns to 'bash-3.2\$'.

while using the `-al` option it will print something more useful, including the file modification date, the size, the owner, and the permissions, also listing hidden files (files starting with a `.` :

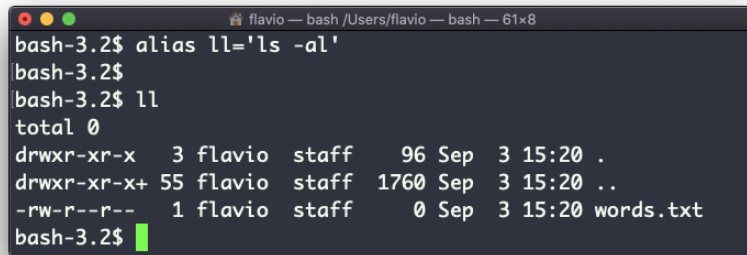
A terminal window titled 'flavio — bash /Users/flavio — bash — 61x10'. The prompt is 'bash-3.2\$'. The user enters 'ls', and the output is 'words.txt'. The prompt returns to 'bash-3.2\$'. The user enters 'ls -al'. The output is:
total 0
drwxr-xr-x 3 flavio staff 96 Sep 3 15:20 .
drwxr-xr-x+ 55 flavio staff 1760 Sep 3 15:20 ..
-rw-r--r-- 1 flavio staff 0 Sep 3 15:20 words.txt
The prompt returns to 'bash-3.2\$'.

You can create a new command, for example I like to call it `ll`, that is an alias to `ls -al`.

You do it in this way:

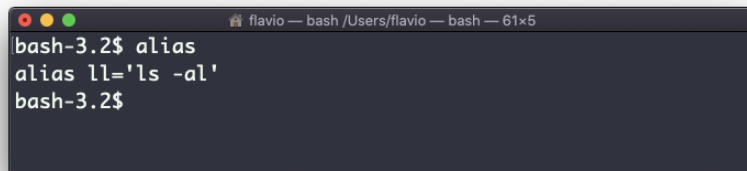
```
alias ll='ls -al'
```

Once you do, you can call `ll` just like it was a regular UNIX command:



```
flavio — bash /Users/flavio — bash — 61x8
bash-3.2$ alias ll='ls -al'
bash-3.2$
bash-3.2$ ll
total 0
drwxr-xr-x  3 flavio  staff   96 Sep  3 15:20 .
drwxr-xr-x+ 55 flavio  staff 1760 Sep  3 15:20 ..
-rw-r--r--  1 flavio  staff   0 Sep  3 15:20 words.txt
bash-3.2$
```

Now calling `alias` without any option will list the aliases defined:



```
flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ alias
alias ll='ls -al'
bash-3.2$
```

The alias will work until the terminal session is closed.

To make it permanent, you need to add it to the shell configuration, which could be `~/.bashrc` or `~/.profile` or `~/.bash_profile` if you use the Bash shell, depending on the use case.

Be careful with quotes if you have variables in the command: using double quotes the variable is resolved at definition time, using single quotes it's resolved at invocation time. Those 2 are different:

```
alias lsthis="ls $PWD"  
alias lscurrent='ls $PWD'
```

\$PWD refers to the current folder the shell is into. If you now navigate away to a new folder, `lscurrent` lists the files in the new folder, `lsthis` still lists the files in the folder you were when you defined the alias.

cat

Similar to `tail` in some way, we have `cat`. Except `cat` can also add content to a file, and this makes it super powerful.

In its simplest usage, `cat` prints a file's content to the standard output:

```
cat file
```

You can print the content of multiple files:

```
cat file1 file2
```

and using the output redirection operator `>` you can concatenate the content of multiple files into a new file:

```
cat file1 file2 > file3
```

Using `>>` you can append the content of multiple files into a new file, creating it if it does not exist:

```
cat file1 file2 >> file3
```

When watching source code files it's great to see the line numbers, and you can have `cat` print them using the `-n` option:

```
cat -n file1
```

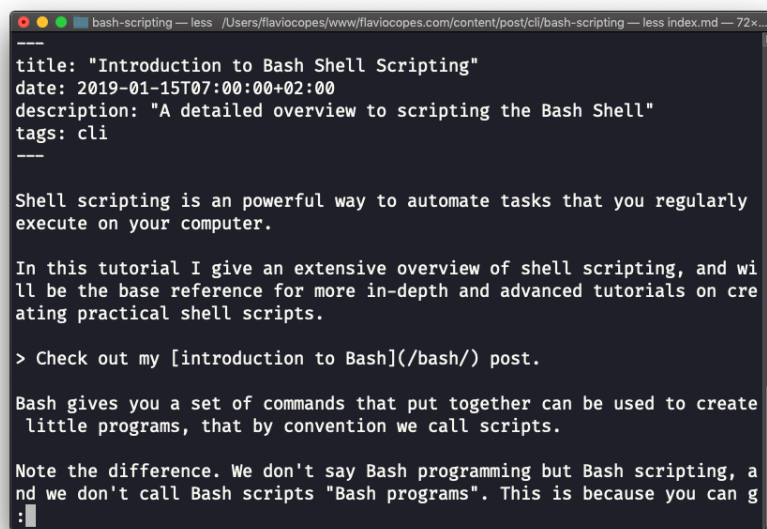
You can only add a number to non-blank lines using `-b`, or you can also remove all the multiple empty lines using `-s`.

`cat` is often used in combination with the pipe operator `|` to feed a file content as input to another command: `cat file1 | anothercommand`.

less

The `less` command is one I use a lot. It shows you the content stored inside a file, in a nice and interactive UI.

Usage: `less <filename>` .

A screenshot of a terminal window with a dark background. The window title is "bash-scripting — less /Users/flaviocopes/www/flaviocopes.com/content/post/cli/bash-scripting — less index.md — 72x...". The content shows the output of the 'less' command for a file named 'index.md'. It displays metadata like title, date, description, and tags, followed by the main text of the article. The text is formatted with code blocks and lists. The cursor is at the bottom of the screen.

```
-----
title: "Introduction to Bash Shell Scripting"
date: 2019-01-15T07:00:00+02:00
description: "A detailed overview to scripting the Bash Shell"
tags: cli
-----

Shell scripting is an powerful way to automate tasks that you regularly
execute on your computer.

In this tutorial I give an extensive overview of shell scripting, and wi
ll be the base reference for more in-depth and advanced tutorials on cre
ating practical shell scripts.

> Check out my [introduction to Bash](/bash/) post.

Bash gives you a set of commands that put together can be used to create
little programs, that by convention we call scripts.

Note the difference. We don't say Bash programming but Bash scripting, a
nd we don't call Bash scripts "Bash programs". This is because you can g
:|
```

Once you are inside a `less` session, you can quit by pressing `q` .

You can navigate the file contents using the `up` and `down` keys, or using the `space bar` and `b` to navigate page by page. You can also jump to the end of the file pressing `G` and jump back to the start pressing `g` .

You can search contents inside the file by pressing `/` and typing a word to search. This searches *forward*. You can search backwards using the `?` symbol and typing a word.

This command just visualises the file's content. You can directly open an editor by pressing `v`. It will use the system editor, which in most cases is `vim`.

Pressing the `F` key enters *follow mode*, or *watch mode*. When the file is changed by someone else, like from another program, you get to see the changes *live*. By default this is not happening, and you only see the file version at the time you opened it. You need to press `ctrl-C` to quit this mode. In this case the behaviour is similar to running the `tail -f <filename>` command.

You can open multiple files, and navigate through them using `:n` (to go to the next file) and `:p` (to go to the previous).

tail

The best use case of tail in my opinion is when called with the `-f` option. It opens the file at the end, and watches for file changes. Any time there is new content in the file, it is printed in the window. This is great for watching log files, for example:

```
tail -f /var/log/system.log
```

To exit, press `ctrl-C` .

You can print the last 10 lines in a file:

```
tail -n 10 <filename>
```

You can print the whole file content starting from a specific line using `+` before the line number:

```
tail -n +10 <filename>
```

`tail` can do much more and as always my advice is to check `man tail` .

WC

The `wc` command gives us useful information about a file or input it receives via pipes.

```
echo test >> test.txt
wc test.txt
1      1      5 test.txt
```

Example via pipes, we can count the output of running the `ls -al` command:

```
ls -al | wc
6      47     284
```

The first column returned is the number of lines. The second is the number of words. The third is the number of bytes.

We can tell it to just count the lines:

```
wc -l test.txt
```

or just the words:

```
wc -w test.txt
```

or just the bytes:

```
wc -c test.txt
```

Bytes in ASCII charsets equate to characters, but with non-ASCII charsets, the number of characters might differ because some characters might take multiple bytes, for example this happens in Unicode.

In this case the `-m` flag will help getting the correct value:

```
wc -m test.txt
```

grep

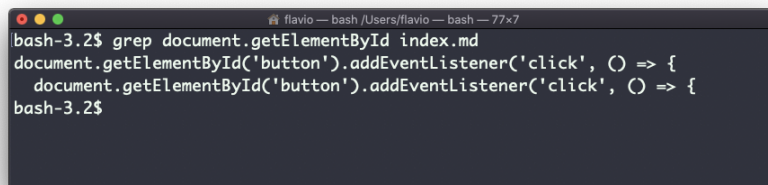
The `grep` command is a very useful tool, that when you master will help you tremendously in your day to day.

If you're wondering, `grep` stands for *global regular expression print*

You can use `grep` to search in files, or combine it with pipes to filter the output of another command.

For example here's how we can find the occurrences of the `document.getElementById` line in the `index.md` file:

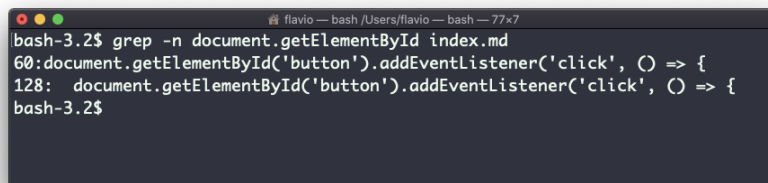
```
grep document.getElementById index.md
```



```
flavio — bash /Users/flavio — bash — 77x7
bash-3.2$ grep document.getElementById index.md
document.getElementById('button').addEventListener('click', () => {
  document.getElementById('button').addEventListener('click', () => {
bash-3.2$
```

Using the `-n` option it will show the line numbers:

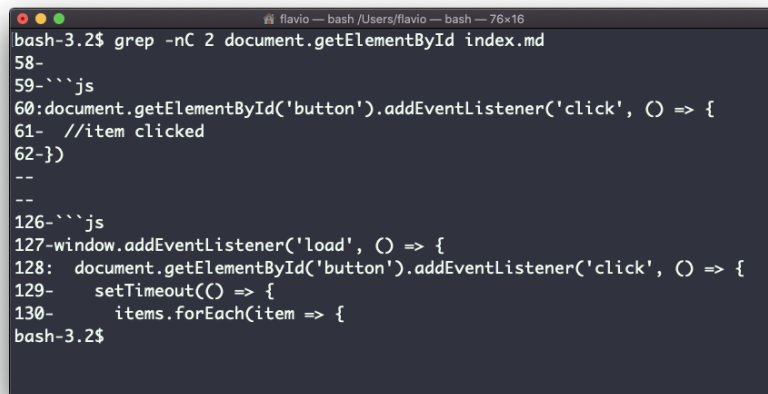
```
grep -n document.getElementById index.md
```



```
flavio — bash /Users/flavio — bash — 77x7
bash-3.2$ grep -n document.getElementById index.md
60:document.getElementById('button').addEventListener('click', () => {
128: document.getElementById('button').addEventListener('click', () => {
bash-3.2$
```


One very useful thing is to tell `grep` to print 2 lines before, and 2 lines after the matched line, to give us more context. That's done using the `-C` option, which accepts a number of lines:

```
grep -nC 2 document.getElementById index.md
```



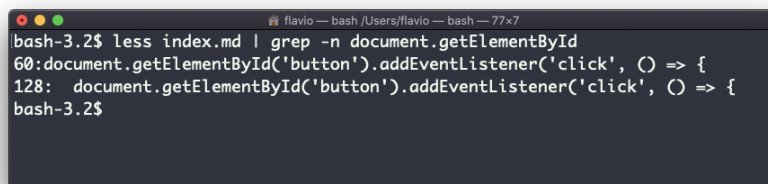
A terminal window showing the command `grep -nC 2 document.getElementById index.md` and its output. The output displays two matches from the file `index.md`, each with two lines of context before and after the match. The first match is on line 60, and the second is on line 128.

```
bash-3.2$ grep -nC 2 document.getElementById index.md
58-
59-```js
60:document.getElementById('button').addEventListener('click', () => {
61-  //item clicked
62-})
--
--
126-```js
127-window.addEventListener('load', () => {
128:  document.getElementById('button').addEventListener('click', () => {
129-    setTimeout(() => {
130-      items.forEach(item => {
bash-3.2$
```

Search is case sensitive by default. Use the `-i` flag to make it insensitive.

As mentioned, you can use `grep` to filter the output of another command. We can replicate the same functionality as above using:

```
less index.md | grep -n document.getElementById
```

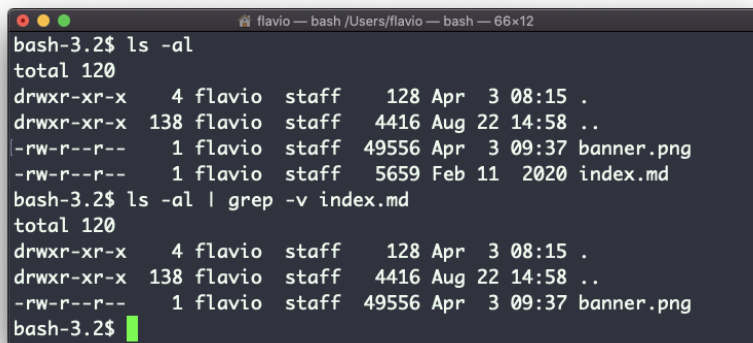


A terminal window showing the command `less index.md | grep -n document.getElementById` and its output. The output shows the same two matches as the previous screenshot, but they are filtered from the `less` command's output. The line numbers 60 and 128 are visible.

```
bash-3.2$ less index.md | grep -n document.getElementById
60:document.getElementById('button').addEventListener('click', () => {
128:  document.getElementById('button').addEventListener('click', () => {
bash-3.2$
```

The search string can be a regular expression, and this makes `grep` very powerful.

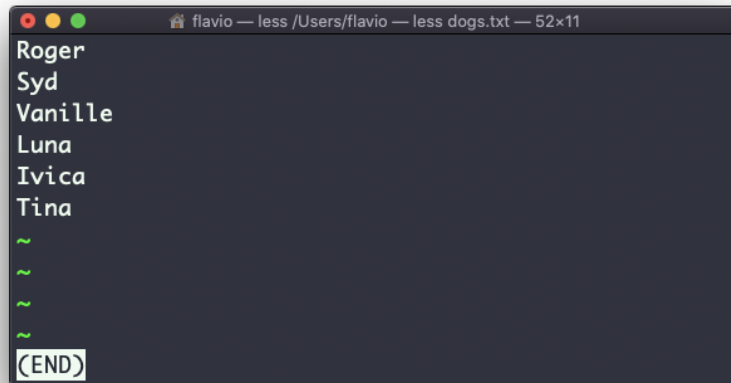
Another thing you might find very useful is to invert the result, excluding the lines that match a particular string, using the `-v` option:



```
flavio — bash /Users/flavio — bash — 66x12
bash-3.2$ ls -al
total 120
drwxr-xr-x  4 flavio  staff   128 Apr  3 08:15 .
drwxr-xr-x 138 flavio  staff  4416 Aug 22 14:58 ..
-rw-r--r--  1 flavio  staff 49556 Apr  3 09:37 banner.png
-rw-r--r--  1 flavio  staff  5659 Feb 11 2020 index.md
bash-3.2$ ls -al | grep -v index.md
total 120
drwxr-xr-x  4 flavio  staff   128 Apr  3 08:15 .
drwxr-xr-x 138 flavio  staff  4416 Aug 22 14:58 ..
-rw-r--r--  1 flavio  staff 49556 Apr  3 09:37 banner.png
bash-3.2$
```

sort

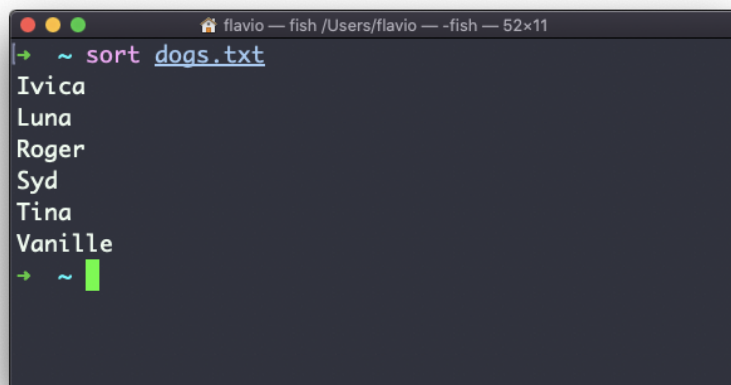
Suppose you have a text file which contains the names of dogs:



```
flavio — less /Users/flavio — less dogs.txt — 52x11
Roger
Syd
Vanille
Luna
Ivica
Tina
~
~
~
~
(END)
```

This list is unordered.

The `sort` command helps us sorting them by name:



```
flavio — fish /Users/flavio — -fish — 52x11
→ ~ sort dogs.txt
Ivica
Luna
Roger
Syd
Tina
Vanille
→ ~
```

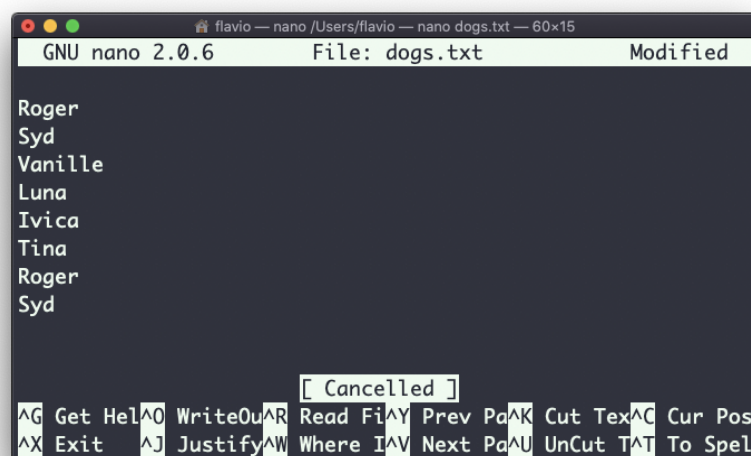
Use the `r` option to reverse the order:



```
flavio — fish /Users/flavio — -fish — 51x9
~ sort -r dogs.txt
Vanille
Tina
Syd
Roger
Luna
Ivica
~
```

Sorting by default is case sensitive, and alphabetic. Use the `--ignore-case` option to sort case insensitive, and the `-n` option to sort using a numeric order.

If the file contains duplicate lines:

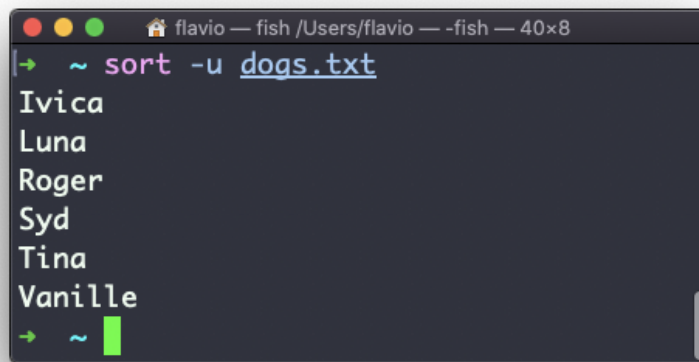


```
flavio — nano /Users/flavio — nano dogs.txt — 60x15
GNU nano 2.0.6      File: dogs.txt      Modified

Roger
Syd
Vanille
Luna
Ivica
Tina
Roger
Syd

[ Cancelled ]
^G Get Hel^O WriteOu^R Read Fi^Y Prev Pa^K Cut Tex^C Cur Pos
^X Exit   ^J Justify^W Where I^V Next Pa^U UnCut T^T To Spel
```

You can use the `-u` option to remove them:

A terminal window with a dark background. The title bar shows 'flavio — fish /Users/flavio — -fish — 40x8'. The prompt is '~'. The command 'sort -u dogs.txt' has been entered. The output is a list of names: Ivica, Luna, Roger, Syd, Tina, and Vanille. The prompt is now '~' with a green cursor.

`sort` does not just work on files, as many UNIX commands it also works with pipes, so you can use on the output of another command, for example you can order the files returned by `ls` with:

```
ls | sort
```

`sort` is very powerful and has lots more options, which you can explore calling `man sort`.

```
# flaviu — man /Users/flaviu — less - man sort — 115x55
SORT(1) BSD General Commands Manual SORT(1)

NAME
  sort -- sort or merge records (lines) of text and binary files

SYNOPSIS
  sort [-bcCdFghiRmMnrsuVz] [-k field1[,field2]] [-S memsize] [-T dir] [-t char] [-o output]
      [file ...]
  sort --help
  sort --version

DESCRIPTION
  The sort utility sorts text and binary files by lines. A line is a record separated from the sub-
  sequent record by a newline (default) or NUL '\0' character (-z option). A record can contain any
  printable or unprintable characters. Comparisons are based on one or more sort keys extracted
  from each line of input, and are performed lexicographically, according to the current locale's
  collating rules and the specified command-line options that can tune the actual sorting behavior.
  By default, if keys are not given, sort uses entire lines for comparison.

  The command line options are as follows:

  -c, --check, -C, --check=silentquiet
      Check that the single input file is sorted. If the file is not sorted, sort produces the
      appropriate error messages and exits with code 1, otherwise returns 0. If -C or
      --check=silent is specified, sort produces no output. This is a "silent" version of -c.

  -m, --merge
      Merge only. The input files are assumed to be pre-sorted. If they are not sorted the
      output order is undefined.

  -o output, --output=output
      Print the output to the output file instead of the standard output.

  -S size, --buffer-size=size
      Use size for the maximum size of the memory buffer. Size modifiers %,b,K,M,G,T,P,E,Z,Y
      can be used. If a memory limit is not explicitly specified, sort takes up to about 90% of
      available memory. If the file size is too big to fit into the memory buffer, the tempo-
      rary disk files are used to perform the sorting.

  -T dir, --temporary-directory=dir
      Store temporary files in the directory dir. The default path is the value of the environ-
      ment variable TMPDIR or /var/tmp if TMPDIR is not defined.

  -u, --unique
      Unique keys. Suppress all lines that have a key that is equal to an already processed
      one. This option, similarly to -s, implies a stable sort. If used with -c or -C, sort
      also checks that there are no lines with duplicate keys.

  -s
      Stable sort. This option maintains the original record order of records that have an
      equal key. This is a non-standard feature, but it is widely accepted and used.

  --version
      Print the version and silently exits.
```

uniq

`uniq` is a command useful to sort lines of text.

You can get those lines from a file, or using pipes from the output of another command:

```
uniq dogs.txt  
  
ls | uniq
```

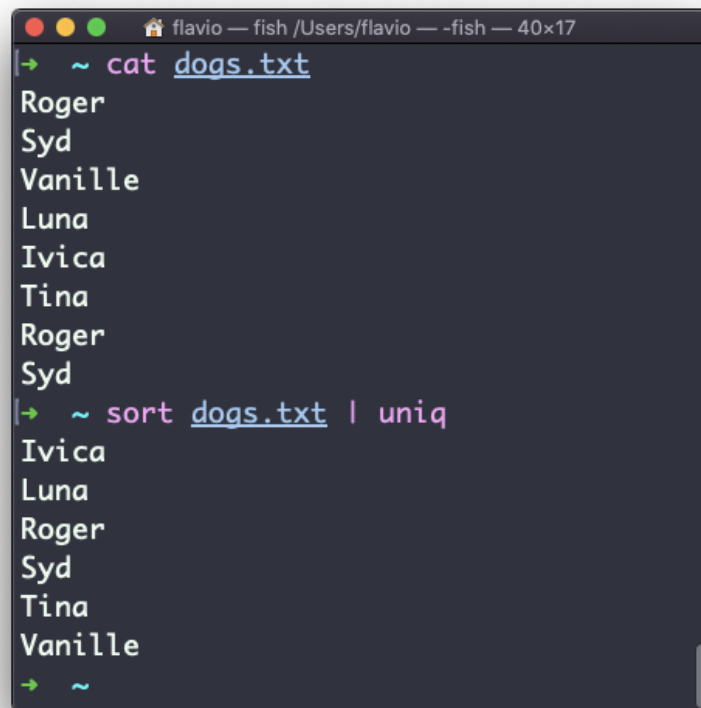
You need to consider this key thing: `uniq` will only detect adjacent duplicate lines.

This implies that you will most likely use it along with `sort` :

```
sort dogs.txt | uniq
```

The `sort` command has its own way to remove duplicates with the `-u` (*unique*) option. But `uniq` has more power.

By default it removes duplicate lines:



```
flavio — fish /Users/flavio — -fish — 40x17
~ cat dogs.txt
Roger
Syd
Vanille
Luna
Ivica
Tina
Roger
Syd
~ sort dogs.txt | uniq
Ivica
Luna
Roger
Syd
Tina
Vanille
~
```

You can tell it to only display duplicate lines, for example, with the `-d` option:

```
sort dogs.txt | uniq -d
```



```
flavio — fish /Users/flavio — -fish — 40x13
→ ~ cat dogs.txt
Roger
Syd
Vanille
Luna
Ivica
Tina
Roger
Syd
→ ~ sort dogs.txt | uniq -d
Roger
Syd
→ ~
```

You can use the `-u` option to only display non-duplicate lines:

```
flavio — fish /Users/flavio — -fish — 40x15
→ ~ cat dogs.txt
Roger
Syd
Vanille
Luna
Ivica
Tina
Roger
Syd
→ ~ sort dogs.txt | uniq -u
Ivica
Luna
Tina
Vanille
→ ~
```

You can count the occurrences of each line with the `-c` option:

```
flavio — fish /Users/flavio — -fish — 40x17
→ ~ cat dogs.txt
Roger
Syd
Vanille
Luna
Ivica
Tina
Roger
Syd
→ ~ sort dogs.txt | uniq -c
  1 Ivica
  1 Luna
  2 Roger
  2 Syd
  1 Tina
  1 Vanille
→ ~
```

Use the special combination:

```
sort dogs.txt | uniq -c | sort -nr
```

to then sort those lines by most frequent:

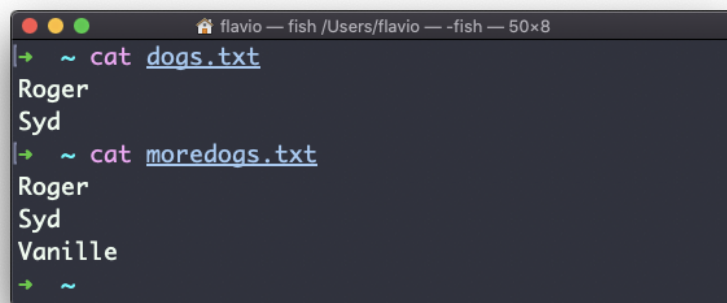
```
flavio — fish /Users/flavio — -fish — 44x8
→ ~ sort dogs.txt | uniq -c | sort -nr
  2 Syd
  2 Roger
  1 Vanille
  1 Tina
  1 Luna
  1 Ivica
→ ~
```

diff

`diff` is a handy command. Suppose you have 2 files, which contain almost the same information, but you can't find the difference between the two.

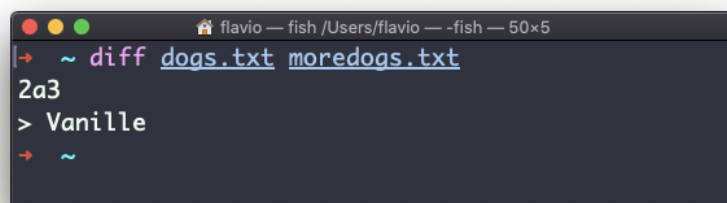
`diff` will process the files and will tell you what's the difference.

Suppose you have 2 files: `dogs.txt` and `moredogs.txt`. The difference is that `moredogs.txt` contains one more dog name:



```
flavio — fish /Users/flavio — -fish — 50x8
→ ~ cat dogs.txt
Roger
Syd
→ ~ cat moredogs.txt
Roger
Syd
Vanille
→ ~
```

`diff dogs.txt moredogs.txt` will tell you the second file has one more line, line 3 with the line `Vanille`:



```
flavio — fish /Users/flavio — -fish — 50x5
→ ~ diff dogs.txt moredogs.txt
2a3
> Vanille
→ ~
```

If you invert the order of the files, it will tell you that the second file is missing line 3, whose content is Vanille :

```
flavio — fish /Users/flavio — -fish — 50x5
~ diff moredogs.txt dogs.txt
3d2
< Vanille
~
```

Using the `-y` option will compare the 2 files line by line:

```
flavio — fish /Users/flavio — -fish — 86x9
~ diff -y dogs.txt moredogs.txt
Roger                                Roger
Syd                                Syd
> Vanille
~
```

The `-u` option however will be more familiar to you, because that's the same used by the Git version control system to display differences between versions:

```
flavio — fish /Users/flavio — -fish — 67x8
~ diff -u dogs.txt moredogs.txt
--- dogs.txt      2020-09-07 08:54:56.000000000 +0200
+++ moredogs.txt  2020-09-07 08:55:09.000000000 +0200
@@ -1,2 +1,3 @@
 Roger
 Syd
+Vanille
~
```

Comparing directories works in the same way. You must use the `-r` option to compare recursively (going into subdirectories):

```
testing — fish /Users/flavio/testing — -fish — 68x13
→ testing ls dir1
dogs.txt
→ testing ls dir2
dogs.txt
→ testing diff -u dir1 dir2
diff -u dir1/dogs.txt dir2/dogs.txt
--- dir1/dogs.txt      2020-09-07 08:54:56.000000000 +0200
+++ dir2/dogs.txt      2020-09-07 08:55:09.000000000 +0200
@@ -1,2 +1,3 @@
   Roger
   Syd
+Vanille
→ testing
```

In case you're interested in which files differ, rather than the content, use the `r` and `q` options:

```
testing — fish /Users/flavio/testing — -fish — 68x5
→ testing diff -rq dir1 dir2
Files dir1/dogs.txt and dir2/dogs.txt differ
→ testing
```

There are many more options you can explore in the man page running `man diff`:

```
DIFF(1)                                User Commands                                DIFF(1)

NAME
    diff - compare files line by line

SYNOPSIS
    diff [OPTION]... FILES

DESCRIPTION
    Compare files line by line.

    -i --ignore-case
        Ignore case differences in file contents.

    --ignore-file-name-case
        Ignore case when comparing file names.

    --no-ignore-file-name-case
        Consider case when comparing file names.

    -E --ignore-tab-expansion
        Ignore changes due to tab expansion.

    -b --ignore-space-change
        Ignore changes in the amount of white space.

    -w --ignore-all-space
        Ignore all white space.

    -B --ignore-blank-lines
        Ignore changes whose lines are all blank.

    -I RE --ignore-matching-lines=RE
        Ignore changes whose lines all match RE.

    --strip-trailing-cr
        Strip trailing carriage return on input.

    -a --text
        Treat all files as text.

    -C NUM --context[=NUM]
        Output NUM (default 3) lines of copied context.

    -u -U NUM --unified[=NUM]
        Output NUM (default 3) lines of unified context.

    --label LABEL
        Use LABEL instead of file name.

    -p --show-c-function
        Show which C function each change is in.

    -F RE --show-function-line=RE
```

echo

The `echo` command does one simple job: it prints to the output the argument passed to it.

This example:

```
echo "hello"
```

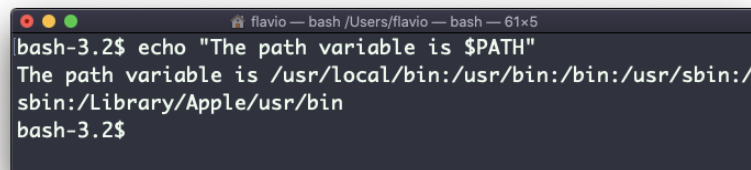
will print `hello` to the terminal.

We can append the output to a file:

```
echo "hello" >> output.txt
```

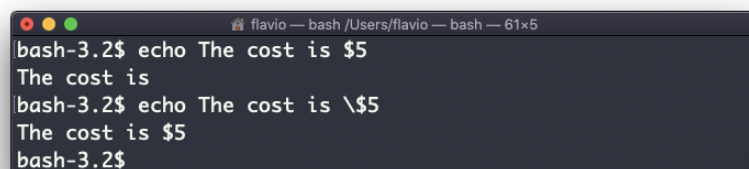
We can interpolate environment variables:

```
echo "The path variable is $PATH"
```



```
flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ echo "The path variable is $PATH"
The path variable is /usr/local/bin:/usr/bin:/bin:/usr/sbin:/
sbin:/Library/Apple/usr/bin
bash-3.2$
```

Beware that special characters need to be escaped with a backslash `\` . `$` for example:



```
flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ echo The cost is $5
The cost is
bash-3.2$ echo The cost is \$5
The cost is $5
bash-3.2$
```

This is just the start. We can do some nice things when it comes to interacting with the shell features.

We can echo the files in the current folder:

```
echo *
```

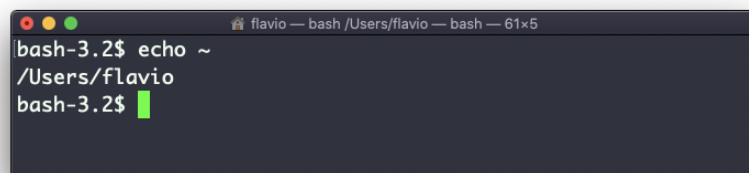
We can echo the files in the current folder that start with the letter `o` :

```
echo o*
```

Any valid Bash (or any shell you are using) command and feature can be used here.

You can print your home folder path:

```
echo ~
```

A screenshot of a terminal window. The title bar shows 'flavio — bash /Users/flavio — bash — 61x5'. The terminal content shows 'bash-3.2\$ echo ~' followed by the output '/Users/flavio' on the next line. The prompt 'bash-3.2\$' is followed by a green cursor bar.

You can also execute commands, and print the result to the standard output (or to file, as you saw):

```
echo $(ls -al)
```



```

flavio — bash /Users/flavio — bash — 61x6
bash-3.2$ echo $(ls -al)
total 8 drwxr-xr-x 4 flavio staff 128 Sep 3 15:43 . drwxr-xr-
x+ 55 flavio staff 1760 Sep 3 15:20 .. -rw-r--r-- 1 flavio st
aff 6 Sep 3 15:43 output.txt -rw-r--r-- 1 flavio staff 0 Sep
3 15:20 words.txt
bash-3.2$

```

Note that whitespace is not preserved by default. You need to wrap the command in double quotes to do so:

```

flavio — bash /Users/flavio — bash — 61x7
bash-3.2$ echo "$(ls -al)"
total 8
drwxr-xr-x  4 flavio  staff   128 Sep  3 15:43 .
drwxr-xr-x+ 55 flavio  staff  1760 Sep  3 15:20 ..
-rw-r--r--   1 flavio  staff    6 Sep  3 15:43 output.txt
-rw-r--r--   1 flavio  staff    0 Sep  3 15:20 words.txt
bash-3.2$

```

You can generate a list of strings, for example ranges:

```
echo {1..5}
```

```

flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ echo {1..5}
1 2 3 4 5
bash-3.2$

```

chown

Every file/directory in an Operating System like Linux or macOS (and every UNIX systems in general) has an **owner**.

The owner of a file can do everything with it. It can decide the fate of that file.

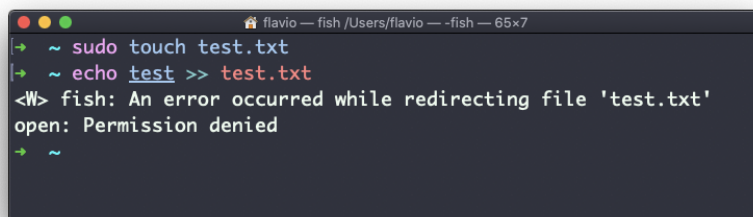
The owner (and the `root` user) can change the owner to another user, too, using the `chown` command:

```
chown <owner> <file>
```

Like this:

```
chown flavio test.txt
```

For example if you have a file that's owned by `root`, you can't write to it as another user:

A terminal window titled 'flavio — fish /Users/flavio — -fish — 65x7' showing a sequence of commands and an error. The user runs 'sudo touch test.txt', then 'echo test >> test.txt'. The second command fails with the message: '<W> fish: An error occurred while redirecting file 'test.txt' open: Permission denied'. The prompt returns to '~'.

```
flavio — fish /Users/flavio — -fish — 65x7
~$ sudo touch test.txt
~$ echo test >> test.txt
<W> fish: An error occurred while redirecting file 'test.txt'
open: Permission denied
~$
```

You can use `chown` to transfer the ownership to you:

A terminal window with a dark background and light text. The title bar shows 'flavio — fish /Users/flavio — -fish — 65x7'. The commands and output are:

```
→ ~ sudo touch test.txt
→ ~ echo test >> test.txt
<W> fish: An error occurred while redirecting file 'test.txt'
open: Permission denied
→ ~ sudo chown flavio test.txt
→ ~ echo test >> test.txt
→ ~
```

It's rather common to have the need to change the ownership of a directory, and recursively all the files contained, plus all the subdirectories and the files contained in them, too.

You can do so using the `-R` flag:

```
chown -R <owner> <file>
```

Files/directories don't just have an owner, they also have a **group**. Through this command you can change that simultaneously while you change the owner:

```
chown <owner>:<group> <file>
```

Example:

```
chown flavio:users test.txt
```

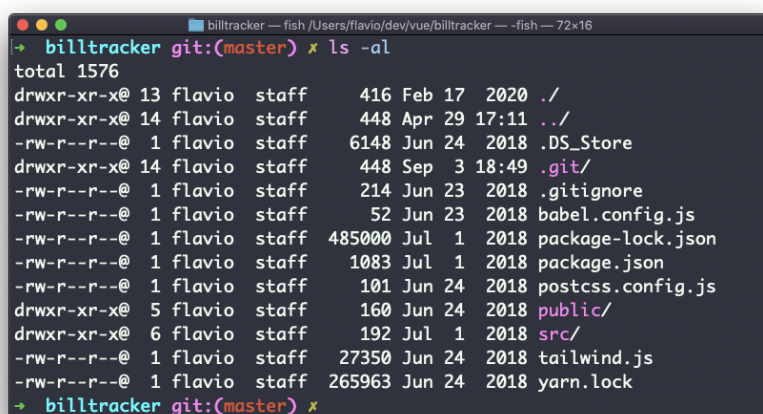
You can also just change the group of a file using the `chgrp` command:

```
chgrp <group> <filename>
```

chmod

Every file in the Linux / macOS Operating Systems (and UNIX systems in general) has 3 permissions: Read, write, execute.

Go into a folder, and run the `ls -al` command.



```
billtracker git:(master) x ls -al
total 1576
drwxr-xr-x@ 13 flavio  staff   416 Feb 17  2020 ./
drwxr-xr-x@ 14 flavio  staff   448 Apr 29 17:11 ../
-rw-r--r--@ 1 flavio  staff  6148 Jun 24  2018 .DS_Store
drwxr-xr-x@ 14 flavio  staff   448 Sep  3 18:49 .git/
-rw-r--r--@ 1 flavio  staff   214 Jun 23  2018 .gitignore
-rw-r--r--@ 1 flavio  staff    52 Jun 23  2018 babel.config.js
-rw-r--r--@ 1 flavio  staff 485000 Jul  1  2018 package-lock.json
-rw-r--r--@ 1 flavio  staff  1083 Jul  1  2018 package.json
-rw-r--r--@ 1 flavio  staff   101 Jun 24  2018 postcss.config.js
drwxr-xr-x@ 5 flavio  staff   160 Jun 24  2018 public/
drwxr-xr-x@ 6 flavio  staff   192 Jul  1  2018 src/
-rw-r--r--@ 1 flavio  staff 27350 Jun 24  2018 tailwind.js
-rw-r--r--@ 1 flavio  staff 265963 Jun 24  2018 yarn.lock
billtracker git:(master) x
```

The weird strings you see on each file line, like `drwxr-xr-x`, define the permissions of the file or folder.

Let's dissect it.

The first letter indicates the type of file:

- `-` means it's a normal file
- `d` means it's a directory
- `l` means it's a link

Then you have 3 sets of values:

- The first set represents the permissions of the **owner** of the file
- The second set represents the permissions of the members of the **group** the file is associated to

- The third set represents the permissions of the **everyone else**

Those sets are composed by 3 values. `rwX` means that specific *persona* has read, write and execution access. Anything that is removed is swapped with a `-`, which lets you form various combinations of values and relative permissions: `rw-`, `r--`, `r-x`, and so on.

You can change the permissions given to a file using the `chmod` command.

`chmod` can be used in 2 ways. The first is using symbolic arguments, the second is using numeric arguments. Let's start with symbols first, which is more intuitive.

You type `chmod` followed by a space, and a letter:

- `a` stands for *all*
- `u` stands for *user*
- `g` stands for *group*
- `o` stands for *others*

Then you type either `+` or `-` to add a permission, or to remove it. Then you enter one or more permissions symbols (`r`, `w`, `x`).

All followed by the file or folder name.

Here are some examples:

```
chmod a+r filename #everyone can now read
chmod a+rw filename #everyone can now read and write
chmod o-rwx filename #others (not the owner, not in
```

You can apply the same permissions to multiple personas by adding multiple letters before the `+` `/` `-` `:`

```
chmod og-r filename #other and group can't read any
```

In case you are editing a folder, you can apply the permissions to every file contained in that folder using the `-r` (recursive) flag.

Numeric arguments are faster but I find them hard to remember when you are not using them day to day. You use a digit that represents the permissions of the persona. This number value can be a maximum of 7, and it's calculated in this way:

- `1` if has execution permission
- `2` if has write permission
- `4` if has read permission

This gives us 4 combinations:

- `0` no permissions
- `1` can execute
- `2` can write
- `3` can write, execute
- `4` can read
- `5` can read, execute
- `6` can read, write
- `7` can read, write and execute

We use them in pairs of 3, to set the permissions of all the 3 groups altogether:

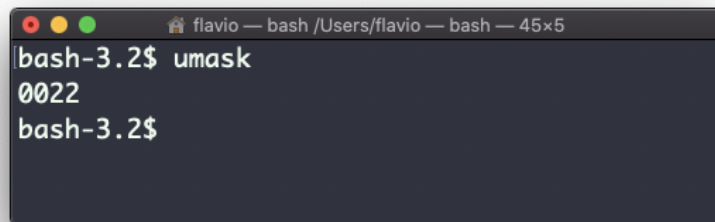
```
chmod 777 filename  
chmod 755 filename  
chmod 644 filename
```

umask

When you create a file, you don't have to decide permissions up front. Permissions have defaults.

Those defaults can be controlled and modified using the `umask` command.

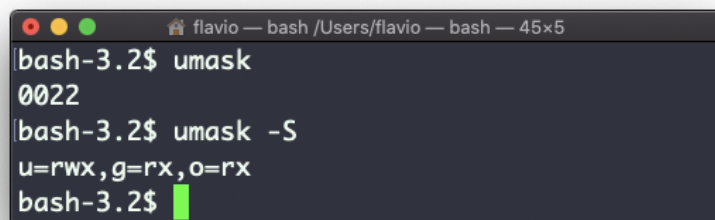
Typing `umask` with no arguments will show you the current umask, in this case `0022` :

A terminal window titled 'flavio — bash /Users/flavio — bash — 45x5'. The prompt is 'bash-3.2\$'. The user enters 'umask' and the output is '0022'. The prompt returns to 'bash-3.2\$'.

What does `0022` mean? That's an octal value that represent the permissions.

Another common value is `0002` .

Use `umask -S` to see a human-readable notation:

A terminal window titled 'flavio — bash /Users/flavio — bash — 45x5'. The prompt is 'bash-3.2\$'. The user enters 'umask' and the output is '0022'. The user then enters 'umask -S' and the output is 'u=rwx,g=rx,o=rx'. The prompt returns to 'bash-3.2\$'.

In this case, the user (`u`), owner of the file, has read, write and execution permissions on files.

Other users belonging to the same group (`g`) have read and execution permission, same as all the other users (`o`).

In the numeric notation, we typically change the last 3 digits.

Here's a list that gives a meaning to the number:

- `0` read, write, execute
- `1` read and write
- `2` read and execute
- `3` read only
- `4` write and execute
- `5` write only
- `6` execute only
- `7` no permissions

Note that this numeric notation differs from the one we use in `chmod` .

We can set a new value for the mask setting the value in numeric format:

```
umask 002
```

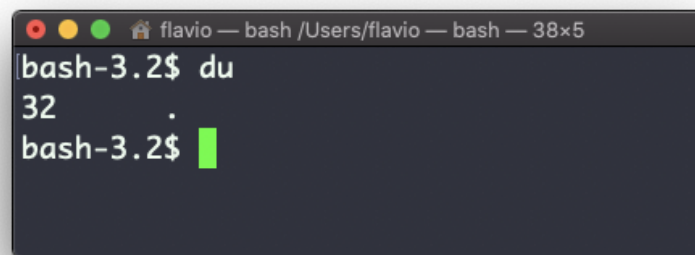
or you can change a specific role's permission:

```
umask g+r
```

du

The `du` command will calculate the size of a directory as a whole:

```
du
```

A terminal window titled 'flavio — bash /Users/flavio — bash — 38x5' showing the command 'du' being executed. The output is '32' followed by a newline and a dot '.'. The prompt 'bash-3.2\$' is visible on the first line, and the second line shows the prompt again with a green cursor.

```
bash-3.2$ du
32
.
bash-3.2$
```

The `32` number here is a value expressed in bytes.

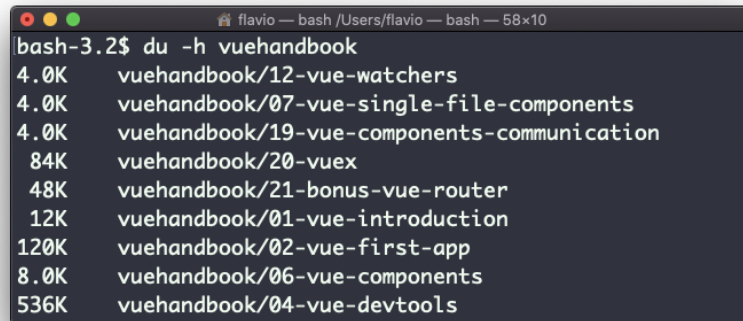
Running `du *` will calculate the size of each file individually:

A terminal window titled 'flavio — bash /Users/flavio — bash — 38x6' showing the command 'du *' being executed. The output lists four files with their sizes in bytes: '8 Card.vue', '8 CardList.vue', '8 Form.vue', and '8 HelloWorld.vue'. The prompt 'bash-3.2\$' is visible on the first line, and the last line shows the prompt again with a green cursor.

```
bash-3.2$ du *
8      Card.vue
8      CardList.vue
8      Form.vue
8      HelloWorld.vue
bash-3.2$
```

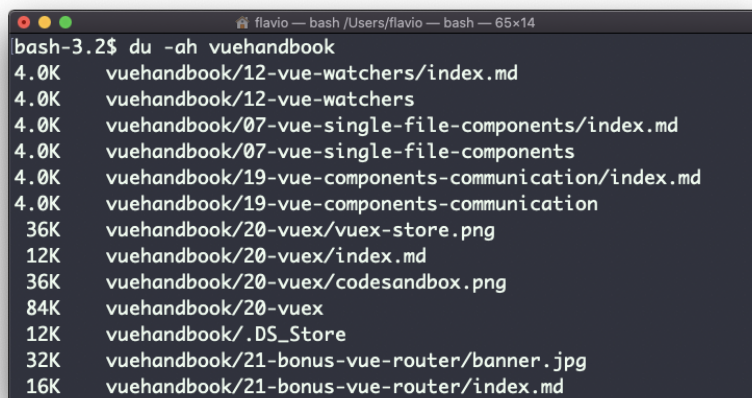
You can set `du` to display values in MegaBytes using `du -m`, and GigaBytes using `du -g`.

The `-h` option will show a human-readable notation for sizes, adapting to the size:



```
flavio — bash /Users/flavio — bash — 58x10
bash-3.2$ du -h vuehandbook
4.0K  vuehandbook/12-vue-watchers
4.0K  vuehandbook/07-vue-single-file-components
4.0K  vuehandbook/19-vue-components-communication
84K   vuehandbook/20-vuex
48K   vuehandbook/21-bonus-vue-router
12K   vuehandbook/01-vue-introduction
120K  vuehandbook/02-vue-first-app
8.0K  vuehandbook/06-vue-components
536K  vuehandbook/04-vue-devtools
```

Adding the `-a` option will print the size of each file in the directories, too:



```
flavio — bash /Users/flavio — bash — 65x14
bash-3.2$ du -ah vuehandbook
4.0K  vuehandbook/12-vue-watchers/index.md
4.0K  vuehandbook/12-vue-watchers
4.0K  vuehandbook/07-vue-single-file-components/index.md
4.0K  vuehandbook/07-vue-single-file-components
4.0K  vuehandbook/19-vue-components-communication/index.md
4.0K  vuehandbook/19-vue-components-communication
36K   vuehandbook/20-vuex/vuex-store.png
12K   vuehandbook/20-vuex/index.md
36K   vuehandbook/20-vuex/codesandbox.png
84K   vuehandbook/20-vuex
12K   vuehandbook/.DS_Store
32K   vuehandbook/21-bonus-vue-router/banner.jpg
16K   vuehandbook/21-bonus-vue-router/index.md
```

A handy thing is to sort the directories by size:

```
du -h <directory> | sort -nr
```

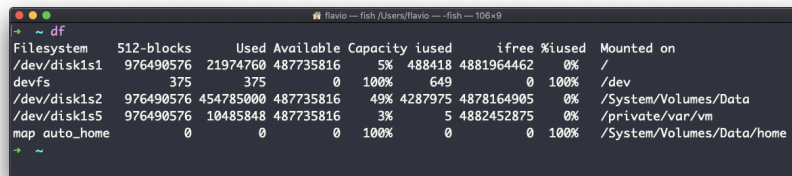
and then piping to `head` to only get the first 10 results:

```
flavio — bash /Users/flavio — bash — 63x13
bash-3.2$ du -h vuehandbook | sort -nr | head
932K  vuehandbook/05-vue-vscode
636K  vuehandbook/.git/objects/75
544K  vuehandbook/03-vue-cli
536K  vuehandbook/04-vue-devtools
120K  vuehandbook/02-vue-first-app
88K   vuehandbook/.git/objects/pack
88K   vuehandbook/.git/objects/b0
84K   vuehandbook/20-vuex
76K   vuehandbook/.git/objects/6f
64K   vuehandbook/.git/objects/41
bash-3.2$
```

df

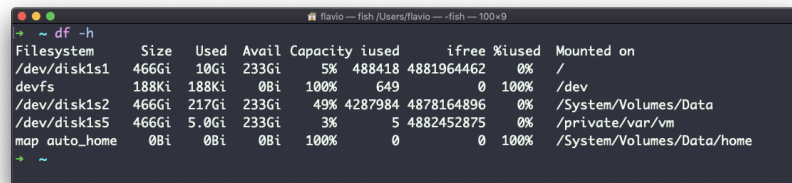
The `df` command is used to get disk usage information.

Its basic form will print information about the volumes mounted:



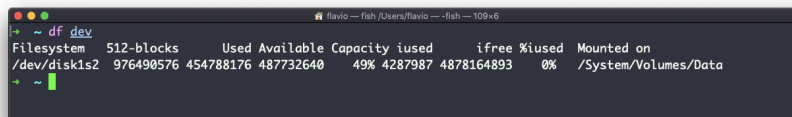
```
flavio ~ fish /Users/flavio — fish — 100x9
~ df
Filesystem      512-blocks    Used Available Capacity iused      ifree %iused  Mounted on
/dev/disk1s1    976490576    21974760 487735816      5% 488418 4881964462      0% /
devfs           375          375      0      100%    649      0      100% /dev
/dev/disk1s2    976490576 454785000 487735816     49% 4287975 4878164905      0% /System/Volumes/Data
/dev/disk1s5    976490576 10485848 487735816      3%    5 4882452875      0% /private/var/vm
map auto_home      0          0      0      100%      0      0      100% /System/Volumes/Data/home
```

Using the `-h` option (`df -h`) will show those values in a human-readable format:



```
flavio ~ fish /Users/flavio — fish — 100x9
~ df -h
Filesystem      Size  Used Avail Capacity iused      ifree %iused  Mounted on
/dev/disk1s1    466Gi  10Gi  233Gi      5% 488418 4881964462      0% /
devfs           188Ki  188Ki   0Bi     100%    649      0      100% /dev
/dev/disk1s2    466Gi  217Gi  233Gi     49% 4287984 4878164896      0% /System/Volumes/Data
/dev/disk1s5    466Gi   5.0Gi  233Gi      3%    5 4882452875      0% /private/var/vm
map auto_home    0Bi    0Bi   0Bi     100%      0      0      100% /System/Volumes/Data/home
```

You can also specify a file or directory name to get information about the specific volume it lives on:



```
flavio ~ fish /Users/flavio — fish — 100x6
~ df dev
Filesystem      512-blocks    Used Available Capacity iused      ifree %iused  Mounted on
/dev/disk1s2    976490576 454788176 487732640     49% 4287987 4878164893      0% /System/Volumes/Data
```

basename

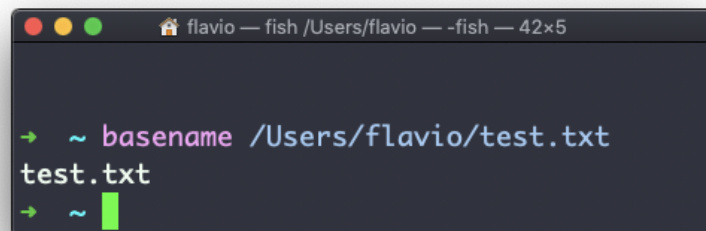
Suppose you have a path to a file, for example

```
/Users/flavio/test.txt .
```

Running

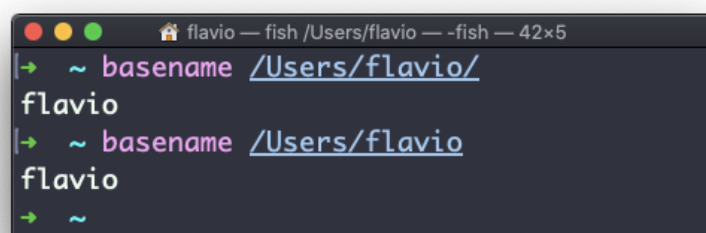
```
basename /Users/flavio/test.txt
```

will return the `test.txt` string:



```
flavio — fish /Users/flavio — -fish — 42x5
→ ~ basename /Users/flavio/test.txt
test.txt
→ ~
```

If you run `basename` on a path string that points to a directory, you will get the last segment of the path. In this example, `/Users/flavio` is a directory:



```
flavio — fish /Users/flavio — -fish — 42x5
→ ~ basename /Users/flavio/
flavio
→ ~ basename /Users/flavio
flavio
→ ~
```

dirname

Suppose you have a path to a file, for example

```
/Users/flavio/test.txt .
```

Running

```
dirname /Users/flavio/test.txt
```

will return the `/Users/flavio` string:

A terminal window with a dark background and light-colored text. The window title bar shows 'flavio — fish /Users/flavio — -fish — 44x5'. The prompt is '~'. The command 'dirname /Users/flavio/test.txt' is entered and executed. The output is '/Users/flavio'. The prompt is '~' again.

```
flavio — fish /Users/flavio — -fish — 44x5
~ $ dirname /Users/flavio/test.txt
/Users/flavio
~ $
```

ps

Your computer is running, at all times, tons of different processes.

You can inspect them all using the `ps` command:

A terminal window titled 'flavio — fish /Users/flavio — -fish — 63x12' showing the output of the 'ps' command. The output is a table with columns: PID, TTY, TIME, and CMD. The data rows are:

PID	TTY	TIME	CMD
59474	ttys000	0:00.13	/usr/local/bin/fish -l
941	ttys002	0:00.61	/usr/local/bin/fish -l
71366	ttys004	0:01.08	-fish
4216	ttys009	0:00.31	/usr/local/bin/fish -l
68714	ttys009	0:20.82	hugo serve
5088	ttys013	0:02.89	-fish

This is the list of user-initiated processes currently running in the current session.

Here I have a few `fish` shell instances, mostly opened by VS Code inside the editor, and an instances of Hugo running the development preview of a site.

Those are just the commands assigned to the current user. To list **all** processes we need to pass some options to `ps` .

The most common I use is `ps ax` :


```
flavio ~ — fish — 86x24
~ ps ax
PID  TT  STAT      TIME COMMAND
  1  ??  Ss      43:24.80 /sbin/launchd
 92  ??  Ss      2:03.80 /usr/sbin/syslogd
 93  ??  Ss      4:56.03 /usr/libexec/UserEventAgent (System)
 96  ??  Ss      0:18.74 /System/Library/PrivateFrameworks/Uninstall.framework/Reso
 97  ??  Ss      1:36.94 /usr/libexec/kextd
 98  ??  Ss     12:31.61 /System/Library/Frameworks/CoreServices.framework/Versions
 99  ??  Ss      0:21.48 /System/Library/PrivateFrameworks/MediaRemote.framework/Su
102  ??  Ss     22:56.23 /usr/sbin/systemstats --daemon
103  ??  Ss      2:25.78 /usr/libexec/configd
105  ??  Ss      5:32.22 /System/Library/CoreServices/powerd.bundle/powerd
109  ??  Ss      9:48.23 /usr/libexec/logd
110  ??  Ss      0:01.46 /usr/libexec/keybagd -t 15
113  ??  Ss      0:31.41 /usr/libexec/watchdogd
117  ??  Ss     44:38.55 /System/Library/Frameworks/CoreServices.framework/Framework
118  ??  Ss      0:00.55 /System/Library/CoreServices/iconservicesd
119  ??  Ss      0:49.70 /usr/libexec/diskarbitrationd
123  ??  Ss      1:09.30 /usr/libexec/coreduetd
126  ??  Ss      7:31.22 /usr/libexec/opendirectoryd
127  ??  Ss      0:51.05 /System/Library/PrivateFrameworks/ApplePushService.framework
128  ??  Ss      0:00.54 /Library/PrivilegedHelperTools/com.docker.vmneta
129  ??  Ss     19:37.63 /System/Library/CoreServices/launchservicesd
130  ??  Ss      0:14.36 /usr/libexec/timed
```

The `a` option is used to also list other users processes, not just our own. `x` shows processes not linked to any terminal (not initiated by users through a terminal).

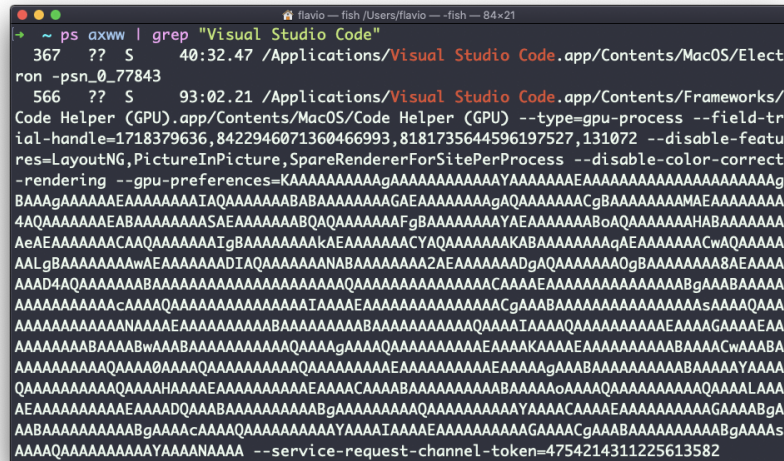
As you can see, the longer commands are cut. Use the command `ps axww` to continue the command listing on a new line instead of cutting it:

```
flavio ~ — fish — 84x21
~ ps axww
PID  TT  STAT      TIME COMMAND
  1  ??  Ss      43:25.81 /sbin/launchd
 92  ??  Ss      2:03.82 /usr/sbin/syslogd
 93  ??  Ss      4:56.05 /usr/libexec/UserEventAgent (System)
 96  ??  Ss      0:18.74 /System/Library/PrivateFrameworks/Uninstall.framework/Res
sources/uninstalld
 97  ??  Ss      1:36.94 /usr/libexec/kextd
 98  ??  Ss     12:31.92 /System/Library/Frameworks/CoreServices.framework/Version
s/A/Frameworks/FSEvents.framework/Versions/A/Support/fsevents
 99  ??  Ss      0:21.48 /System/Library/PrivateFrameworks/MediaRemote.framework/S
upport/mediaremoted
102  ??  Ss     22:56.23 /usr/sbin/systemstats --daemon
103  ??  Ss      2:25.80 /usr/libexec/configd
105  ??  Ss      5:32.38 /System/Library/CoreServices/powerd.bundle/powerd
109  ??  Ss      9:48.48 /usr/libexec/logd
110  ??  Ss      0:01.46 /usr/libexec/keybagd -t 15
113  ??  Ss      0:31.42 /usr/libexec/watchdogd
117  ??  Ss     44:38.88 /System/Library/Frameworks/CoreServices.framework/Framework
rks/Metadata.framework/Support/mds
118  ??  Ss      0:00.55 /System/Library/CoreServices/iconservicesd
```

We need to specify `w` 2 times to apply this setting, it's not a typo.

You can search for a specific process combining `grep` with a pipe, like this:

```
ps axww | grep "Visual Studio Code"
```



```
flavio ~ fish /Users/flavio -- fish -- 84x21
+ ~ ps axww | grep "Visual Studio Code"
367 ?? S      40:32.47 /Applications/Visual Studio Code.app/Contents/MacOS/Elect
ron -psn_0_77843
566 ?? S      93:02.21 /Applications/Visual Studio Code.app/Contents/Frameworks/
Code Helper (GPU).app/Contents/MacOS/Code Helper (GPU) --type=gpu-process --field-tr
ial-handle=1718379636,8422946071360466993,8181735644596197527,131072 --disable-featu
res=LayoutNG,PictureInPicture,SpareRendererForSitePerProcess --disable-color-correct
-rendering --gpu-preferences=KAAAAAAAAAgAAAAAAAAAYAAAAAAAAEAAAAAAAAAAAAAAAAAAg
BAAAgAAAAAEAAAAAAAAIAQAAAAAAAAABAAAAAAAAAGAEAAAAAAAAAgAQAAAAAACgBAAAAAAAAMAEAAAAAA
4AQAAAAAAAEABAAAAAAASAEAAAAABQAQAAAAAFgBAAAAAAAYAEAAAAABoAQAAAAAAHABAAAAAA
AeAEAAAAAAcAAQAAAAAAIgBAAAAAAkAEAAAAACYAQAAAAAAKABAAAAAAqAEAAAAAAcWAAQAAAA
AALgBAAAAAAAwEAAAAAADIAQAAAAAANABAAAAAA2AEAAAAADgAQAAAAAA0gBAAAAAA8AEAAAA
AAD4AQAAAAABAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQ
AAAAAAQAAAAcAAAAQAAAAAAQAAAAAAIAAAAEAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAA
AAAAAAQAAAAANAAAAEAAAAAAABAAAAAAABAAAAAAQAAAAIAAAQAAAAAAAEAAAAAGAAAAEAA
AAAAABAAAAABwAAABAAAAAAQAAAAgAAAAQAAAAAAAEAAAAKAAAAEAAAAABAAAAcWAAABBA
AAAAAAQAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAAAAQAAAA
QAAAAAAQAAAAHAAAAEAAAAAAEAAAAcAAAAABAAAAAAABAAAAAoAAAAQAAAAAAQAAAAALAAA
AEAAAAAAAEAAADQAAABAAAAAAABgAAAAAAQAAAAAAAYAAAAcAAAAEAAAAAAAGAAAABgA
AABAAAAAAABgAAAAcAAAAQAAAAAAAYAAAAIAAAAEAAAAAAAGAAAAcGAAABAAAAAAABgAAAAs
AAAAQAAAAAAAYAAAAAA --service-request-channel-token=4754214311225613582
```

The columns returned by `ps` represent some key information.

The first information is `PID`, the process ID. This is key when you want to reference this process in another command, for example to kill it.

Then we have `TT` that tells us the terminal id used.

Then `STAT` tells us the state of the process:

`I` a process that is idle (sleeping for longer than about 20 seconds) `R` a runnable process `S` a process that is sleeping for less than about 20 seconds `T` a stopped process `U` a process in uninterruptible wait `Z` a dead process (a *zombie*)

If you have more than one letter, the second represents further information, which can be very technical.

It's common to have `+` which indicates the process is in the foreground in its terminal. `s` means the process is a [session leader](#).

`TIME` tells us how long the process has been running.

top

A quick guide to the `top` command, used to list the processes running in real time

The `top` command is used to display dynamic real-time information about running processes in the system.

It's really handy to understand what is going on.

Its usage is simple, you just type `top`, and the terminal will be fully immersed in this new view:

```
flavio — top /Users/flavio — top — 80x24
Processes: 574 total, 2 running, 572 sleeping, 3807 threads      11:39:53
Load Avg: 1.24, 1.93, 2.23  CPU usage: 4.61% user, 3.68% sys, 91.69% idle
SharedLibs: 234M resident, 62M data, 16M linkedit.
MemRegions: 369828 total, 6215M resident, 163M private, 1603M shared.
PhysMem: 14G used (3684M wired), 1742M unused.
VM: 3080G vsize, 1991M framework vsize, 316459873(128) swapins, 322451178(0) swa
Networks: packets: 41309179/63G in, 44958830/24G out.
Disks: 68270763/1950G read, 43422129/1680G written.

PID  COMMAND      %CPU  TIME    #TH   #WQ   #PORT  MEM    PURG   CMPRS  PGRP
229  WindowServer  16.1  13:11:15 10    4    6607+  845M+  1216K- 173M   229
1185 com.docker.h  7.6   15:21:25 18    0    42    2374M  0B     316M   1162
82661 top           7.4   00:01.00 1/1    0    27+   8352K+ 0B     0B     82661
381  Terminal     6.0   04:31.43 11    4    436   105M   14M    28M-   381
1152 1Password 7   5.6   02:00:08 8     2    2473- 530M-  244K   349M   1152
0    kernel_task  3.9   06:15:45 262/12 0     0     510M+  0B     0B     0
82622 Google Chrom 2.8   00:07.20 16    1    209   81M    4096B  0B     38992
377  Bear         2.1   41:08.16 12    5    1760- 557M-  13M-   460M-   377
98108 Books        2.1   38:16.24 6     1    822   302M   0B     286M   98108
347  cloudd       2.0   10:10.00 16    5    785+  31M+   384K+  12M-   347
149  hidd         1.9   45:27.12 7     2    560   7208K  0B     2496K   149
50060 Music        1.5   45:57.57 22    3    684   201M+  460K   77M    50060
148  bluetoothd   1.5   44:49.50 3     1    801   18M    0B     7520K   148
1154 Rectangle    1.3   07:31.75 3     1    210-  52M-   0B     45M    1154
```

The process is long-running. To quit, you can type the `q` letter or `ctrl-C`.

There's a lot of information being given to us: the number of processes, how many are running or sleeping, the system load, the CPU usage, and a lot more.

Below, the list of processes taking the most memory and CPU is constantly updated.

By default, as you can see from the `%CPU` column highlighted, they are sorted by the CPU used.

You can add a flag to sort processes by memory utilized:

```
top -o mem
```

kill

Linux processes can receive **signals** and react to them.

That's one way we can interact with running programs.

The `kill` program can send a variety of signals to a program.

It's not just used to terminate a program, like the name would suggest, but that's its main job.

We use it in this way:

```
kill <PID>
```

By default, this sends the `TERM` signal to the process id specified.

We can use flags to send other signals, including:

```
kill -HUP <PID>
kill -INT <PID>
kill -KILL <PID>
kill -TERM <PID>
kill -CONT <PID>
kill -STOP <PID>
```

`HUP` means **hang up**. It's sent automatically when a terminal window that started a process is closed before terminating the process.

`INT` means **interrupt**, and it sends the same signal used when we press `ctrl-C` in the terminal, which usually terminates the process.

`KILL` is not sent to the process, but to the operating system kernel, which immediately stops and terminates the process.

`TERM` means **terminate**. The process will receive it and terminate itself. It's the default signal sent by `kill` .

`CONT` means **continue**. It can be used to resume a stopped process.

`STOP` is not sent to the process, but to the operating system kernel, which immediately stops (but does not terminate) the process.

You might see numbers used instead, like `kill -1 <PID>` . In this case,

`1` corresponds to `HUP` . `2` corresponds to `INT` . `9` corresponds to `KILL` . `15` corresponds to `TERM` . `18` corresponds to `CONT` . `19` corresponds to `STOP` .

killall

Similar to the `kill` command, `killall` instead of sending a signal to a specific process id will send the signal to multiple processes at once.

This is the syntax:

```
killall <name>
```

where `name` is the name of a program. For example you can have multiple instances of the `top` program running, and `killall top` will terminate them all.

You can specify the signal, like with `kill` (and check the `kill` tutorial to read more about the specific kinds of signals we can send), for example:

```
killall -HUP top
```


jobs

When we run a command in Linux / macOS, we can set it to run in the background using the `&` symbol after the command. For example we can run `top` in the background:

```
top &
```

This is very handy for long-running programs.

We can get back to that program using the `fg` command. This works fine if we just have one job in the background, otherwise we need to use the job number: `fg 1` , `fg 2` and so on. To get the job number, we use the `jobs` command.

Say we run `top &` and then `top -o mem &` , so we have 2 top instances running. `jobs` will tell us this:

A terminal window titled 'flavio — bash /Users/flavio — bash — 57x6' shows the output of the 'jobs' command. The output lists two stopped jobs: '[1]- Stopped top' and '[2]+ Stopped top -o mem'. The prompt 'bash-3.2\$' is visible at the bottom.

```
bash-3.2$ jobs
[1]- Stopped top
[2]+ Stopped top -o mem
bash-3.2$
```

Now we can switch back to one of those using `fg <jobid>` . To stop the program again we can hit `cmd-Z` .

Running `jobs -l` will also print the process id of each job.

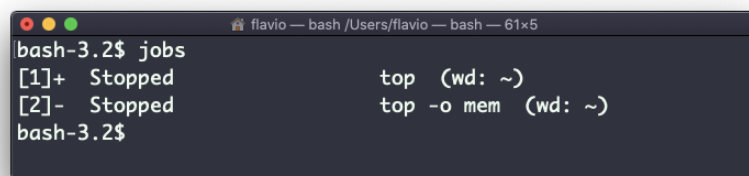
bg

When a command is running you can suspend it using `ctrl-Z`.

The command will immediately stop, and you get back to the shell terminal.

You can resume the execution of the command in the background, so it will keep running but it will not prevent you from doing other work in the terminal.

In this example I have 2 commands stopped:



```
flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ jobs
[1]+  Stopped                  top (wd: ~)
[2]-  Stopped                  top -o mem (wd: ~)
bash-3.2$
```

I can run `bg 1` to resume in the background the execution of the job #1.

I could have also said `bg` without any option, as the default is to pick the job #1 in the list.

fg

When a command is running in the background, because you started it with `&` at the end (example: `top &` or because you put it in the background with the `bg` command, you can put it to the foreground using `fg` .

Running

`fg`

will resume to the foreground the last job that was suspended.

You can also specify which job you want to resume to the foreground passing the job number, which you can get using the `jobs` command.

A terminal window with a dark background and light text. The title bar shows 'flavio — bash /Users/flavio — bash — 61x5'. The terminal content shows the command 'jobs' being executed, resulting in two jobs listed: '[1]+ Stopped top (wd: ~)' and '[2]- Stopped top -o mem (wd: ~)'. The prompt 'bash-3.2\$' is visible at the bottom.

```
bash-3.2$ jobs
[1]+  Stopped                  top (wd: ~)
[2]-  Stopped                  top -o mem (wd: ~)
bash-3.2$
```

Running `fg 2` will resume job #2:

```
flavio — bash /Users/flavio — top — 61x23
Processes: 574 total, 2 running, 572 sleeping, 3823 threads
16:12:54 Load Avg: 1.44, 1.60, 1.74
CPU usage: 3.76% user, 2.99% sys, 93.23% idle
SharedLibs: 235M resident, 62M data, 16M linkedit.
MemRegions: 365072 total, 6752M resident, 166M private, 1835M
PhysMem: 15G used (3692M wired), 738M unused.
VM: 3092G vsize, 1991M framework vsize, 318057086(0) swapins,
Networks: packets: 43489522/66G in, 47248873/25G out.
Disks: 69434725/1968G read, 44158267/1695G written.

PID    COMMAND      %CPU  TIME    #TH    #WQ    #PORT  MEM
1185   com.docker.h  9.1   15:40:34 18      0      42     2374M
566    Code Helper  0.0   93:50.08 8        1      267    1606M
85440  hugo         0.0   09:19.41 20       0      29     1029M
229    WindowServer 16.0  13:29:14 10       4     6193+  851M
1152   1Password 7  6.9   02:10:02 8        2     2400   581M
377    Bear         0.0   43:56.57 13       6     1763   570M+
605    Code Helper  0.1   63:35.69 27       1      226    557M
42433  Photos      0.0   02:37.69 7        1      535    524M
588    Figma Helper 0.0   26:13.88 8        1      219    522M
0      kernel_task  3.2   06:24:14 262/12  0       0      514M
38992  Google Chrom 0.6   02:52:53 35       2     1238   506M
594    Code Helper  0.3   54:59.85 27       1      228    471M
```

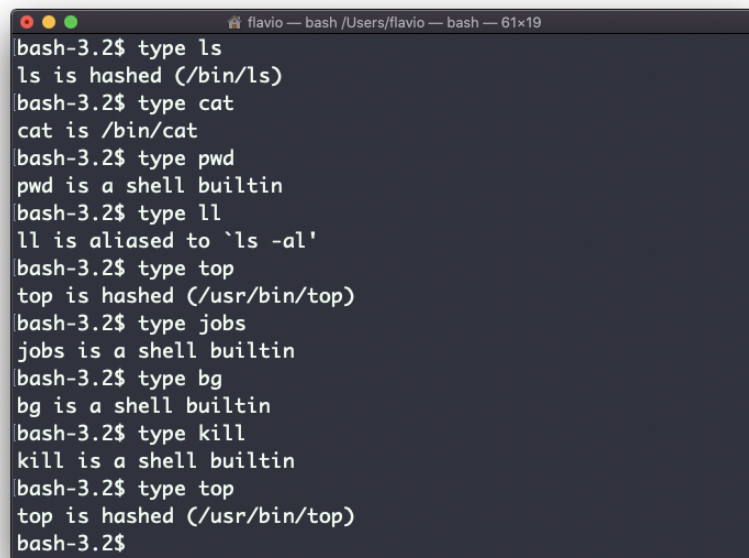
type

A command can be one of those 4 types:

- an executable
- a shell built-in program
- a shell function
- an alias

The `type` command can help figure out this, in case we want to know or we're just curious. It will tell you how the command will be interpreted.

The output will depend on the shell used. This is Bash:

A terminal window titled 'flavio — bash /Users/flavio — bash — 61x19' showing the output of the 'type' command for various utilities. The output indicates that 'ls', 'cat', 'top', and 'kill' are hashed executables, while 'pwd', 'jobs', 'bg', and 'll' are shell builtins or aliases. 'll' is specifically noted as being aliased to 'ls -al'.

```
bash-3.2$ type ls
ls is hashed (/bin/ls)
bash-3.2$ type cat
cat is /bin/cat
bash-3.2$ type pwd
pwd is a shell builtin
bash-3.2$ type ll
ll is aliased to `ls -al`
bash-3.2$ type top
top is hashed (/usr/bin/top)
bash-3.2$ type jobs
jobs is a shell builtin
bash-3.2$ type bg
bg is a shell builtin
bash-3.2$ type kill
kill is a shell builtin
bash-3.2$ type top
top is hashed (/usr/bin/top)
bash-3.2$
```

This is Zsh:

```
flavio@mbp ~ % type ls
ls is /bin/ls
flavio@mbp ~ % type cat
cat is /bin/cat
flavio@mbp ~ % type pwd
pwd is a shell builtin
flavio@mbp ~ % type ll
ll not found
flavio@mbp ~ % alias ll='ls -al'
flavio@mbp ~ % type ll
ll is an alias for ls -al
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ % type jobs
jobs is a shell builtin
flavio@mbp ~ % type bg
bg is a shell builtin
flavio@mbp ~ % type kill
kill is a shell builtin
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ %
```

This is Fish:

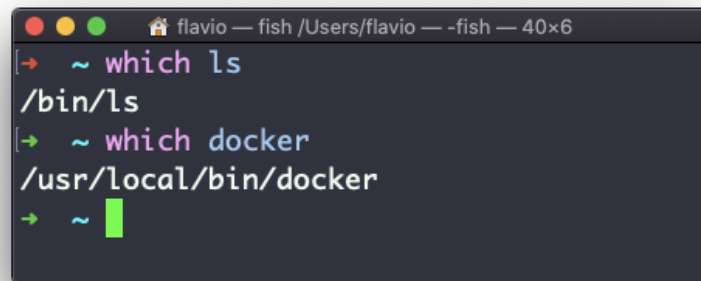
```
flavio@mbp ~ % type ls
ls is a function with definition
# Defined in /usr/local/Cellar/fish/3.1.0/share/fish/functions/ls.fish @ line 13
function ls --description 'List contents of directory'
    set -l opt -G
    isatty stdout
    and set -a opt -F
    command ls $opt $argv
end
flavio@mbp ~ % type cat
cat is /bin/cat
flavio@mbp ~ % type pwd
pwd is a builtin
flavio@mbp ~ % type ll
ll is a function with definition
# Defined in /usr/local/Cellar/fish/3.1.0/share/fish/functions/ll.fish @ line 4
function ll --description 'List contents of directory using long format'
    ls -lh $argv
end
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ % type jobs
jobs is a builtin
flavio@mbp ~ % type bg
bg is a function with definition
# Defined in /usr/local/Cellar/fish/3.1.0/share/fish/config.fish @ line 274
function bg
    set -l jobbltn bg
    builtin $jobbltn (__fish_expand_pid_args $argv)
end
flavio@mbp ~ % type kill
kill is a function with definition
# Defined in /usr/local/Cellar/fish/3.1.0/share/fish/config.fish @ line 279
function kill
    command kill (__fish_expand_pid_args $argv)
end
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ %
```

One of the most interesting things here is that for aliases it will tell you what is aliasing to. You can see the `ll` alias, in the case of Bash and Zsh, but Fish provides it by default, so it will tell you it's a built-in shell function.

which

Suppose you have a command you can execute, because it's in the shell path, but you want to know where it is located.

You can do so using `which`. The command will return the path to the command specified:

A terminal window with a dark background and light-colored text. The window title bar shows 'flavio — fish /Users/flavio — -fish — 40x6'. The terminal shows three commands and their outputs: 1. Command: `~ which ls`, Output: `/bin/ls`. 2. Command: `~ which docker`, Output: `/usr/local/bin/docker`. 3. Command: `~` followed by a green cursor block.

```
flavio — fish /Users/flavio — -fish — 40x6
→ ~ which ls
/bin/ls
→ ~ which docker
/usr/local/bin/docker
→ ~
```

`which` will only work for executables stored on disk, not aliases or built-in shell functions.

nohup

Sometimes you have to run a long-lived process on a remote machine, and then you need to disconnect.

Or you simply want to prevent the command to be halted if there's any network issue between you and the server.

The way to make a command run even after you log out or close the session to a server is to use the `nohup` command.

Use `nohup <command>` to let the process continue working even after you log out.

xargs

The `xargs` command is used in a UNIX shell to convert input from standard input into arguments to a command.

In other words, through the use of `xargs` the output of a command is used as the input of another command.

Here's the syntax you will use:

```
command1 | xargs command2
```

We use a pipe (`|`) to pass the output to `xargs` . That will take care of running the `command2` command, using the output of `command1` as its argument(s).

Let's do a simple example. You want to remove some specific files from a directory. Those files are listed inside a text file.

We have 3 files: `file1` , `file2` , `file3` .

In `todelete.txt` we have a list of files we want to delete, in this example `file1` and `file3` :

A terminal window titled 'testing — fish /Users/flavio/testing — -fish — 62x7' shows the following commands and output:

```
→ testing ls
file1      file2      file3      todelete.txt
→ testing cat todelete.txt
file1
file3
→ testing
```

We will channel the output of `cat todelete.txt` to the `rm` command, through `xargs` .

In this way:

```
cat todelete.txt | xargs rm
```

That's the result, the files we listed are now deleted:

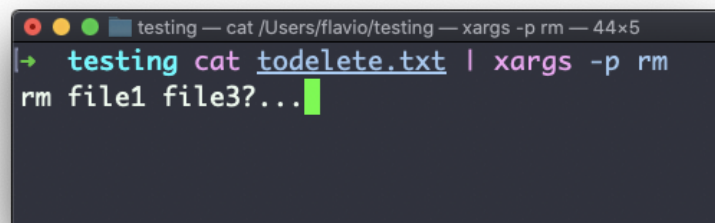


```
testing — fish /Users/flavio/testing — -fish — 62x10
→ testing ls
file1      file2      file3      todelete.txt
→ testing cat todelete.txt
file1
file3
→ testing cat todelete.txt | xargs rm
→ testing ls
file2      todelete.txt
→ testing
```

The way it works is that `xargs` will run `rm` 2 times, one for each line returned by `cat` .

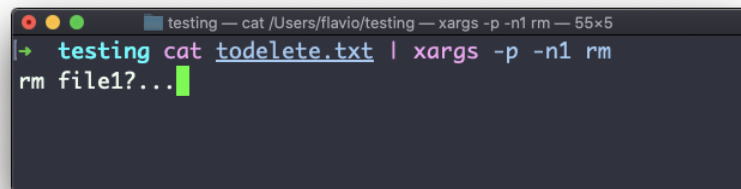
This is the simplest usage of `xargs` . There are several options we can use.

One of the most useful in my opinion, especially when starting to learn `xargs` , is `-p` . Using this option will make `xargs` print a confirmation prompt with the action it's going to take:



```
testing — cat /Users/flavio/testing — xargs -p rm — 44x5
→ testing cat todelete.txt | xargs -p rm
rm file1 file3?...
```

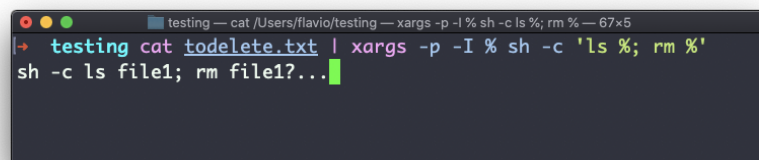
The `-n` option lets you tell `xargs` to perform one iteration at a time, so you can individually confirm them with `-p`. Here we tell `xargs` to perform one iteration at a time with `-n1`:

A terminal window titled 'testing' with the command prompt 'cat /Users/flavio/testing -- xargs -p -n1 rm -- 55x5'. The user enters 'testing cat todelete.txt | xargs -p -n1 rm'. The output shows 'rm file1?...' followed by a green cursor.

The `-I` option is another widely used one. It allows you to get the output into a placeholder, and then you can do various things.

One of them is to run multiple commands:

```
command1 | xargs -I % /bin/bash -c 'command2 %; comr
```

A terminal window titled 'testing' with the command prompt 'cat /Users/flavio/testing -- xargs -p -I % sh -c ls %; rm % -- 67x5'. The user enters 'testing cat todelete.txt | xargs -p -I % sh -c 'ls %; rm %''. The output shows 'sh -c ls file1; rm file1?...' followed by a green cursor.

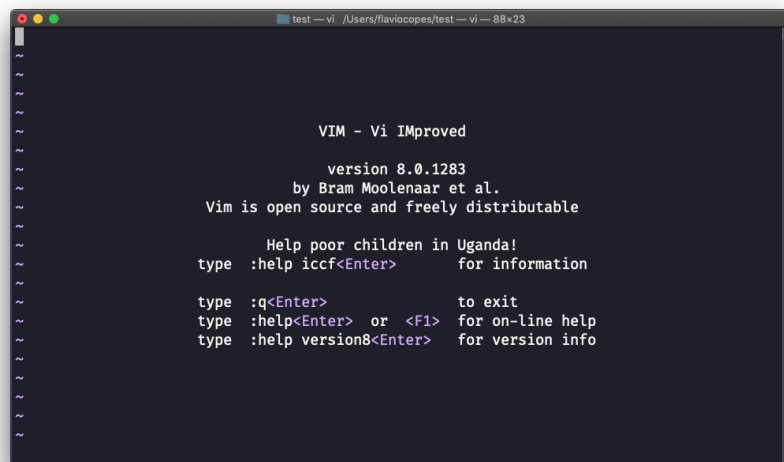
You can swap the `%` symbol I used above with anything else, it's a variable

vim

`vim` is a **very** popular file editor, especially among programmers. It's actively developed and frequently updated, and there's a very big community around it. There's even a [Vim conference](#)!

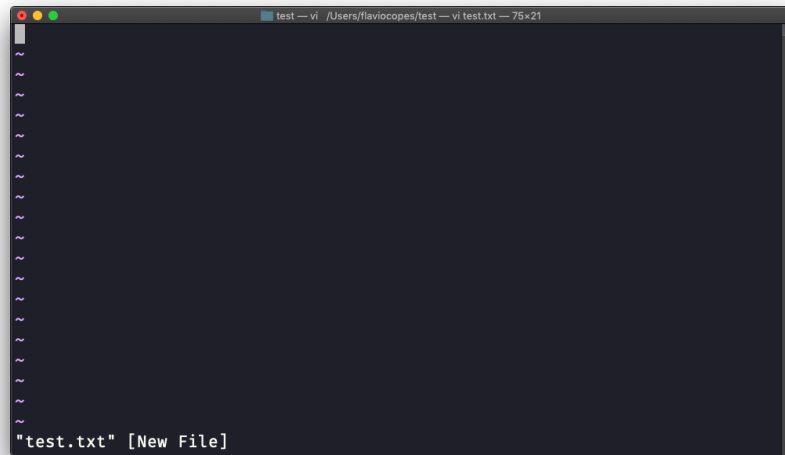
`vi` in modern systems is just an alias to `vim`, which means `vi` i m proved.

You start it by running `vi` on the command line.



You can specify a filename at invocation time to edit that specific file:

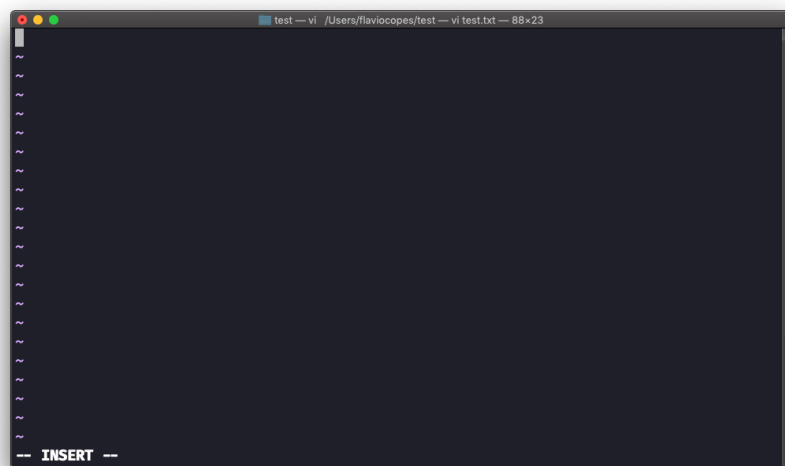
```
vi test.txt
```



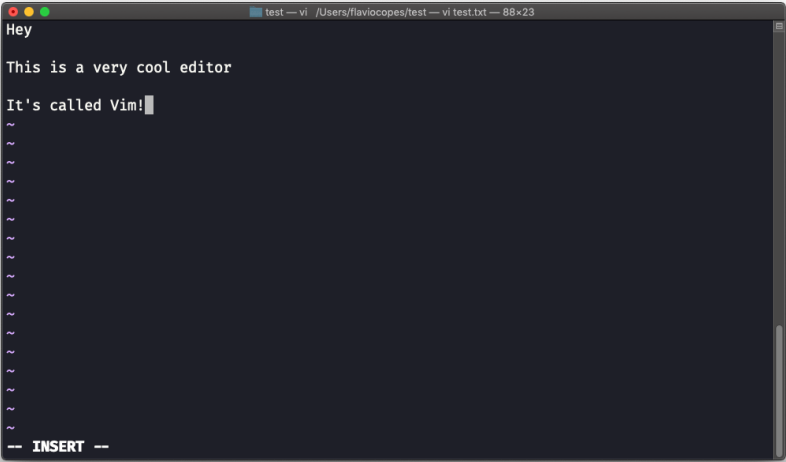
You have to know that Vim has 2 main modes:

- *command* (or *normal*) mode
- *insert* mode

When you start the editor, you are in command mode. You can't enter text like you expect from a GUI-based editor. You have to enter **insert mode**. You can do this by pressing the `i` key. Once you do so, the `-- INSERT --` word appear at the bottom of the editor:

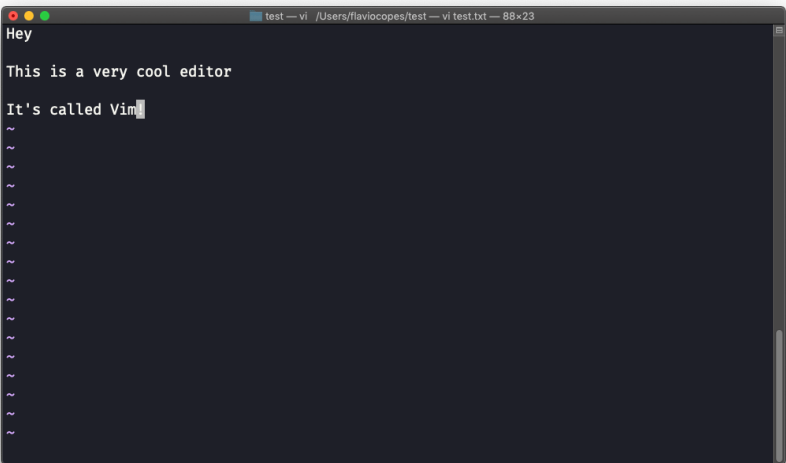


Now you can start typing and filling the screen with the file contents:



You can move around the file with the arrow keys, or using the `h` - `j` - `k` - `l` keys. `h-l` for left-right, `j-k` for down-up.

Once you are done editing you can press the `esc` key to exit insert mode, and go back to **command mode**.



At this point you can navigate the file, but you can't add content to it (and be careful which keys you press as they might be commands).

One thing you might want to do now is **saving the file**. You can do so by pressing `:` (colon), then `w`.

You can **save and quit** pressing `:` then `w` and `q` :
`:wq`

You can **quit without saving**, pressing `:` then `q`
and `! : :q!`

You can **undo** and edit by going to command mode and pressing `u` . You can **redo** (cancel an undo) by pressing `ctrl-r` .

Those are the basics of working with Vim. From here starts a rabbit hole we can't go into in this little introduction.

I will only mention those commands that will get you started editing with Vim:

- pressing the `x` key deletes the character currently highlighted
- pressing `A` goes at the end of the currently selected line
- press `0` to go to the start of the line
- go to the first character of a word and press `d` followed by `w` to delete that word. If you follow it with `e` instead of `w` , the white space before the next word is preserved
- use a number between `d` and `w` to delete more than 1 word, for example use `d3w` to delete 3 words forward
- press `d` followed by `d` to delete a whole entire line. Press `d` followed by `$` to delete the entire line from where the cursor is, until the end

To find out more about Vim I can recommend the [Vim FAQ](#) and especially running the `vimtutor` command, which should already be installed in your system and

will greatly help you start your `vim` explorations.

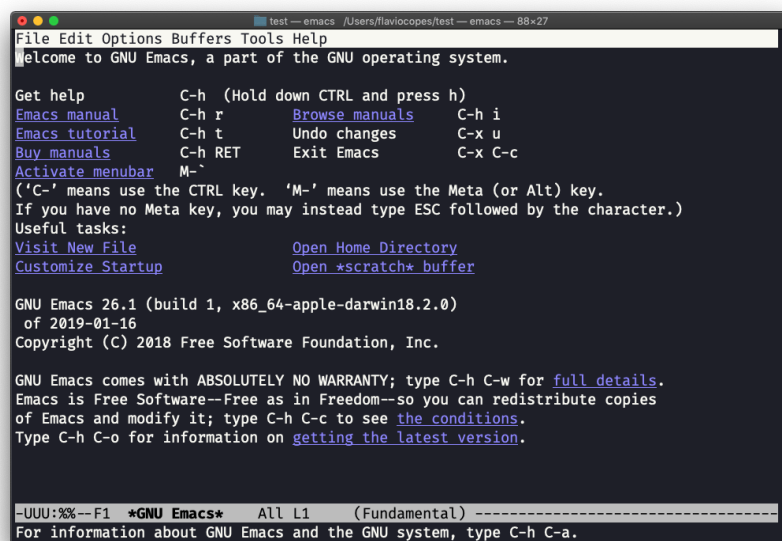
emacs

`emacs` is an awesome editor and it's historically regarded as *the* editor for UNIX systems. Famously `vi` VS `emacs` flame wars and heated discussions caused many unproductive hours for developers around the world.

`emacs` is very powerful. Some people use it all day long as a kind of operating system (<https://news.ycombinator.com/item?id=19127258>).

We'll just talk about the basics here.

You can open a new emacs session simply by invoking `emacs` :



```
test — emacs /Users/flaviocopes/test — emacs — 88x27
File Edit Options Buffers Tools Help
Welcome to GNU Emacs, a part of the GNU operating system.

Get help          C-h (Hold down CTRL and press h)
Emacs manual      C-h r      Browse manuals      C-h i
Emacs tutorial    C-h t      Undo changes       C-x u
Buy manuals      C-h RET    Exit Emacs        C-x C-c
Activate menubar M-`

('C-' means use the CTRL key. 'M-' means use the Meta (or Alt) key.
If you have no Meta key, you may instead type ESC followed by the character.)

Useful tasks:
Visit New File      Open Home Directory
Customize Startup   Open *scratch* buffer

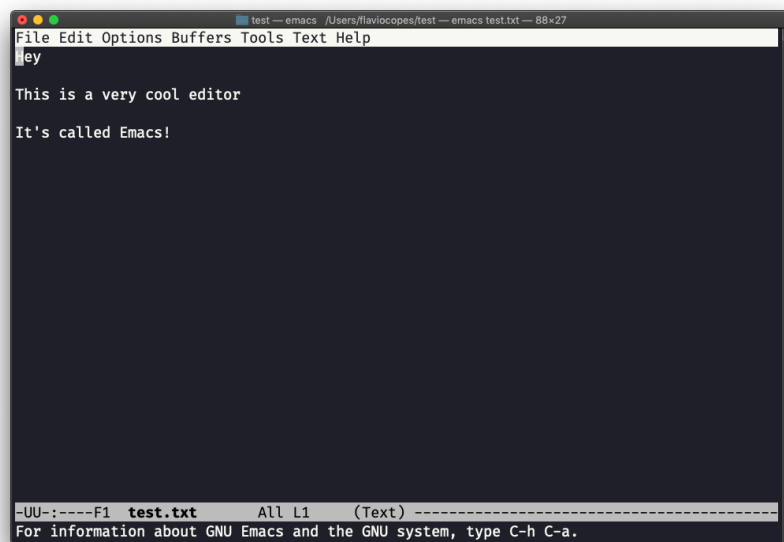
GNU Emacs 26.1 (build 1, x86_64-apple-darwin18.2.0)
of 2019-01-16
Copyright (c) 2018 Free Software Foundation, Inc.

GNU Emacs comes with ABSOLUTELY NO WARRANTY; type C-h C-w for full details.
Emacs is Free Software—Free as in Freedom—so you can redistribute copies
of Emacs and modify it; type C-h C-c to see the conditions.
Type C-h C-o for information on getting the latest version.

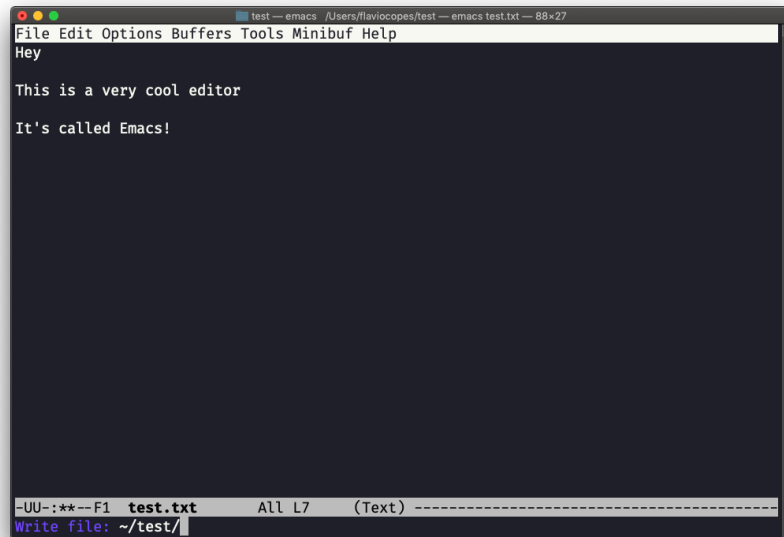
-UUU:%%--F1 *GNU Emacs* All L1 (Fundamental) -----
For information about GNU Emacs and the GNU system, type C-h C-a.
```

macOS users, stop a second now. If you are on Linux there are no problems, but macOS does not ship applications using GPLv3, and every built-in UNIX command that has been updated to GPLv3 has not been updated. While there is a little problem with the commands I listed up to now, in this case using an emacs version from 2007 is not exactly the same as using a version with 12 years of improvements and change. This is not a problem with Vim, which is up to date. To fix this, run `brew install emacs` and running `emacs` will use the new version from Homebrew (make sure you have [Homebrew](#) installed)

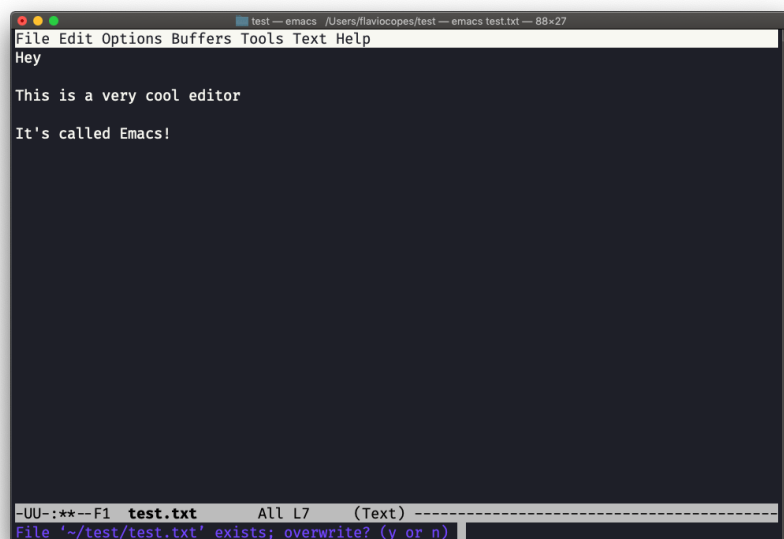
You can also edit an existing file calling `emacs <filename>` :



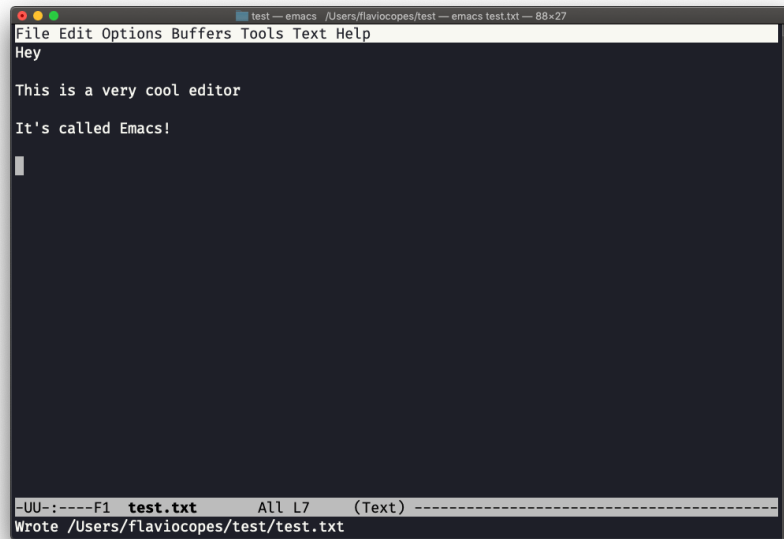
You can start editing and once you are done, press `ctrl-x` followed by `ctrl-w` . You confirm the folder:



and Emacs tell you the file exists, asking you if it should overwrite it:



Answer `y` , and you get a confirmation of success:



You can exit Emacs pressing `ctrl-x` followed by `ctrl-c`. Or `ctrl-x` followed by `c` (keep `ctrl` pressed).

There is a lot to know about Emacs. More than I am able to write in this little introduction. I encourage you to open Emacs and press `ctrl-h` `r` to open the built-in manual and `ctrl-h` `t` to open the official tutorial.

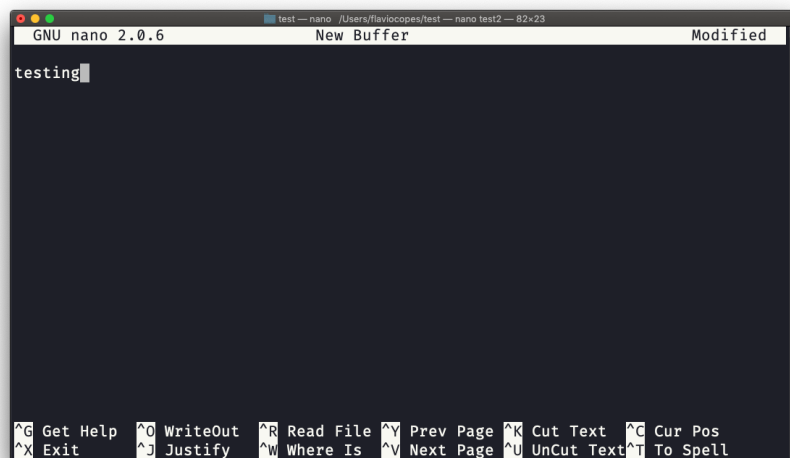
nano

`nano` is a beginner friendly editor.

Run it using `nano <filename> .`

You can directly type characters into the file without worrying about modes.

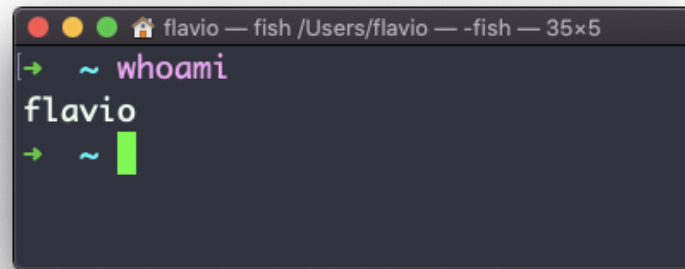
You can quit without editing using `ctrl-X` . If you edited the file buffer, the editor will ask you for confirmation and you can save the edits, or discard them. The help at the bottom shows you the keyboard commands that let you work with the file:



`pico` is more or less the same, although `nano` is the GNU version of `pico` which at some point in history was not open source and the `nano` clone was made to satisfy the GNU operating system license requirements.

whoami

Type `whoami` to print the user name currently logged in to the terminal session:

A screenshot of a terminal window with a dark background. The title bar at the top shows a home icon, the name 'flavio', and the path '— fish /Users/flavio — -fish — 35x5'. The terminal content shows a prompt '→ ~' followed by the command 'whoami' in pink. The output 'flavio' is displayed below. A second prompt '→ ~' is shown with a green cursor at the end.

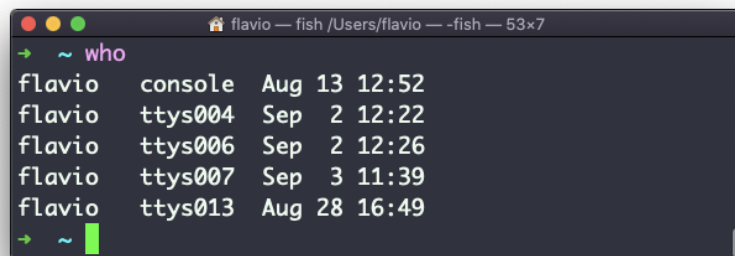
```
→ ~ whoami  
flavio  
→ ~
```

Note: this is different from the `who am i` command, which prints more information

who

The `who` command displays the users logged in to the system.

Unless you're using a server multiple people have access to, chances are you will be the only user logged in, multiple times:

A terminal window titled 'flavio — fish /Users/flavio — -fish — 53x7' showing the output of the 'who' command. The output lists five sessions for the user 'flavio', each with a terminal name and a timestamp. The sessions are: 'console' on 'Aug 13 12:52', 'ttys004' on 'Sep 2 12:22', 'ttys006' on 'Sep 2 12:26', 'ttys007' on 'Sep 3 11:39', and 'ttys013' on 'Aug 28 16:49'. The prompt is '~ who' and the cursor is at the end of the last line.

```
flavio console Aug 13 12:52
flavio ttys004 Sep 2 12:22
flavio ttys006 Sep 2 12:26
flavio ttys007 Sep 3 11:39
flavio ttys013 Aug 28 16:49
```

Why multiple times? Because each shell opened will count as an access.

You can see the name of the terminal used, and the time/day the session was started.

The `-aH` flags will tell `who` to display more information, including the idle time and the process ID of the terminal:


```
flavio — fish /Users/flavio — -fish — 66x15
~ who -aH
USER      LINE      WHEN      IDLE      PID      COMMENT
reboot    ~         Aug 13 12:52 00:01     1
flavio     console   Aug 13 12:52 old        167
flavio     ttys000   Aug 16 18:39 old        70258     term=0 exit=0
flavio     ttys004   Sep  2 12:22 .         71365
flavio     ttys006   Sep  2 12:26 00:41     71675
flavio     ttys007   Sep  3 11:39 00:01     82656
flavio     ttys008   Aug 21 14:33 04:15     25883     term=0 exit=0
flavio     ttys009   Aug 15 19:36 old        52214     term=0 exit=0
flavio     ttys010   Aug 23 16:32 .         38649     term=0 exit=0
flavio     ttys011   Aug 21 10:20 .         18855     term=0 exit=0
flavio     ttys013   Aug 28 16:49 01:33     5086
.         run-level 3
~
```

The special `who am i` command will list the current terminal session details:

```
flavio — fish /Users/flavio — -fish — 45x5
~ who am i
flavio ttys004 Sep  2 12:22
~
```

```
flavio — fish /Users/flavio — -fish — 65x5
~ who -aH am i
USER      LINE      WHEN      IDLE      PID      COMMENT
flavio     ttys004   Sep  2 12:22 .         71365
~
```

su

While you're logged in to the terminal shell with one user, you might have the need to switch to another user.

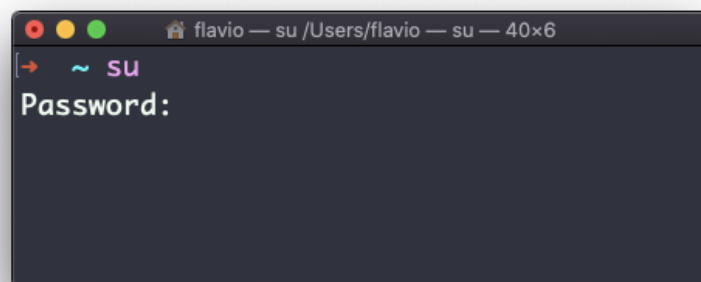
For example you're logged in as root to perform some maintenance, but then you want to switch to a user account.

You can do so with the `su` command:

```
su <username>
```

For example: `su flavio`.

If you're logged in as a user, running `su` without anything else will prompt to enter the `root` user password, as that's the default behavior.



`su` will start a new shell as another user.

When you're done, typing `exit` in the shell will close that shell, and will return back to the current user's shell.

sudo

`sudo` is commonly used to run a command as root.

You must be enabled to use `sudo`, and once you do, you can run commands as root by entering your user's password (*not* the root user password).

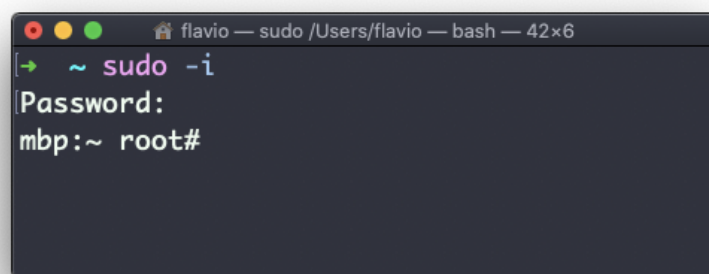
The permissions are highly configurable, which is great especially in a multi-user server environment, and some users can be granted access to running specific commands through `sudo`.

For example you can edit a system configuration file:

```
sudo nano /etc/hosts
```

which would otherwise fail to save since you don't have the permissions for it.

You can run `sudo -i` to start a shell as root:



You can use `sudo` to run commands as any user. `root` is the default, but use the `-u` option to specify another user:

```
sudo -u flavio ls /Users/flavio
```

passwd

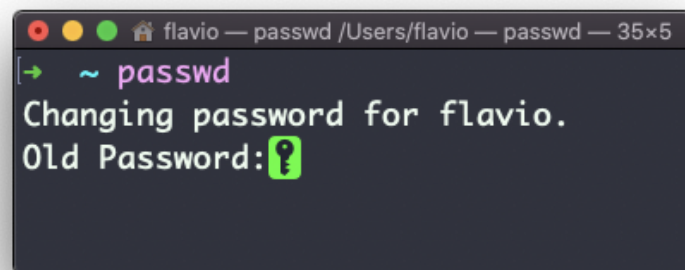
Users in Linux have a password assigned. You can change the password using the `passwd` command.

There are two situations here.

The first is when you want to change your password. In this case you type:

```
passwd
```

and an interactive prompt will ask you for the old password, then it will ask you for the new one:



When you're `root` (or have superuser privileges) you can set the username of which you want to change the password:

```
passwd <username> <new password>
```

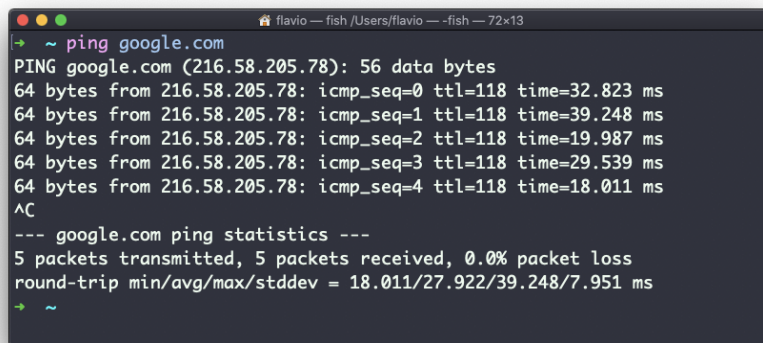
In this case you don't need to enter the old one.

ping

The `ping` command pings a specific network host, on the local network or on the Internet.

You use it with the syntax `ping <host>` where `<host>` could be a domain name, or an IP address.

Here's an example pinging `google.com` :



```
flavio — fish /Users/flavio — -fish — 72x13
➔ ~ ping google.com
PING google.com (216.58.205.78): 56 data bytes
64 bytes from 216.58.205.78: icmp_seq=0 ttl=118 time=32.823 ms
64 bytes from 216.58.205.78: icmp_seq=1 ttl=118 time=39.248 ms
64 bytes from 216.58.205.78: icmp_seq=2 ttl=118 time=19.987 ms
64 bytes from 216.58.205.78: icmp_seq=3 ttl=118 time=29.539 ms
64 bytes from 216.58.205.78: icmp_seq=4 ttl=118 time=18.011 ms
^C
--- google.com ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 18.011/27.922/39.248/7.951 ms
➔ ~
```

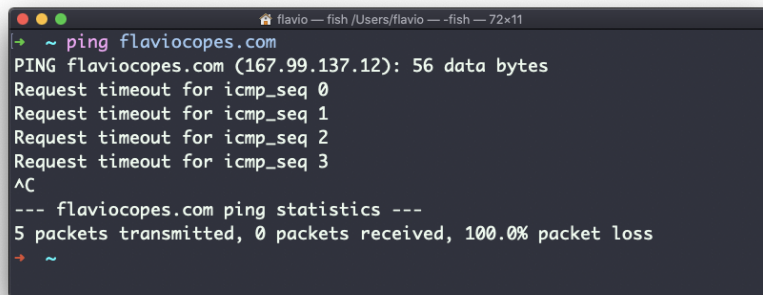
The command sends a request to the server, and the server returns a response.

`ping` keeps sending the request every second, by default, and will keep running until you stop it with `ctrl-C`, unless you pass the number of times you want to try with the `-c` option: `ping -c 2 google.com`.

Once `ping` is stopped, it will print some statistics about the results: the percentage of packages lost, and statistics about the network performance.

As you can see the screen prints the host IP address, and the time that it took to get the response back.

Not all servers support pinging, in case the requests times out:

A terminal window with a dark background and light text. The window title is 'flavio — fish /Users/flavio — -fish — 72x11'. The prompt is '~'. The user has entered 'ping flaviocopes.com'. The output shows 'PING flaviocopes.com (167.99.137.12): 56 data bytes' followed by four 'Request timeout for icmp_seq 0' through 'icmp_seq 3'. The user has pressed 'Ctrl-C' (shown as '^C'). The output then shows '--- flaviocopes.com ping statistics ---' and '5 packets transmitted, 0 packets received, 100.0% packet loss'. The prompt is '~'.

```
~$ ping flaviocopes.com
PING flaviocopes.com (167.99.137.12): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
^C
--- flaviocopes.com ping statistics ---
5 packets transmitted, 0 packets received, 100.0% packet loss
~$
```

Sometimes this is done on purpose, to "hide" the server, or just to reduce the load. The ping packets can also be filtered by firewalls.

`ping` works using the **ICMP protocol** (*Internet Control Message Protocol*), a network layer protocol just like TCP or UDP.

The request sends a packet to the server with the `ECHO_REQUEST` message, and the server returns a `ECHO_REPLY` message. I won't go into details, but this is the basic concept.

Pinging a host is useful to know if the host is reachable (supposing it implements ping), and how distant it is in terms of how long it takes to get back to you. Usually the nearest the server is geographically, the less time it will take to return back to you, for simple physical laws that cause a longer distance to introduce more delay in the cables.

traceroute

When you try to reach a host on the Internet, you go through your home router, then you reach your ISP network, which in turn goes through its own upstream network router, and so on, until you finally reach the host.

Have you ever wanted to know what are the steps that your packets go through to do that?

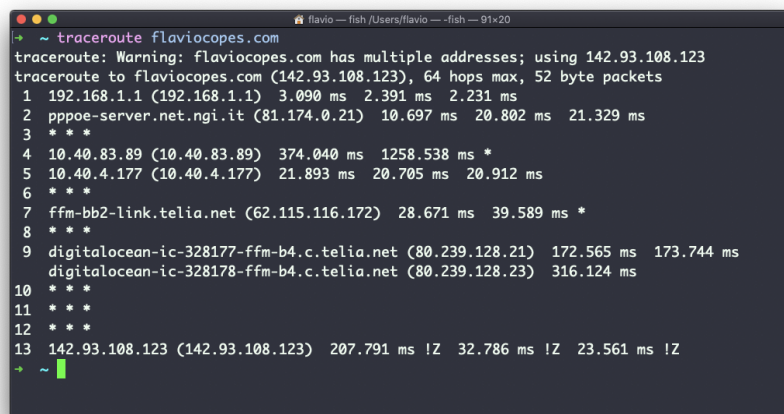
The `traceroute` command is made for this.

You invoke

```
traceroute <host>
```

and it will (slowly) gather all the information while the packet travels.

In this example I tried reaching for my blog with `traceroute flaviocopes.com` :



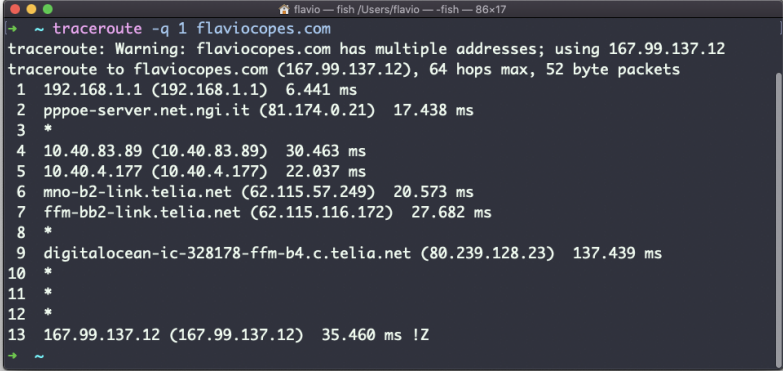
```
flavio ~ — fish — flaviocopes.com
~$ traceroute flaviocopes.com
traceroute: Warning: flaviocopes.com has multiple addresses; using 142.93.108.123
traceroute to flaviocopes.com (142.93.108.123), 64 hops max, 52 byte packets
 1  192.168.1.1 (192.168.1.1)  3.090 ms  2.391 ms  2.231 ms
 2  pppoe-server.net.ngi.it (81.174.0.21)  10.697 ms  20.802 ms  21.329 ms
 3  * * *
 4  10.40.83.89 (10.40.83.89)  374.040 ms  1258.538 ms *
 5  10.40.4.177 (10.40.4.177)  21.893 ms  20.705 ms  20.912 ms
 6  * * *
 7  ffm-bb2-link.telia.net (62.115.116.172)  28.671 ms  39.589 ms *
 8  * * *
 9  digitalocean-ic-328177-ffm-b4.c.telia.net (80.239.128.21)  172.565 ms  173.744 ms
   digitalocean-ic-328178-ffm-b4.c.telia.net (80.239.128.23)  316.124 ms
10  * * *
11  * * *
12  * * *
13  142.93.108.123 (142.93.108.123)  207.791 ms !Z  32.786 ms !Z  23.561 ms !Z
~$
```


Not every router travelled returns us information. In this case, `traceroute` prints `* * *`. Otherwise, we can see the hostname, the IP address, and some performance indicator.

For every router we can see 3 samples, which means `traceroute` tries by default 3 times to get you a good indication of the time needed to reach it. This is why it takes this long to execute `traceroute` compared to simply doing a `ping` to that host.

You can customize this number with the `-q` option:

```
traceroute -q 1 flaviocopes.com
```



```
flavio ~ flaviocopes.com
flavio ~ fish /Users/flavio — fish — 86x17
➔ ~ traceroute -q 1 flaviocopes.com
traceroute: Warning: flaviocopes.com has multiple addresses; using 167.99.137.12
traceroute to flaviocopes.com (167.99.137.12), 64 hops max, 52 byte packets
 1  192.168.1.1 (192.168.1.1)  6.441 ms
 2  pppoe-server.net.ngi.it (81.174.0.21)  17.438 ms
 3  *
 4  10.40.83.89 (10.40.83.89)  30.463 ms
 5  10.40.4.177 (10.40.4.177)  22.037 ms
 6  mno-b2-link.telia.net (62.115.57.249)  20.573 ms
 7  ffm-bb2-link.telia.net (62.115.116.172)  27.682 ms
 8  *
 9  digitalocean-ic-328178-ffm-b4.c.telia.net (80.239.128.23)  137.439 ms
10  *
11  *
12  *
13  167.99.137.12 (167.99.137.12)  35.460 ms 1Z
➔ ~
```

clear

Type `clear` to clear all the previous commands that were ran in the current terminal.

The screen will clear and you will just see the prompt at the top:



Note: this command has a handy shortcut: `ctrl-L`

Once you do that, you will lose access to scrolling to see the output of the previous commands entered.

So you might want to use `clear -x` instead, which still clears the screen, but lets you go back to see the previous work by scrolling up.

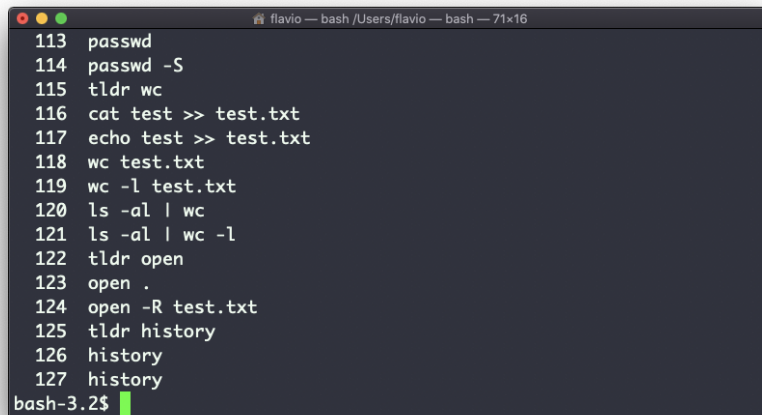
history

Every time we run a command, that's memorized in the history.

You can display all the history using:

```
history
```

This shows the history with numbers:

A terminal window titled 'flavio — bash /Users/flavio — bash — 71x16' showing a list of commands with line numbers from 113 to 127. The commands are: 113 passwd, 114 passwd -S, 115 tldr wc, 116 cat test >> test.txt, 117 echo test >> test.txt, 118 wc test.txt, 119 wc -l test.txt, 120 ls -al | wc, 121 ls -al | wc -l, 122 tldr open, 123 open ., 124 open -R test.txt, 125 tldr history, 126 history, 127 history. The prompt 'bash-3.2\$' is visible at the bottom with a green cursor.

```
113 passwd
114 passwd -S
115 tldr wc
116 cat test >> test.txt
117 echo test >> test.txt
118 wc test.txt
119 wc -l test.txt
120 ls -al | wc
121 ls -al | wc -l
122 tldr open
123 open .
124 open -R test.txt
125 tldr history
126 history
127 history
bash-3.2$
```

You can use the syntax `!<command number>` to repeat a command stored in the history, in the above example typing `!121` will repeat the `ls -al | wc -l` command.

Typically the last 500 commands are stored in the history.

You can combine this with `grep` to find a command you ran:

```
history | grep docker
```

```
flavio — bash /Users/flavio — bash — 71x8
bash-3.2$ history | grep docker
 7  git clone https://github.com/docker/getting-started.git
 8  git clone https://github.com/docker/getting-started.git
14  docker container stop $(docker container ls)
15  docker container stop $(docker container ls)
16  docker container stop $(docker container ls -aq)
130 history | grep docker
bash-3.2$
```

To clear the history, run `history -c`

export

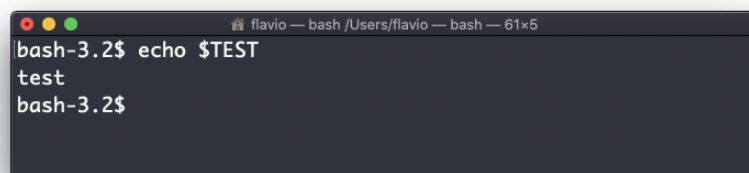
The `export` command is used to export variables to child processes.

What does this mean?

Suppose you have a variable `TEST` defined in this way:

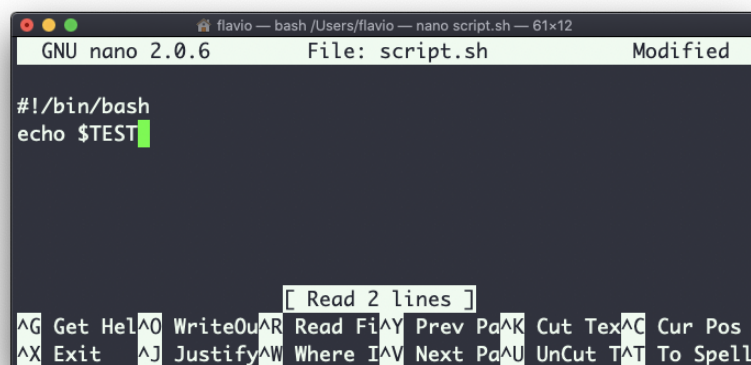
```
TEST="test"
```

You can print its value using `echo $TEST` :



```
flavio — bash /Users/flavio — bash — 61x5
bash-3.2$ echo $TEST
test
bash-3.2$
```

But if you try defining a Bash script in a file `script.sh` with the above command:



```
flavio — bash /Users/flavio — nano script.sh — 61x12
GNU nano 2.0.6      File: script.sh      Modified

#!/bin/bash
echo $TEST
```

Then you set `chmod u+x script.sh` and you execute this script with `./script.sh`, the `echo $TEST` line will print nothing!

This is because in Bash the `TEST` variable was defined local to the shell. When executing a shell script or another command, a subshell is launched to execute it, which does not contain the current shell local variables.

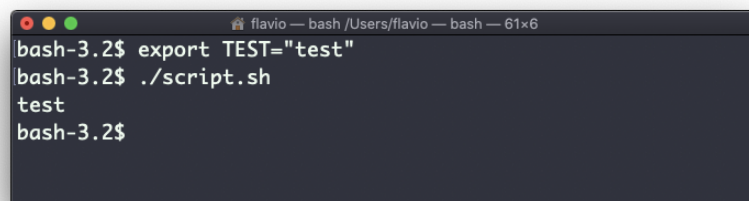
To make the variable available there we need to define `TEST` not in this way:

```
TEST="test"
```

but in this way:

```
export TEST="test"
```

Try that, and running `./script.sh` now should print "test":

A terminal window with a dark background and light text. The window title is 'flavio — bash /Users/flavio — bash — 61x6'. The prompt is 'bash-3.2\$'. The user enters 'export TEST="test"', followed by './script.sh'. The output of the script is 'test'. The prompt returns to 'bash-3.2\$'.

Sometimes you need to append something to a variable. It's often done with the `PATH` variable. You use this syntax:

```
export PATH=$PATH:/new/path
```

It's common to use `export` when you create new variables in this way, but also when you create variables in the `.bash_profile` or `.bashrc` configuration files with Bash, or in `.zshenv` with Zsh.

To remove a variable, use the `-n` option:

```
export -n TEST
```

Calling `export` without any option will list all the exported variables.

crontab

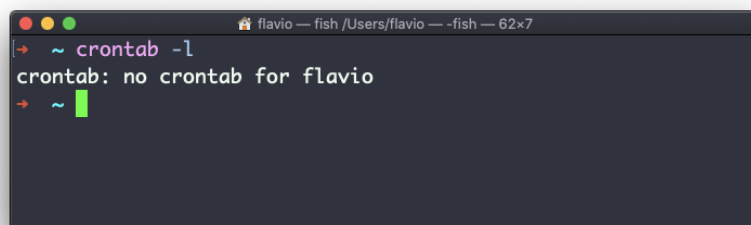
Cron jobs are jobs that are scheduled to run at specific intervals. You might have a command perform something every hour, or every day, or every 2 weeks. Or on weekends. They are very powerful, especially on servers to perform maintenance and automations.

The `crontab` command is the entry point to work with cron jobs.

The first thing you can do is to explore which cron jobs are defined by you:

```
crontab -l
```

You might have none, like me:

A terminal window with a dark background. The title bar shows 'flavio — fish /Users/flavio — fish — 62x7'. The prompt is '~' followed by a green cursor. The command 'crontab -l' has been entered. The output is 'crontab: no crontab for flavio'. The prompt '~' is shown again with a green cursor.

Run

```
crontab -e
```

to edit the cron jobs, and add new ones.

By default this opens with the default editor, which is usually `vim`. I like `nano` more, you can use this line to use a different editor:

```
EDITOR=nano crontab -e
```

Now you can add one line for each cron job.

The syntax to define cron jobs is kind of scary. This is why I usually use a website to help me generate it without errors: <https://crontab-generator.org/>

The screenshot shows the Crontab Generator website in a web browser. The page has a title "Crontab Generator" and a subtitle "Get frustrated with Cron on your server? Try our Webcron Service." Below this, there is a paragraph explaining the purpose of the site and two methods for generating cron jobs. Method 1 is using an online cron job service, and Method 2 is using the Crontab Generator website. The main section is titled "Complete the following form to generate a cron tab line" and contains five form sections: Minutes, Hours, Days, Months, and Weekday. Each section has radio buttons for different frequency options and a list of values to choose from. At the bottom, there is a "Command To Execute" field and a "Command Examples" section with a sample PHP script.

Crontab Generator

Get frustrated with Cron on your server? Try our [Webcron Service](#).

If you want to periodically perform a task (e.g. sending Emails, backing up database, doing regular maintenance, etc.) at specified times and dates, there are two ways to set scheduled tasks:

Method 1: Use our [online cron job service](#) that will save you a headache.

Method 2: Use Cron available in Unix/Linux systems.

If you go with method 2, the following generator can help you produce a crontab syntax that you can copy & paste to your crontab file (You can open the file by using command `crontab -e`). Below the generated crontab syntax, a list of run times will be displayed too. The predictions will help you ensure that you set the time and date right.

Complete the following form to generate a crontab line

Ctrl-click (or command-click on the Mac) to select multiple entries

Minutes

☒ Every Minute ☐ Even Minutes ☐ Odd Minutes ☐ Every 5 Minutes ☐ Every 15 Minutes ☐ Every 30 Minutes

0 1 2 3 4 5 6 7 8 9

Hours

☒ Every Hour ☐ Even Hours ☐ Odd Hours ☐ Every 6 Hours ☐ Every 12 Hours

Midnight 1am 2am 3am 4am 5am 6am 7am 8am 9am

Days

☒ Every Day ☐ Even Days ☐ Odd Days ☐ Every 5 Days ☐ Every 10 Days ☐ Every Half Month

1 2 3 4 5 6 7 8 9 10

Months

☒ Every Month ☐ Even Months ☐ Odd Months ☐ Every 4 Months ☐ Every Half Year

Jan Feb Mar Apr May Jun Jul Aug Sep Oct

Weekday

☒ Every Weekday ☐ Monday-Friday ☐ Weekend Days

Sun Mon Tue Wed Thu Fri Sat

Command To Execute

Command Examples:

Execute PHP script:

`/usr/bin/php /home/username/public_html/cron.php`

You pick a time interval for the cron job, and you type the command to execute.

I chose to run a script located in `/Users/flavio/test.sh` every 12 hours. This is the crontab line I need to run:

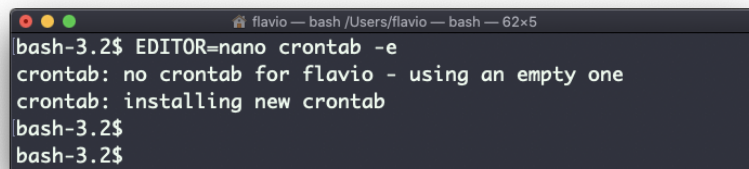
```
* */12 * * * /Users/flavio/test.sh >/dev/null 2>&1
```

I run `crontab -e` :

```
EDITOR=nano crontab -e
```

and I add that line, then I press `ctrl-X` and press `y` to save.

If all goes well, the cron job is set up:

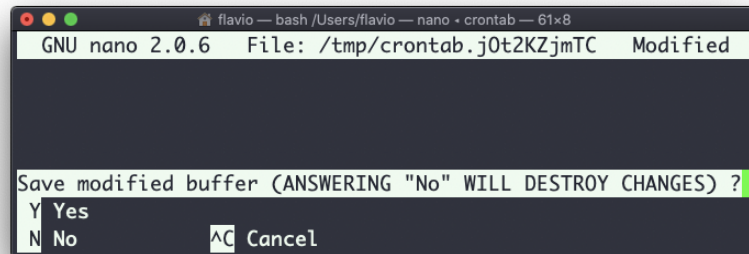
A terminal window titled 'flavio — bash /Users/flavio — bash — 62x5'. The prompt is 'bash-3.2\$'. The user enters 'EDITOR=nano crontab -e'. The terminal output shows 'crontab: no crontab for flavio - using an empty one' and 'crontab: installing new crontab'. The prompt returns to 'bash-3.2\$'.

Once this is done, you can see the list of active cron jobs by running:

```
crontab -l
```

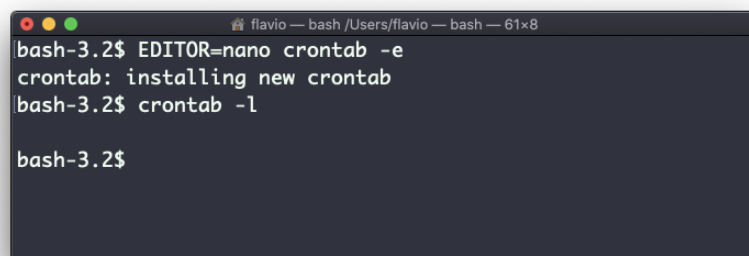
A terminal window titled 'flavio — bash /Users/flavio — bash — 61x8'. The prompt is 'bash-3.2\$'. The user enters 'crontab -l'. The terminal output shows the cron job: '* */12 * * * /Users/flavio/test.sh >/dev/null 2>&1'. The prompt returns to 'bash-3.2\$' with a green cursor.

You can remove a cron job running `crontab -e` again, removing the line and exiting the editor:



```
flavio — bash /Users/flavio — nano • crontab — 61x8
GNU nano 2.0.6 File: /tmp/crontab.j0t2KZjmTC Modified

Save modified buffer (ANSWERING "No" WILL DESTROY CHANGES) ?
Y Yes
N No ^C Cancel
```



```
flavio — bash /Users/flavio — bash — 61x8
bash-3.2$ EDITOR=nano crontab -e
crontab: installing new crontab
bash-3.2$ crontab -l

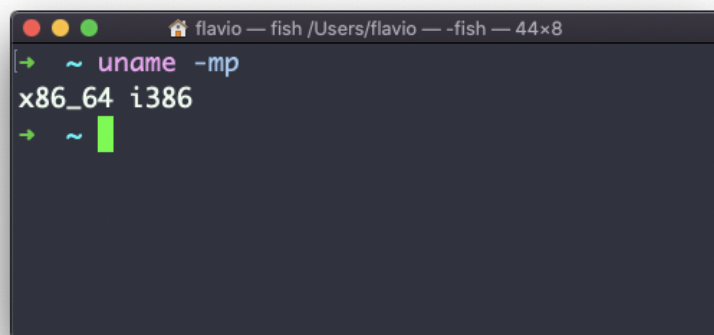
bash-3.2$
```

uname

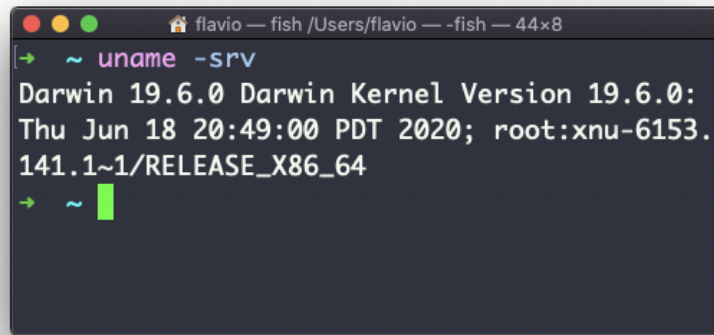
Calling `uname` without any options will return the Operating System codename:

A terminal window titled 'flavio — fish /Users/flavio — -fish — 44x8'. The prompt is '~'. The user enters 'uname' and the output is 'Darwin'. The prompt is now '~ '.

The `m` option shows the hardware name (`x86_64` in this example) and the `p` option prints the processor architecture name (`i386` in this example):

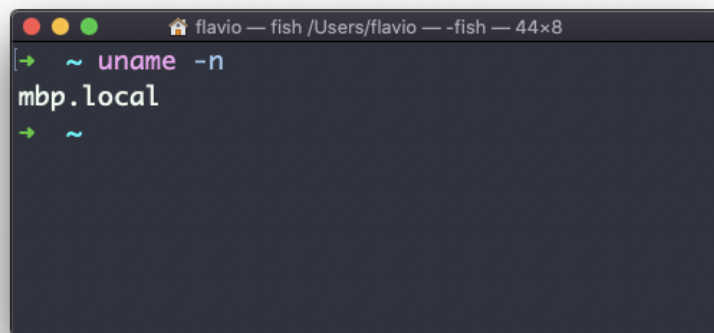
A terminal window titled 'flavio — fish /Users/flavio — -fish — 44x8'. The prompt is '~'. The user enters 'uname -mp' and the output is 'x86_64 i386'. The prompt is now '~ '.

The `s` option prints the Operating System name. `r` prints the release, `v` prints the version:



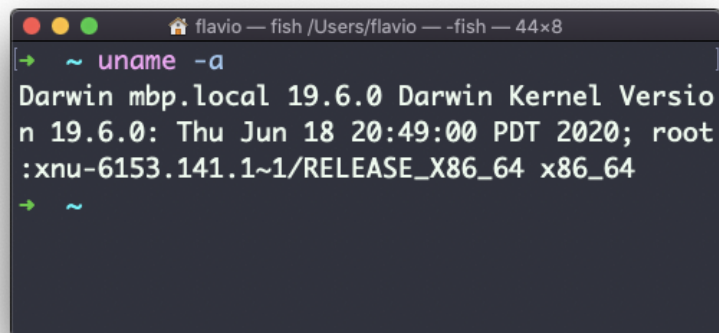
```
flavio — fish /Users/flavio — -fish — 44x8
→ ~ uname -srv
Darwin 19.6.0 Darwin Kernel Version 19.6.0:
Thu Jun 18 20:49:00 PDT 2020; root:xnu-6153.
141.1~1/RELEASE_X86_64
→ ~
```

The `n` option prints the node network name:



```
flavio — fish /Users/flavio — -fish — 44x8
→ ~ uname -n
mbp.local
→ ~
```

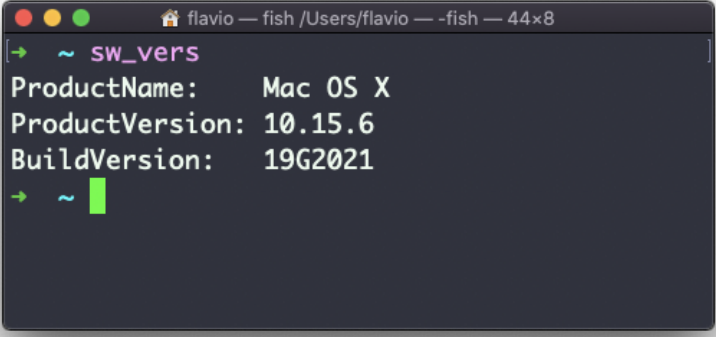
The `a` option prints all the information available:



```
flavio — fish /Users/flavio — -fish — 44x8
→ ~ uname -a
Darwin mbp.local 19.6.0 Darwin Kernel Versio
n 19.6.0: Thu Jun 18 20:49:00 PDT 2020; root
:xnu-6153.141.1~1/RELEASE_X86_64 x86_64
→ ~
```

On macOS you can also use the `sw_vers` command to print more information about the macOS Operating System. Note that this differs from the Darwin (the Kernel) version, which above is `19.6.0` .

Darwin is the name of the kernel of macOS. The kernel is the "core" of the Operating System, while the Operating System as a whole is called macOS. In Linux, Linux is the kernel, GNU/Linux would be the Operating System name, although we all refer to it as "Linux"

A terminal window with a dark background and light text. The window title bar shows 'flavio — fish /Users/flavio — -fish — 44x8'. The prompt is '~'. The command 'sw_vers' has been entered and executed. The output is: 'ProductName: Mac OS X', 'ProductVersion: 10.15.6', and 'BuildVersion: 19G2021'. The prompt is now '~' with a green cursor.

```
flavio — fish /Users/flavio — -fish — 44x8
→ ~ sw_vers
ProductName:    Mac OS X
ProductVersion: 10.15.6
BuildVersion:   19G2021
→ ~
```

env

The `env` command can be used to pass environment variables without setting them on the outer environment (the current shell).

Suppose you want to run a Node.js app and set the `USER` variable to it.

You can run

```
env USER=flavio node app.js
```

and the `USER` environment variable will be accessible from the Node.js app via the Node `process.env` interface.

You can also run the command clearing all the environment variables already set, using the `-i` option:

```
env -i node app.js
```

In this case you will get an error saying `env: node: No such file or directory` because the `node` command is not reachable, as the `PATH` variable used by the shell to look up commands in the common paths is unset.

So you need to pass the full path to the `node` program:

```
env -i /usr/local/bin/node app.js
```

Try with a simple `app.js` file with this content:

```
console.log(process.env.NAME)
console.log(process.env.PATH)
```

You will see the output being

```
undefined
undefined
```

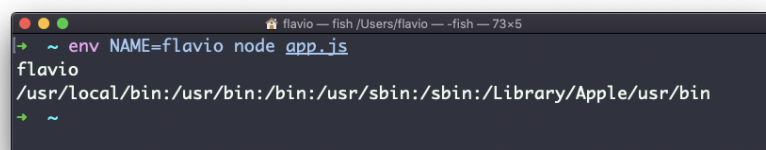
You can pass an env variable:

```
env -i NAME=flavio node app.js
```

and the output will be

```
flavio
undefined
```

Removing the `-i` option will make `PATH` available again inside the program:

A terminal window with a dark background. The title bar shows 'flavio — fish /Users/flavio — -fish — 73x5'. The prompt is '~'. The command entered is 'env NAME=flavio node app.js'. The output shows 'flavio' on the first line and the full PATH on the second line: '/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/Library/Apple/usr/bin'. The prompt is '~'.

The `env` command can also be used to print out all the environment variables, if ran with no options:

```
env
```


it will return a list of the environment variables set, for example:

```
HOME=/Users/flavio
LOGNAME=flavio
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/l
PWD=/Users/flavio
SHELL=/usr/local/bin/fish
```

You can also make a variable inaccessible inside the program you run, using the `-u` option, for example this code removes the `HOME` variable from the command environment:

```
env -u HOME node app.js
```

printenv

A quick guide to the `printenv` command, used to print the values of environment variables

In any shell there are a good number of environment variables, set either by the system, or by your own shell scripts and configuration.

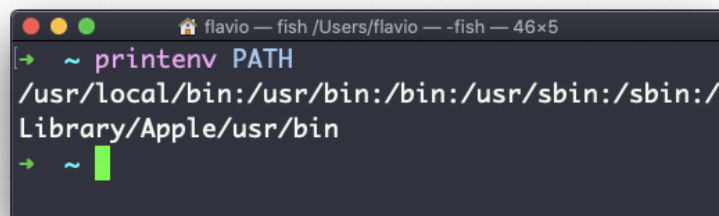
You can print them all to the terminal using the `printenv` command. The output will be something like this:

```
HOME=/Users/flavio
LOGNAME=flavio
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/l
PWD=/Users/flavio
SHELL=/usr/local/bin/fish
```

with a few more lines, usually.

You can append a variable name as a parameter, to only show that variable value:

```
printenv PATH
```



Conclusion

Thanks a lot for reading this book.

I hope it will inspire you to know more about Linux and its capabilities.

To learn more, check out my blog flaviocopes.com.

Send any feedback, errata or opinions at hey@flaviocopes.com

Linux Tutorial

Version 1.21

Jon Wakelin, Liam Gretton, Gary Gilchrist, Teri Forey, University of Leicester.

*Adapted from Michael Stonebank's original course
['UNIX Tutorial for beginners'](#)*

This tutorial has been adapted to make use of the University of Leicester HPC facilities SPECTRE and ALICE. If you use either of these facilities for research work which results in a publication you should acknowledge this with one of the following statements:

This research used the ALICE High Performance Computing Facility at the University of Leicester

or

This research used the SPECTRE High Performance Computing Facility at the University of Leicester

Tutorial One

1.1 Listing files and directories (ls)

When you first login, your current working directory is your home directory. Your home directory has the same name as your user-name, for example, *nye1*, and it is where your personal files and subdirectories are saved.

To find out what is in your home directory type

```
ls
```

The ls command lists the contents of your current working directory.

However, it does not cause all the files in your home directory to be listed, but only those ones whose name does not begin with a dot (.) Files beginning with a dot (.) are known as hidden files and usually contain important program configuration information. They are hidden because you should not change them unless you are familiar with Linux.

To list all files in your home directory including those whose names begin with a dot, type

```
ls -a
```

ls is an example of a command which can take options: -a is an example of an option. The options change the behaviour of the command. There are online manual pages that tell you what options a particular command can take, and how each option modifies the behaviour of the command. The online manual command is covered in tutorial 4.3.

```
ls -l  
ls -lt  
ls -ls  
ls -lrs  
ls -lrt
```

1.2 Making Directories (mkdir)

We will now make a subdirectory in your home directory to hold the files you will be creating and using in the course of this tutorial. To make a subdirectory called unixstuff in your current working directory type

```
mkdir unixstuff
```

To see the directory you have just created, type

```
ls
```

1.3 Changing to a different directory (cd)

The command `cd directory` means change the current working directory to 'directory'. The current working directory may be thought of as the directory you are in, i.e. your current position in the file-system tree.

To change to the directory you have just made, type

```
cd unixstuff
```

Type `ls` to see the contents (which should be empty)

Exercise 1a

Make another directory inside the **unixstuff** directory called **backups**

1.4 The directories `.` and `..`

Still in the **unixstuff** directory, type

```
ls -a
```

As you can see, in the **unixstuff** directory (and in all other directories), there are two special directories called `.` and `..`.

In Linux `.` means the current directory, so typing

```
cd .
```

There is a space between `cd` and the dot. There is normally always a space between the command and the argument.

This may not seem very useful at first, but using `(.)` as the name of the current directory will save a lot of typing, as we shall see later in the tutorial. `(..)` means the parent of the current directory, so typing

```
cd ..
```

will take you one directory up the hierarchy (back to your home directory). Try it now.

Typing `cd` with no argument always returns you to your home directory. This is very useful if you are lost in the file system.

1.5 Pathnames (pwd)

Pathnames enable you to work out where you are in relation to the whole file-system. For example, to find out the absolute pathname of your home-directory, type `cd` to get back to your home-directory and then type

```
pwd
/home/n/nye1
```

Exercise 1b

Use the commands `ls`, `pwd` and `cd` to explore the file system.

(Remember, if you get lost, type `cd` by itself to return to your home-directory)

1.6 More about home directories and pathnames

Understanding pathnames

First type `cd` to get back to your home-directory, then type

```
ls unixstuff
```

to list the contents of your `unixstuff` directory. Now type

```
ls backups
backups: No such file or directory
```

This is simply because you have not created a directory called `backups`.

Now, create a sub-directory of `unixstuff` named `backups`:

```
cd unixstuff/
mkdir backups
ls backups/
```

Note that it is not necessary to be in the `unixstuff` directory to create a subdirectory of it. A quicker alternative would be:

```
mkdir unixstuff/backups
ls unixstuff/backups
```

~ (your home directory)

Home directories can also be referred to by the tilde ~ character. It can be used to specify paths starting at your home directory. So typing

```
ls ~/unixstuff
```

will list the contents of your unixstuff directory, no matter where you currently are in the file system.

What do you think the following would list?

```
ls ~
```

What do you think the following would list?

```
ls ../..
```

1.7 Shell Shortcuts for bash

```
Ctrl-A (jump to start of line)
Ctrl-E (jump to end of line)
Ctrl-K (delete (kill) everything from the cursor onwards)
Ctrl-W (delete the previous word only)
Ctrl-Y (paste whatever was just deleted)
Ctrl-C (kill/exit a running process)
Ctrl-L (clear the screen)
Ctrl-R (search for previously executed commands)
Tab (auto-complete command or file/directory name)
↑ / ↓ (scroll back / forwards through previously entered commands)
```

Summary

ls	list files and directories
ls -a	list all files and directories
mkdir	make a directory
cd <i>directory</i>	change to named directory
cd	change to home-directory
cd ~	change to home-directory
cd ..	change to parent directory
pwd	display the path of the current directory

Tutorial Two

2.1 Copying Files and Directories (cp)

`cp file1 file2` is the command which makes a copy of **file1** in the current working directory and calls it **file2**.

What we are going to do now is to take a file stored in an open access area of the file system, and use the `cp` command to copy it to your *unixstuff* directory.

First, change to your *unixstuff* directory.

```
cd ~/unixstuff
```

Then at the shell prompt type:

```
cp /cm/shared/training/tutorial/science.txt .
```

Don't forget the dot (.) at the end. Remember, in UNIX, the dot means the current directory. The above command means copy the file **science.txt** to the current directory, keeping the name the same.

Directories can also be copied with the `cp` command, but it's necessary to add the option **-R** to do so. This option means 'recursive' and will copy the contents of the directory as well as the directory itself, **for example**:

```
cp -R directory1 directory2
```

Try running

```
cp -R /cm/shared/training/tutorial ~/unixstuff
```

Exercise 2a

Create a backup of your **science.txt** file by copying it to a file called **science.bak**

2.2 Moving files and Directories (mv)

The move command has a variety of similar but subtly different uses. It can be used to move a file to a different location (i.e. a different directory). It can also be used to move *multiple* files to a different directory. It can also be used to rename a file or a directory. **For example**:

```
mv file1 directory1/
```

This would move *file1* from the current directory into *directory1*.

```
mv file1 file2 file3 directory1/
```

This would move *file1*, *file2* and *file3* from the current directory into *directory1*.

```
mv file1 file2
```

This would rename *file1* as *file2*.

```
mv directory1/ directory2/
```

This would rename a directory. Finally,

```
mv file1 directory/file2
```

This would move *and* rename a file in one step.

We are now going to move the file *science.bak* to your backup directory. First, change directories to your *unixstuff* directory (can you remember how?). Then, inside the **unixstuff** directory, type

```
mv science.bak backups/
```

To see if it worked type

```
ls  
ls backups
```

2.3 Removing Files (rm) and Directories (rmdir)

To delete (remove) a file, use the **rm** command. As an example, we are going to create a copy of the **science.txt** file then delete it.

Inside your **unixstuff** directory, type

```
cp science.txt tempfile.txt  
ls  
rm tempfile.txt  
ls
```

In order to delete an empty directory you can use the command

```
rmdir directory
```

However this won't remove directories that already have files in them, instead you can use

```
rm -r directory
```

to recursively delete files in directory (use sparingly - **there is no Recycle bin!**)

You can use the *rmdir* command to remove a directory (make sure it is empty first). Try to remove the **backups** directory. You will not be able to since Linux will not let you remove a non-empty directory.

Exercise 2b

Create a directory called **tempstuff** using *mkdir*, then remove it using the *rmdir* command.

2.4 Displaying the contents of a file on the screen

clear (clear screen)

Before you start the next section, you may like to clear the terminal window of the previous commands so the output of the following commands can be clearly understood.

At the prompt, type

```
clear
```

This will clear all text and leave you with the prompt at the top of the window.

cat (concatenate)

The command *cat* can be used to display the contents of a file on the screen. Type:

```
cat science.txt
```

As you can see, the file is longer than the size of the window, so it scrolls past making it unreadable.

less

The command *less* writes the contents of a file onto the screen a page at a time. Type

```
less science.txt
```

Press the space bar if you want to see another page, type *q* if you want to quit reading. As you can see, *less* is used in preference to *cat* for long files.

head

The *head* command writes the first ten lines of a file to the screen. First clear the screen then type

```
head science.txt
```

Then type

```
head -5 science.txt
```

What difference did the -5 do to the *head* command?

tail

The *tail* command writes the last ten lines of a file to the screen. Clear the screen and type

```
tail science.txt
```

How can you view the last 15 lines of the file?

2.5 Searching the contents of a file

Simple searching using less

Using *less*, you can search through a text file for a keyword (pattern). For example, to search through **science.txt** for the word 'science', type

```
less science.txt
```

then, still in *less* (i.e. don't press **q** to quit), type a forward slash (*/*) followed by the word to search for, e.g.

```
/science
```

As you can see, *less* finds and highlights the keyword. Type **n** to search for the next occurrence of the word.

grep

grep is one of many standard UNIX utilities. It searches files for specified words or patterns. First clear the screen, then type

```
grep science science.txt
```

As you can see, *grep* has printed out each line that contains the word science. Or has it?

Try typing

```
grep science science.txt
```

The *grep* command is case sensitive; it distinguishes between *Science* and *science*.

To ignore upper/lower case distinctions, use the *-i* option, i.e. type

```
grep -i science science.txt
```

Often when there is a lot of text it is useful to highlight the matches (this is a default setting on ALICE / SPECTRE now but may not be on other systems)

```
grep --color -i science science.txt
```

To search for a phrase or pattern, you must enclose it in single quotes (the apostrophe symbol). For example to search for the phrase *spinning top*, type

```
grep -i 'spinning top' science.txt
```

Some of the other options of *grep* are:

- v display those lines that do NOT match
- n precede each matching line with the line number
- c print only the total count of matched lines

Try some of them and see the different results. Don't forget, you can use more than one option at a time, for example, the number of lines without the words *science* or *Science* is

```
grep -ivc science science.txt
```

wc (word count)

A handy little utility is the *wc* command, short for word count. To do a word count on **science.txt**, type

```
wc -w science.txt
```

To find out how many lines the file has, type

```
wc -l science.txt
```

To find out how many characters the file has, type

```
wc -m science.txt
```

Summary

<code>cp <i>file1 file2</i></code>	copy file1 and call it file2
<code>mv <i>file1 file2</i></code>	move or rename file1 to file2
<code>rm <i>file</i></code>	remove a file
<code>rmdir <i>directory</i></code>	remove a directory
<code>cat <i>file</i></code>	Display or concatenate a file
<code>less <i>file</i></code>	display a file a page at a time
<code>head <i>file</i></code>	display the first few lines of a file
<code>tail <i>file</i></code>	display the last few lines of a file
<code>grep 'keyword' <i>file</i></code>	search a file for keywords
<code>wc <i>file</i></code>	count number of lines/words/characters in file

Tutorial Three

3.1 Redirection

It is extremely common for processes initiated by Linux commands write to the standard output (that is, they write to the terminal screen), and many take their input from the standard input (that is, they read it from the keyboard). There is also the standard error, where processes write their error messages, by default, to the terminal screen.

- Standard Input (STDIN) - Usually the keyboard
- Standard Output (STDOUT) - Usually the Terminal
- Standard Error (STDERR) - Usually the Terminal

3.2 Redirecting Standard Output

We use the `>` symbol to redirect the output of a command. Many of the commands we have seen so far write their output to the terminal (for instance *cat*, *ls*, *grep*, *tail*, *head* and *wc* all write to STDOUT). However we can redirect the output of any of these commands to a file instead (the file can have any name you chose. If the file does not exist it will be created, if it does exist it will be replaced).

The command *echo* prints its arguments to standard output. Compare these two commands

```
echo "Hello world"
```

and

```
echo "Hello world" > output.txt
```

You can view the contents of your new file using

```
less output.txt
```

Exercise 3a

Create a file called **list1** using a suitable text editor (see appendices

A.3 Opening a text editor (PuTTY/SSH) and A.4 Opening a text editor (NX) for more information) containing the following items one per line, *orange*, *plum*, *mango*, *grapefruit*. Save and close your file. Now create a second file called **list2** that contains the following items: *apple*, *peach*, *grape*, *orange*. Again save and close your file. You can view your files using a command such as *cat*, *more* or *less*, for example

```
more list1
more list2
```

You should now have two files. We will use the *cat* command to join (concatenate) these files into a new file called **biglist**. Type

```
cat list1 list2 > biglist
```

this command reads the contents of **list1** and **list2** in turn, and then writes the text to the file **biglist**.

3.3 Appending data to an existing file

It was mentioned above that the redirection operator, *>*, will create a new file if one does not exist, but it will overwrite the contents of a file if the file already exists. If we want to add/append data to an existing file, rather than overwrite it, we need to use the *>>* operator instead

For example, to append a *kiwi* to the file **biglist** we would type:

```
echo "kiwi" >> biglist
cat biglist
```

You will see that a kiwi was added to the list. Now repeat this using a single *>* operator.

```
echo "Avocado" > biglist
cat biglist
```

You will see that all of the original content of the file has been lost and replaced with the word *Avocado*

3.4 Redirecting Standard Error

Standard error and standard output are very similar. Both are generally written to the terminal and it is not always obvious what is STDOUT and what is STDERR. However, STDOUT can be easily differentiated from STDERR using redirection. We redirect Standard Error to a file using the operator **2>**

3.5 Redirecting Standard Input

Similarly we can use the **<** operator to redirect STDIN. For example, the **sort** command read input from STDIN (the keyboard) and produces an alphabetically or numerically sort list. Type

```
sort
```

Then type in the names of some vegetables. Press Return after each one, and hit control-d after the last entry to return to the shell.

```
carrot
beetroot
artichoke
^d (control-d to stop)
```

The output will be

```
artichoke
beetroot
carrot
```

Instead of generating STDIN using the keyboard, we can use the **<** operator to redirect the contents of a file to STDIN. For example, to sort your list of fruit, first re-create biglist:

```
cat list1 list2 > biglist
```

then to sort it type:

```
sort < biglist
```

and the sorted list will be output to the screen.

Putting it all together: It is possible to redirect input, output and errors all in one go for example,

```
sort < biglist > sorted_list 2> errors.txt
```

In which case input is read from the file **biglist** (rather than the keyboard), output is sent to the file **sorted_list** (rather than to the terminal) and any error messages are sent to the file **errors.txt** (rather than the terminal).

3.6 Pipes

To see who is on the system with you, type

```
who
```

One method to get a sorted list of names is to type,

```
who > names.txt  
sort < names.txt
```

This is a bit slow and you have to remember to remove the temporary file called *names.txt* when you have finished. What you really want to do is connect the output of the *who* command directly to the input of the *sort* command. This is exactly what pipes do. The symbol for a pipe is the vertical bar |

The pipe / vertical bar character is usually typed with 'shift' and the key to the left of 'z' on the keyboard.

For example, typing

```
who | sort
```

will give the same result as above, but quicker and cleaner. To find out how many users are logged on, type

```
who | wc -l
```

How would you find out how many login sessions *you* have running? Hint: you will need to use *grep* from Tutorial 2.5

Summary

<i>command</i> > <i>file</i>	redirect standard output to a file
<i>command</i> 2> <i>file</i>	redirect standard error to a file
<i>command</i> >> <i>file</i>	append standard output to a file
<i>command</i> < <i>file</i>	redirect standard input from a file
<i>command1</i> <i>command2</i>	pipe the output of <i>command1</i> to the input of <i>command2</i>
<i>cat file1 file2</i> > <i>file0</i>	concatenate <i>file1</i> and <i>file2</i> to <i>file0</i>
<i>sort</i>	sort data

who

list users currently logged in

Tutorial Four

4.1 Wildcards

The characters * and ?

The character ***** is called a wildcard, and will match against none or more character(s) in a file (or directory) name. For example, in your **unixstuff** directory, type

```
ls list*
```

This will list all files in the current directory starting with **list....**

Try typing

```
ls *list
```

This will list all files in the current directory ending with **...list**

The character **?** will match exactly one character. So *ls ?ouse* will match files like **house** and **mouse**, but not **grouse**. Try typing

```
ls ?list  
ls list?
```

If you need to match a limit number of patterns you can use *{pattern1,pattern2,etc}*

```
ls list{1,2}
```

This can be used with most commands:

```
mkdir newdir{1,2,3,4,5}
```

The previous command would create 5 new directories

4.2 Filename conventions

We should note here that a directory is merely a special type of file. So the rules and conventions for naming files apply also to directories.

In naming files, characters with special meanings such as **/ * & %** , should be avoided. Also, avoid using spaces within names. The safest way to name a file is to use only alphanumeric characters, that is, letters and numbers, together with **_** (underscore) and **.** (dot).

File names conventionally start with a lower-case letter, and may end with a dot followed by a group of letters indicating the contents of the file. For example, all files consisting of C code may be named with the ending `.c`, for example, `prog1.c`. Then in order to list all files containing C code in your home directory, you need only type `ls *.c` in that directory.

4.3 Getting Help

On-line Manuals

There are on-line manuals which gives information about most commands. The manual pages tell you which options a particular command can take, and how each option modifies the behaviour of the command. Type *man command* to read the manual page for a particular command.

For example, to find out more about the `wc` (word count) command, type

```
man wc
```

Alternatively

```
whatis wc
```

gives a one-line description of the command, but omits any information about options etc.

When you are not sure of the exact name of a command,

```
man -k keyword
```

will give you the commands with keyword in their manual page header. For example, try typing

```
man -k list
```

Summary

<code>*</code>	match any number of characters
<code>?</code>	match one character
<code>man command</code>	read the online manual page for a command
<code>whatis command</code>	brief description of a command
<code>apropos keyword</code>	match commands with keyword in their man pages

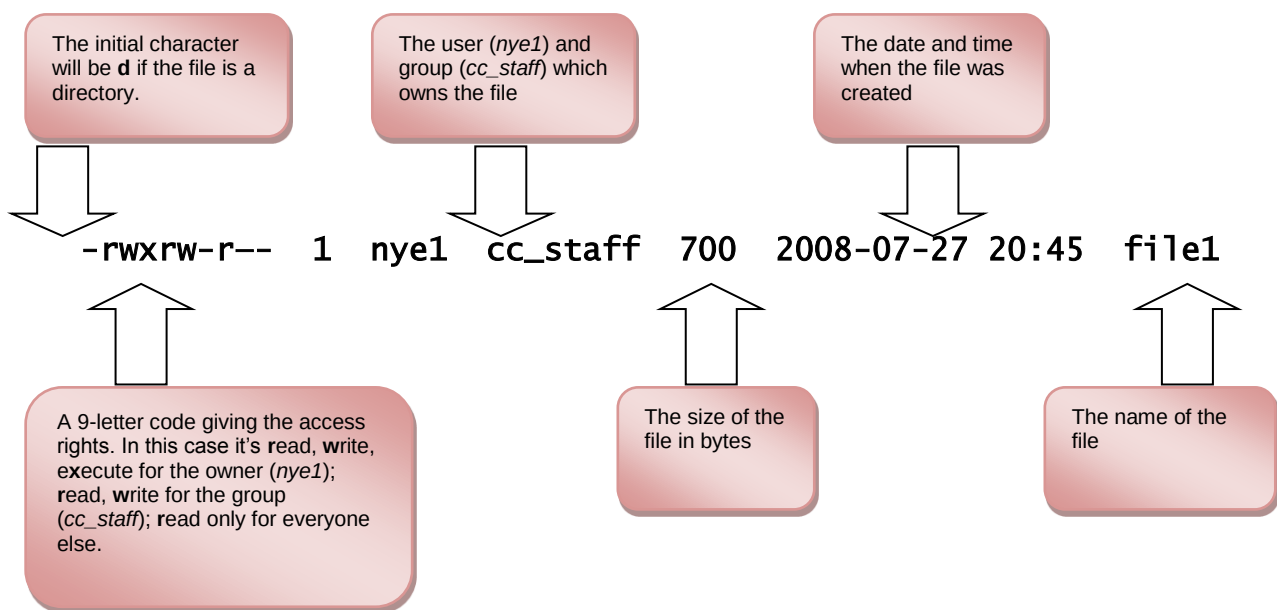
Tutorial Five

5.1 Viewing file and directory permissions

In your unixstuff directory, type

```
ls -l
```

You will see that you now get lots of details about the contents of your directory, similar to the example below.



Each file (and directory) has associated access rights, which may be found by typing `ls -l`.

In the left-hand column is a 10-symbol string consisting of the symbols `d`, `r`, `w`, `x`, `-`, and, occasionally, `s` or `S`. If `d` is present, it will be at the left hand end of the string, and indicates a directory: otherwise `-` will usually be the starting symbol of the string, indicating a normal file.

The 9 remaining symbols indicate the permissions, or access rights, and are taken as three groups of 3.

- The left group of 3 gives the file permissions for the user that owns the file (or directory) (*nye1* in the above example);
- The middle group gives the permissions for the group of people to whom the file (or directory) belongs (*cc_staff* in the above example);

- The rightmost group gives the permissions for everyone else.

The symbols *r*, *w*, etc., have slightly different meanings depending on whether they refer to a simple file or to a directory.

Access rights on files

- **r** (or -), indicates read permission (or otherwise), that is, the presence or absence of permission to read and copy the file
- **w** (or -), indicates write permission (or otherwise), that is, the permission (or otherwise) to change a file
- **x** (or -), indicates execution permission (or otherwise), that is, the permission to execute a file, where appropriate

Access rights on directories

- **r** allows users to list files in the directory;
- **w** means that users may delete files from the directory or move files into it;
- **x** means the right to access files in the directory. This implies that you may read files in the directory provided you have read permission on the individual files.

So, in order to read a file, you must have execute permission on the directory containing that file, and hence on any directory containing that directory as a subdirectory, and so on, up the tree.

Some examples

-rwxrwxrwx a file that everyone can read, write and execute (and delete).

-rw----- a file that only the owner can read and write - no-one else can read or write and no-one has execution rights.

5.2 Changing access rights (chmod)

Only the owner of a file can use *chmod* to change the permissions of a file. The options of *chmod* are as follows

Symbol Meaning

u	user
g	group
o	other
a	all
r	read
w	write (and delete)
x	execute (and access directory)
+	add permission
-	take away permission

For example, to remove read write and execute permissions on the file **biglist** for the group and others, type

```
chmod go-rwx biglist
```

This will leave the other permissions unaffected.

To give read and write permissions on the file **biglist** to all,

```
chmod a+rw biglist
```

Exercise 5a

Try changing access permissions on the file **science.txt** and on the directory **backups**

To check that the permissions have changed, use:

```
ls -l
```

5.3 Processes and Jobs

A process is an executing program identified by a unique PID (process identifier). To see information about your processes, with their associated PID and status, type

```
ps
```

A process may be in the foreground, in the background, or be suspended. In general the shell does not return the UNIX prompt until the current process has finished executing.

Some processes take a long time to run and hold up the terminal. Backgrounding a long process has the effect that the UNIX prompt is returned immediately, and other tasks can be carried out while the original process continues executing.

Running background processes

To background a process, type an **&** at the end of the command line. For example, the command *sleep* waits a given number of seconds before continuing. Type

```
sleep 10
```

This will wait 10 seconds before returning the command prompt. Until the command prompt is returned, you can do nothing except wait.

To run *sleep* in the background, type


```
sleep 10 &  
[1] 6259
```

The **&** runs the job in the background and returns the prompt straight away, allowing you to run other programs while waiting for that one to finish.

The first line in the above example is typed in by the user; the next line, indicating job number and PID, is returned by the machine. The user is notified of a job number (numbered from 1) enclosed in square brackets, together with a PID and is notified when a background process is finished. Backgrounding is useful for jobs which will take a long time to complete.

Backgrounding a current foreground process

At the prompt, type

```
sleep 100
```

You can suspend the process running in the foreground by holding down the Control key and typing **z** (written as **^z**) Then to put it in the background, type

```
bg
```

Note: do not background programs that require user interaction e.g. *nano*.

5.4 Listing suspended and background processes

When a process is running, backgrounded or suspended, it will be entered onto a list along with a job number. To examine this list, type

```
jobs
```

An example of a job list could be

```
[1] Suspended sleep 100  
[2] Running firefox  
[3] Running nedit
```

To restart (foreground) a suspended processes, type

```
fg %jobnumber
```

For example, to restart *sleep 100*, type

```
fg %1
```

Typing *fg* with no job number foregrounds the last suspended process.

5.5 Killing a process

kill (terminate or signal a process)

It is sometimes necessary to kill a process (for example, when an executing program is in an infinite loop)

To kill a job running in the foreground, type **^c** (control-c). For example, run

```
sleep 100  
^c
```

To kill a suspended or background process, type

```
kill %jobnumber
```

For example, run

```
sleep 100 &  
jobs
```

If it is job number 4, type

```
kill %4
```

To check whether this has worked, examine the job list again to see if the process has been removed.

ps (process status)

Alternatively, processes can be killed by finding their process numbers (PIDs) and using `kill PID_number`

```
sleep 100 &
ps
  PID TT S TIME COMMAND
 20077 pts/5 S  0:05 sleep 100
 21563 pts/5 T  0:00 netscape
 21873 pts/5 S  0:25 nedit
```

To kill off the process *sleep 100*, type

```
kill 20077
```

and then type `ps` again to see if it has been removed from the list. If a process refuses to be killed, uses the **-9** option, i.e. type

```
kill -9 20077
```

Note: It is not possible to kill off other users' processes!

Summary

<code>ls -lag</code>	list access rights for all files
<code>chmod [options] file</code>	change access rights for named file
<code>command &</code>	run command in background
<code>^C</code>	kill the job running in the foreground
<code>^Z</code>	suspend the job running in the foreground
<code>bg</code>	background the suspended job
<code>jobs</code>	list current jobs
<code>fg %1</code>	foreground job number 1
<code>kill %1</code>	kill job number 1
<code>ps</code>	list current processes
<code>kill 26152</code>	kill process number 26152

Tutorial Six

Other useful UNIX commands

quota

On SPECTRE / ALICE all accounts are allocated a certain amount of disk space on the file system for personal files, up to 20GB. If you go over your quota, you cannot create any more files.

To check your current quota and how much of it you have used, type

```
quotacheck
```

df

The *df* command reports on the space left on the file system. For example, to find out how much space is left on the fileserver, type

```
df .
Filesystem            1k-blocks      Used   Available Use% Mounted
on
panfs://172.16.3.1:global 933294615568 846657542400 86637073168  91% /panfs

df -h .
Filesystem            Size  Used Avail Use% Mounted on
panfs://172.16.3.1:global 870T  789T   81T  91% /panfs
```

du (disk usage)

The *du* command outputs the number of kilobytes used by each subdirectory. This is useful if you have gone over quota and can no longer log in using NX and you want to find out which directory has the most files (or alternatively, you can use the '*homeusage*' command). In your home-directory, type

```
du *
du -s *
du -sh *
```

homeusage

The *homeusage* command will do the same as running *du -sh ** in your home directory but will output in ascending order of size to make it easy to see where you are using space. You do not need to be in your home directory to run this command.

gzip

This command compresses a file. For example, to compress *science.txt*, type

```
gzip science.txt
```

This will compress the file and place it in a file called *science.txt.gz*. To uncompress the file, use the *gunzip* command.

```
gunzip science.txt.gz
```

file

file classifies the named files according to the type of data they contain, for example *ascii* (text), pictures, compressed data, etc.. To report on all files in your home directory. It can be useful to determine what sort of data a file contains in cases where the file name doesn't give a hint. Type

```
file filename
```

history

The shell keeps an ordered list of all the commands that you have entered. Each command is given a number according to the order it was entered.

```
history
```

You can use the exclamation character (!) to recall commands easily.

```
!!      # recall last command
!-3     # recall third most recent command
!5      # recall 5th command in list
!grep   # recall last command starting with grep
```

You can increase the size of the history buffer by typing

```
HISTSIZE=1000
```

find

find is a powerful but rather complicated command for finding files. By default it searches recursively from the directory specified.

The first argument to the file command is the directory to start searching from. In its simplest form the command then needs a name of an object to search for, and this

must be specified as an argument to the **-name** option. The following example will look for any object called *file1*, and will start searching from the current working directory:

```
find . -name file1
```

To find all objects beginning with *file*, a wildcard can be used, but it must be quoted:

```
find . -name "file*"
```

locate

locate is a very quick way of finding files on a large system. It performs a similar role to the *find* command but works in a very different way. *find* looks through the file system until it finds your files (which can be slow but is almost always correct); *locate* on the other hand searches a database in which the locations of files are maintained. This is far quicker but doesn't reflect very recent changes to the file system, because the database is usually only updated once a day.

```
locate filename  
locate -i filename  
locate -r filename
```

If your home directory is mounted on a shared filesystem such as NFS, then the database which the *locate* command queries may not include your home directory.

wget

wget is a web client (not a browser). It can be used download files from web and ftp sites:

```
wget URL  
wget http://www.ee.surrey.ac.uk/Teaching/Unix/science.txt  
wget -O sci.txt http://www.ee.surrey.ac.uk/Teaching/Unix/science.txt
```

Tutorial Seven

7.1 Variables

Variables are a way of passing information from the shell to programs when you run them. Programs look "in the environment" for particular variables and if they are found will use the values stored. Some are set by the system, others by you, yet others by the shell, or any program that loads another program.

7.2 Environment Variables

An example of an environment variable is the *OSTYPE* variable. The value of this is the current operating system you are using. When using variables it's necessary to refer to them with a **\$** sign at the start so that the shell knows that we are referring to a variable. Type

```
echo $OSTYPE
```

More examples of environment variables are

- \$USER - Your login name
- \$HOME - Path name of your home directory
- \$HOSTNAME - Name of the computer you are using
- \$PATH - Directories the shell searches to find commands
- \$SHELL – The shell you are using (should be bash!)

Environment variables are displayed using the *env* command. To show all values of these variables, type

```
env | less
```

7.3 Setting variables

You can set shell variables using the **=** operator. For example, to change the number of shell commands saved in the history list, you need to set the shell variable HISTSIZE. It is set to 1000 by default on SPECTRE, but you can increase this if you wish.

```
HISTSIZE=2000
```

Check this has worked by typing

```
echo $HISTSIZE
```

However, this will only set the variable for the current shell - it will be lost once you log out. You should also be aware that the new value of the variable will

1. Not be picked up by any forked processes, including sub-shells
2. Not be picked up by any new sessions that you start

To address the first issue, you can export the variable – this means that forked processes and sub-shells will inherit the variables,

```
export HISTSIZE=2000
```

To address the second issue (i.e to make the changes permanent) you will need to add the above command to your **.bashrc** file.

First open the **.bashrc** file in *nano* (or another suitable text editor – (see appendices

*A.3 Opening a text editor (PuTTY/SSH) and A.4 Opening a text editor (NX) for more information)). If you have connected using PuTTY/SSH, use *nano*:*

```
nano ~/.bashrc
```

Add the following line to your **.bashrc** file (it doesn't matter where within the file as long as it's on a line of its own):

```
export HISTSIZE=2000
```

Save the file and force the shell to reread its **.bashrc** file by using the shell *source* command:

```
source ~/.bashrc
```


Alternatively you could log out and then start a new shell. Finally, check this has worked by typing

```
echo $HISTSIZE
```

7.4 Setting the path

When you type a command, your PATH variable defines in which directories the shell will look to find the command you typed. It is a colon-separated list of directories, and on SPECTRE can be very long, reflecting the number of applications and program modules you may have loaded.

```
echo $PATH
```

will show a long list of directories.

If the system returns a message saying *command: command not found*, this indicates that either the command doesn't exist at all on the system or it is simply not in your path.

For example, to run *units* which we will compile and install in Tutorial Eight you either need to directly specify the units path (*~/units174/bin/units*), or you need to have the directory *~/units174/bin* in your path. Come back to the rest of Tutorial 7.4 once you have completed Tutorial Eight.

You can add it to the end of your existing path (the **\$PATH** represents this) by issuing the command:

```
export PATH=$PATH:~/units174/bin
```

Don't forget the colon, which separates the existing list of directories from the one you are adding to the list.

Test that this worked by trying to run *units* in any directory other than where *units* is actually located.

```
cd; units
```

HINT: You can run multiple commands on one line by separating them with a semicolon.

To add this path **permanently**, add the following line to your *.bashrc* list of other commands.

```
PATH=$PATH:~/units174/bin
```

which

The which command shows you the full path to a command (provided that the file is in the path)

```
which command  
which wget
```

If there are multiple programs with the same name, you can use:

```
which -a command
```

to list them all. However you should realize that if there are multiple programs with the same name in your path only the one listed first will be executed.

Tutorial Eight

8.1 Compiling software packages

We have many public domain and commercial software packages installed on SPECTRE which are available to all users. However, users are allowed to download and install small software packages in their own home directory, software usually only useful to them personally.

There are a number of steps needed to install the software.

- Locate and download the source code (which is usually compressed)
- Unpack the source code
- Compile the code
- Install the resulting executable
- Set paths to the installation directory

Of the above steps, probably the most difficult is the compilation stage.

Compiling Source Code

All high-level language code must be converted into a form the computer understands. For example, C language source code is converted into a one or more object files containing low-level machine code. The final stage in compiling a program involves linking the object files to libraries which contain certain built-in functions. This final stage produces an executable program, code which the computer's CPU can execute directly.

To do all these steps by hand is complicated and beyond the capability of the ordinary user. A number of utilities and tools have been developed for programmers and end-users to simplify these steps.

make and the Makefile

The *make* command allows programmers to manage large programs or groups of programs. It aids in developing large programs by keeping track of which portions of the entire program have been changed, compiling only those parts of the program which have changed since the last compile.

The *make* program gets its set of compile rules from a text file called **Makefile** which resides in the same directory as the source files. It contains information on how to compile the software, e.g. the optimisation level, whether to include debugging info in the executable. It also contains information on where to install the finished compiled binaries (executables), manual pages, data files, dependent library files, configuration files, etc.

Some packages require you to edit the Makefile by hand to set the final installation directory and any other parameters. However, many packages are now distributed with the GNU configure utility.

configure

As the number of UNIX variants increased, it became harder to write programs which could run on all variants. Developers frequently did not have access to every system, and the characteristics of some systems changed from version to version. The GNU configure and build system simplifies the building of programs distributed as source code. All programs are built using a simple, standardised, two-step process. The program builder need not install any special tools in order to build the program.

The configure shell script attempts to guess correct values for various system-dependent variables used during compilation. It uses those values to create a **Makefile** in each directory of the package.

The simplest way to compile a package is:

1. **cd** to the directory containing the package's source code.
2. Type **./configure** to configure the package for your system.
3. Type **make** to compile the package.
4. Optionally, type **make check** to run any self-tests that come with the package.
5. Type **make install** to install the programs and any data files and documentation.
6. Optionally, type **make clean** to remove the program binaries and object files from the source code directory

The *configure* utility supports a wide variety of options. There is usually a help option available to get a list of interesting options for a particular configure script.

```
./configure --help
```

The only generic options you are likely to use are the `--prefix` and `--exec-prefix` options. These options are used to specify the installation directories.

The directory named by the `--prefix` option will hold machine independent files such as documentation, data and configuration files.

The directory named by the `--exec-prefix` option, (which is normally a subdirectory of the `--prefix` directory), will hold machine dependent files such as executables.

8.2 Downloading source code

For this example, we will download a piece of free software that converts between different units of measurements.

First create a new directory then copy the software and save it to your new directory.

```
mkdir download
cd download
cp /cm/shared/training/tutorial/units-1.74.tar.gz .
```

8.3 Extracting the source code

Make sure you are within the **download** directory and list the contents.

```
cd ~/download
ls -l
```

As you can see, the filename ends in **.tar.gz**. This is a common file format for distributing software packages in source form. It comprises a tar file which has been compressed with gzip (tar.gz files are often called *tarballs*). A tar file is a collection of directories and files packaged as a single file with the *tar* command. Sometimes files of this type are named ending with **.tgz**

First uncompress the file using the gunzip command. This will create a .tar file.

```
gunzip units-1.74.tar.gz
```

Then extract the contents of the tar file.

```
tar -xvf units-1.74.tar
```

Alternatively the two steps can be combined into a single command:

```
tar -zxvf units-1.74.tar.gz
```

Notice the extra **-z** flag. This instructs tar to gunzip the file before unpacking the archive. Again, list the contents of the **download** directory, then go to the **units-1.74** sub-directory.

```
cd units-1.74
```

8.4 Configuring the package

The first thing to do is carefully read the **README** and **INSTALL** text files (use the *less* command). These contain important information on how to compile and run the software.

The units package uses the GNU configure system to compile the source code. We will need to specify the installation directory, since the default will be the main system area which you will not have write permissions for. We need to create an install directory in your home directory.

```
mkdir ~/units174
```

Then run the configure utility setting the installation path to this.

```
./configure --prefix=$HOME/units174
```

If configure has run correctly, it will have created a Makefile with all necessary options. You can view the Makefile if you wish (use the *less* command), but do not edit the contents of this.

8.5 Building the package

Now you can go ahead and build the package by running the make command.

```
make
```

After a minute or two (depending on the speed of the computer), the executables will be created. You can check to see everything compiled successfully by typing

```
make check
```

If everything is okay, you can now install the package.

```
make install
```

This will install the files into the **~/units174** directory you created earlier. It is important to realize that while the

```
./configure  
make  
make check  
make install
```

sequence is extremely common, it is not a standard and there is no absolute guarantee that software will install this way. However, the *make* command is a standard Linux command which reads in a user written file (the makefile) that describes how to build and install software. If you are writing your own software (and in particular if you are distributing your software) you should look to use the make command - although its use is beyond the scope of this course. The configure command is more in-depth still and will create a makefile for a given system from a template file, you will probably not need to create your own *configure* scripts unless you are working on developing a very large software application and intended to distribute it widely.

8.6 Running the software

You are now ready to run the software (assuming everything worked).

```
cd ~/units174
```

If you list the contents of the units directory, you will see a number of subdirectories.

```
bin    The binary executables
info   GNU info formatted documentation
man    Man pages
share  Shared data files
```

To run the program, change to the **bin** directory and type

```
./units
You have: 6 feet
You want: metres
* 1.8288
/ 0.54680665
(ctrl-d to exit)
```

If you get the answer 1.8288, congratulations, it worked. To view what units it can convert between, view the data file in the share directory (the list is quite comprehensive). To read the full documentation, change into the **info** directory and type

```
info --file=units.info
```

8.7 Stripping unnecessary code

When a piece of software is being developed, it is useful for the programmer to include debugging information into the resulting executable. This way, if there are problems encountered when running the executable, the programmer can load the executable into a debugging software package and track down any software bugs.

This is useful for the programmer, but unnecessary for the user. We can assume that the package, once finished and available for download has already been tested and debugged. However, when we compiled the software above, debugging information was still compiled into the final executable. Since it is unlikely that we are going to need this debugging information, we can strip it out of the final executable. One of the advantages of this is a much smaller executable, which should run slightly faster.

What we are going to do is look at the before and after size of the binary file. First change into the **bin** directory of the *units* installation directory.

```
cd ~/units174/bin  
ls -l
```

As you can see, the file is over 100kB in size. You can get more information on the type of file by using the *file* command.

```
file units  
units: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), for GNU/Linux  
2.6.4, dynamically linked (uses shared libs), not stripped
```

To strip all the debug and line numbering information out of the binary file, use the strip command

```
strip units  
ls -l
```

As you can see, the file is now 45 kB – less than half its original size. Half of the binary file was debug code! Check the file information again.

```
file units  
units: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), for GNU/Linux  
2.6.4, dynamically linked (uses shared libs), stripped
```

Notice now that the file is **stripped**.

Tutorial Nine

Automation of commands using shell scripts

Once you have become used to basic commands and finding your way around a Linux system you may want to automate tasks which are executed frequently. This can be done with shell scripts – lists of commands to run in sequence. This is a simple but powerful form of programming which with loops and conditional statements allows complex sequences of commands to be run, saving much time.

9.1 Basic shell script

Create and edit a file called *test.sh*:

```
#!/bin/bash  
  
echo "Hello $USER"
```

The line at the beginning is called a shebang, sha-bang, hashbang, pound-bang or hash-pling and tells the computer that the script should be run in a bash shell. The script knows that everything beginning with a dollar sign (\$) is a variable and will use the value of the environment variable \$USER (see section 7.2)

Once you have finished editing the file, close the editor and make the file executable with 'chmod', which you used in section 5.2

```
chmod +x test.sh
```

Before you run the script, what do you think the output will be?

Now run the script:

```
./test.sh
```

9.2 Using variables

As well as using environment variables, new variables can be created and used within your script. The example below uses a new variable, NUMLOGINS.

Note that you do not need to use the dollar sign when you create the variable, only to refer to it once it has been created. The \$(*command*) construct is used in this instance to pass the output of a command or commands and assign the value to the new variable.

```
#!/bin/bash

echo "Hello $USER"
NUMLOGINS=$(who | grep $USER | wc -l)
echo "You have $NUMLOGINS login sessions"
```

9.3 Looping with 'for'

Rather than simply process a sequence of commands one after the other, loops can perform the same commands for different variables. The example below uses the variable `$user` which is set up at the beginning of the 'for' loop. Note that this is not the same as the `$USER` environment variable as variable names are case sensitive.

```
#!/bin/bash

for user in gg78 ljg2 root
do
    NUMLOGINS=$(who | grep $user | wc -l)
    echo "$user has $NUMLOGINS sessions"
done
```

The script loops over the section between 'do' and 'done' once for each value of `$user` specified by the line beginning with 'for'.

Numbers can be used instead of variables in the loop so that the script can keep track of which number loop it is on:

```
#!/bin/bash

for i in 1 2 3 4 5
do
    echo "loop $i"
done
```

If the script needs to loop a lot of times, this notation can be used:

```
for i in {1..20}
```

This will loop 20 times with the value of `$i` incrementing by 1 each time. If incrementing by a value other than 1 is required then use the 'seq' command, eg:

```
for i in $(seq 0 0.5 4)
```

This will give values for `$i` from 0 to 4, incrementing by 0.5

9.4 Conditional statements

“if / then / else” statements allow conditional branching within a script. This script tests the value of a variable and then decides which branch to take (and therefore what to output), depending on that value.

```
#!/bin/bash

NUMLOGINS=$(who | grep $USER | wc -l)

if [ $NUMLOGINS -gt 1 ]
then
    echo "$USER is logged in with at least 2 sessions"
else
    echo "$USER has less than 2 sessions"
fi
```

The ‘if ... then ... else’ statement must be finished with ‘fi’. The ‘else’ option is optional – the script will carry on past ‘fi’ if you do not use it.

‘-gt’ is short for ‘greater than’. Other tests include:

- -eq is equal to
- -ne is *not* equal to
- -ge is greater than or equal to
- -lt is less than
- -le is less than or equal to

Exercise

Write a script which will test how many logins you and the ‘root’ user have and tell you who has the most (or if equal)

Appendix A: Connecting to SPECTRE

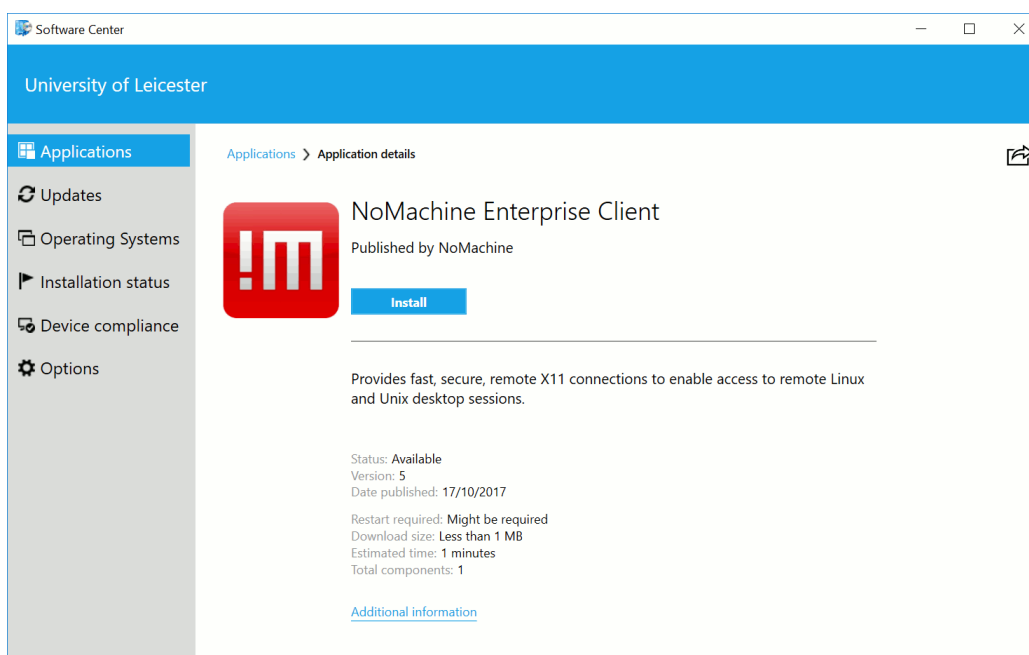
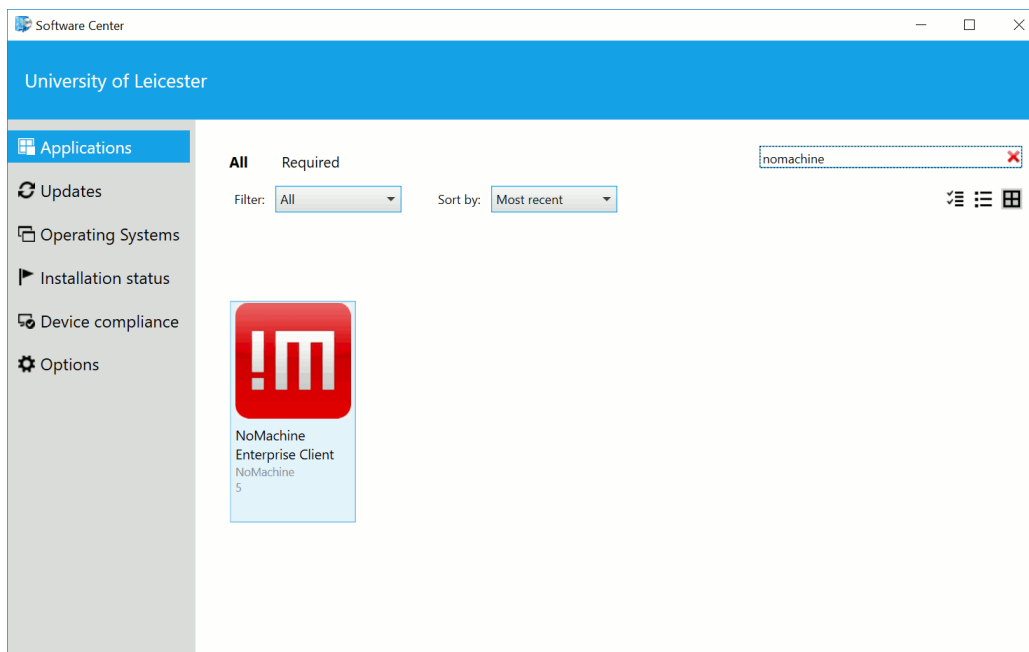
A.1 Using NoMachine (NX) Client

Windows 10

To install and then start NoMachine on a standard Windows 10 managed PC use

Start > Software Center

Wait for the software list to populate, search for NoMachine and click to install:



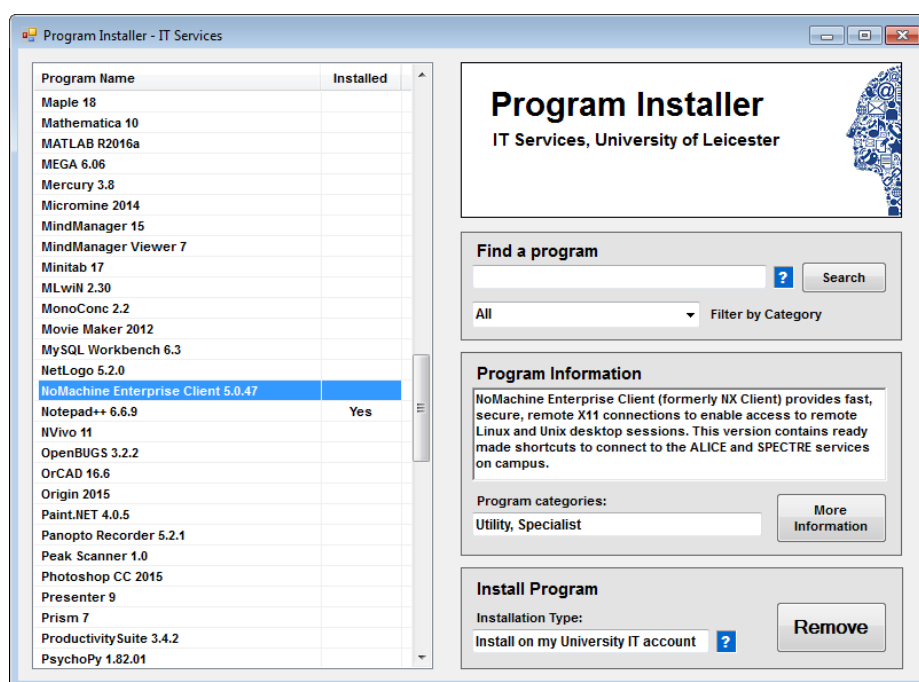
Once installed, you can log in to SPECTRE with the NoMachine client by doing:

Start > NoMachine Enterprise Client > NoMachine Enterprise Client > UoL HPC

Windows 7

To install NoMachine from a standard Windows 7 managed PC use

Start > All Programs > Program Installer



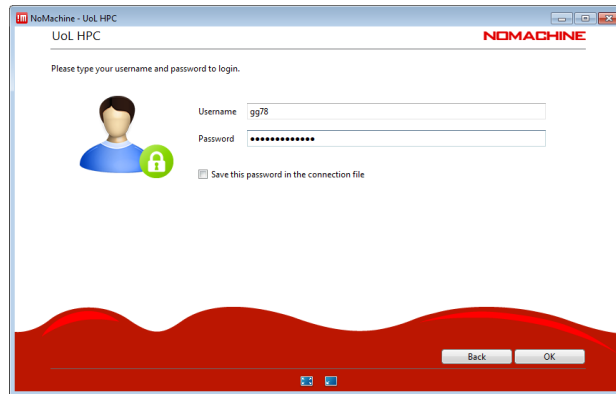
Select **NoMachine Enterprise Client 5.0.47** from the list, then click *Install*.

Once installed you can log in to SPECTRE with the NoMachine client by going to

Start > All Programs > NoMachine Enterprise Client 5.0.47 > UoLHPC

Windows 10 / Windows 7

The window shown below will now appear. Enter your username and password and click *Login*.



After a short delay a window will open showing the SPECTRE desktop environment.

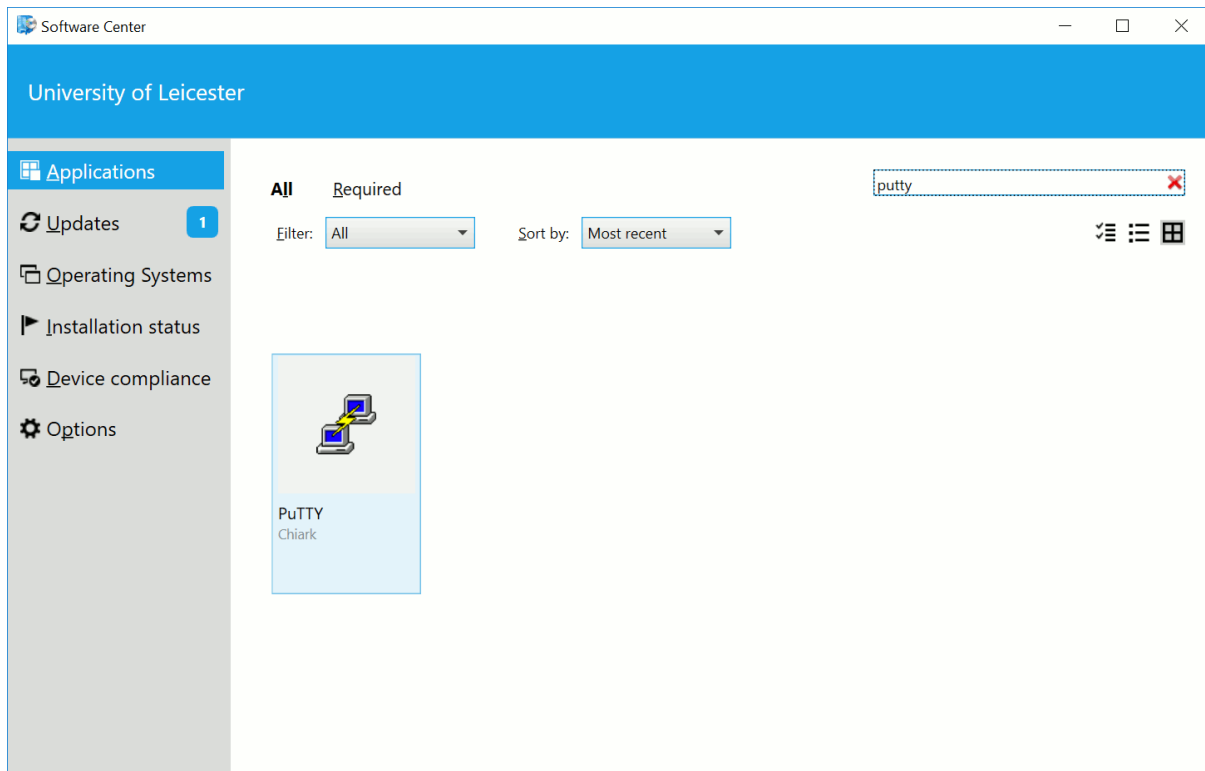
A.2 Using PuTTY Client

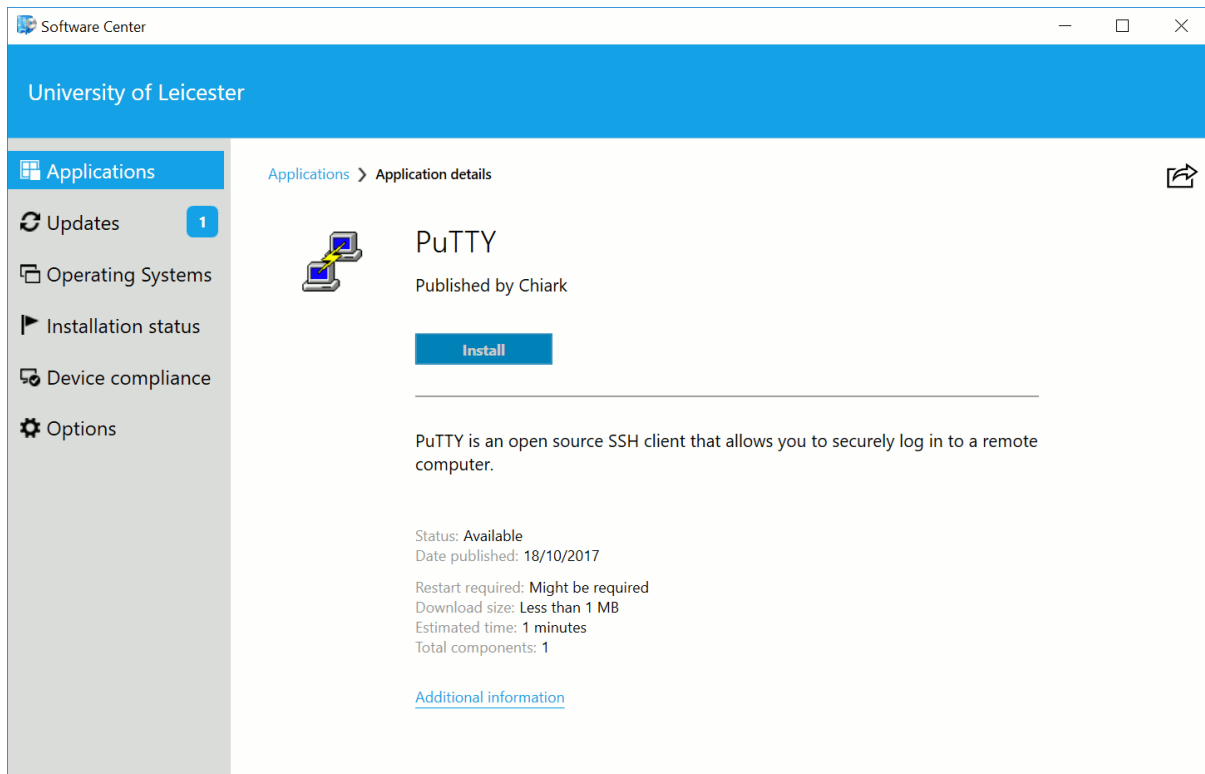
Windows 10

To install and then start PuTTY on a standard Windows 10 managed PC use

Start > Software Center

Wait for the software list to populate (this can take several minutes), search for PuTTY and click to install:





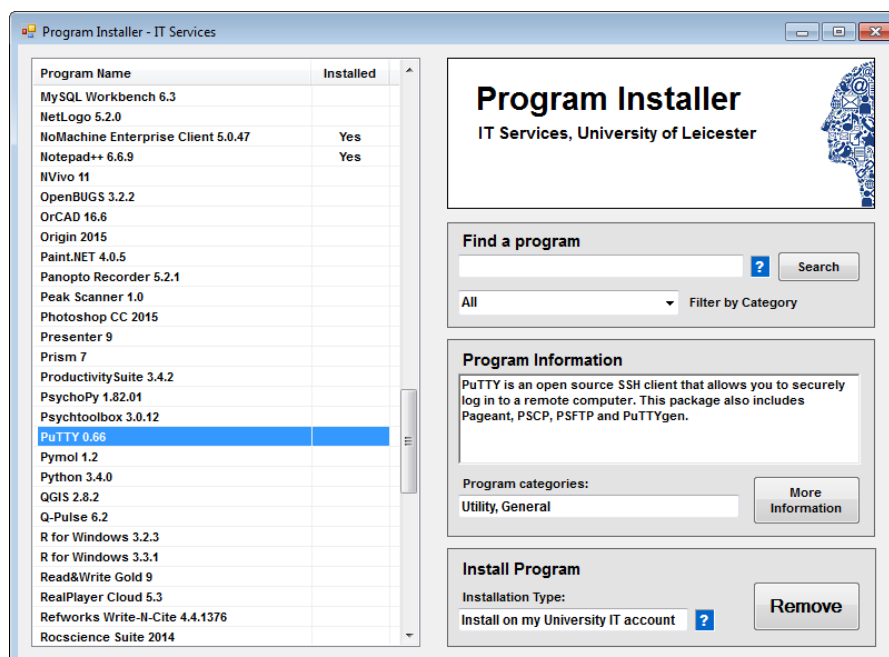
Once installed, you can start PuTTY by doing:

Start > PuTTY > PuTTY

Windows 7

To install and start PuTTY on a standard Windows 7 managed PC use

Start > All Programs > Program Installer



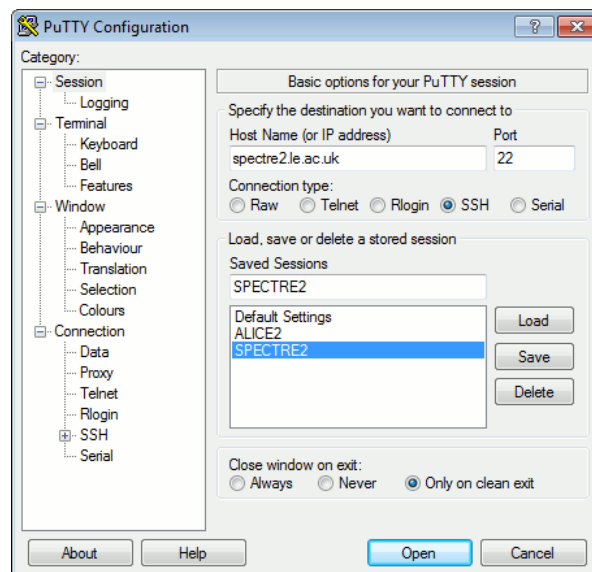
Select **PuTTY 0.66** from the list of applications, then click *Install* to install it.

Once installed you start PuTTY by going to

Start > All Programs > PuTTY > PuTTY

Windows 7 / Windows 10

Once you have installed and started PuTTY you will see the window shown below. Double-click on SPECTRE2 in the Saved Sessions list. A new window will open and prompt for your username and password. The first time you connect you will be asked to confirm the server key's hash.



A.3 Opening a text editor (PuTTY/SSH)

If you have logged on using PuTTY you will need to use a *command line text editor*. We recommend using *nano* in this course. At the command prompt type

```
nano filename
```

This will open the text editor in the terminal and you will not be able to run other commands until you've closed *nano*. You will see the following screen in the terminal:

```
GNU nano 1.3.12          New Buffer

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit     ^J Justify  ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

At this point you can enter the text you want to appear. You will need to know two *nano* specific commands, to save your file, type:

```
Ctrl-O
```

and press enter to confirm the file name,

then to close *nano* type:

```
Ctrl-X
```

You will now be returned to the command prompt.

A.4 Opening a text editor (NX)

If you logged on using NX you will be able to use a *graphical text editor* such as *gedit*, at the command prompt, you should be able to type *gedit* followed by the name of the file you would like to open (or create if it does not exist):

```
gedit <filename>
```

This will open the text editor in a new window.

Alternatively, you can find *gedit* on SPECTRE in Applications > Accessories > Text Editor

A.5 Summary of commands used in this Tutorial

ls	list files and directories
ls -a	list all files and directories
mkdir	make a directory
cd <i>directory</i>	change to named directory
cd	change to home-directory
cd ~	change to home-directory
cd ..	change to parent directory
pwd	display the path of the current directory
cp file1 file2	copy file1 and call it file2
mv file1 file2	move or rename file1 to file2
rm file	remove a file
rmdir directory	remove a directory
cat file	Display or concatenate a file
less file	display a file a page at a time
head file	display the first few lines of a file
tail file	display the last few lines of a file
grep 'keyword' file	search a file for keywords
wc file	count number of lines/words/characters in file
command > file	redirect standard output to a file
command 2> file	redirect standard error to a file
command >> file	append standard output to a file
command < file	redirect standard input from a file
command1 command2	pipe the output of command1 to the input of command2
cat file1 file2 > file0	concatenate file1 and file2 to file0
sort	sort data
who	list users currently logged in

*	match any number of characters
?	match one character
man command	read the online manual page for a command
whatis command	brief description of a command
apropos keyword	match commands with keyword in their man pages
ls -lag	list access rights for all files
chmod [options] file	change access rights for named file
command &	run command in background
^C	kill the job running in the foreground
^Z	suspend the job running in the foreground
bg	background the suspended job
jobs	list current jobs
fg %1	foreground job number 1
kill %1	kill job number 1
ps	list current processes
kill 26152	kill process number 26152

Les commandes de base de LINUX

Les commandes de gestion des répertoires et des fichiers

pwd (affiche le chemin absolu du répertoire courant)

ls (list, affiche les répertoires et les fichiers du répertoire actif)

ls (affiche seulement les noms)

ls toto* (affiche les fichiers commençant par toto)

ls -l (affiche le format long : types + droits + Nbre de liens +)

cd (change directory)

cd chemin (vers le répertoire dont le chemin absolu est donné)

cd .. (répertoire parent)

cd ~ (répertoire de base)

cd - (répertoire précédent)

cd / (répertoire racine)

cp (copie)

cp rapport*.txt sauvegarde

cp * dossier (copie)

mv (move, renomme et déplace un fichier)

mv source destination

mv * dossier (déplace tous les fichiers du répertoire actif vers le répertoire dossier)

mkdir (créer un répertoire)

mkdir répertoire

rmdir (effacer un répertoire)

rmdir dossier (supprime un répertoire vide)

rm (remove, efface!!!)

rm -R (enlèvement récursif!!!)

rm fichier

rm -i fichier (interactivement, avec demande de confirmation)

rm -f fichier (avec force, sans demande de confirmation)

rm -r fichier (avec récursivité, avec les sous répertoires)

rm -rf dossier (supprime le répertoire et tout son contenu, sans confirmation)

Les commandes de recherche

grep (recherche les occurrences de mots à l'intérieur de fichier)

grep motif fichier

grep -i motif fichier (sans tenir compte de la casse)

grep -c motif fichier (en comptant les occurrences)

grep -v motif fichier (inverse la recherche, en excluant le "motif")

grep expression /répertoire/fichier

grep [aFm]in /répertoire/fichier

grep "\$" *.txt

Les commandes d'édition

more ("pager" qui affiche page par page sans retour en arrière, "h" affiche l'aide contextuelle)

more fichier
more fichier1 fichier2
more *.txt

cat (concatenate avec le code de fin de fichier eof=CTRL + D)

cat fichier-un fichier-deux > fichier-un-deux

cat -n fichier > fichier-numéroté (crée un fichier dont les lignes sont numérotés)

cat -nb fichier (affiche sur la sortie standard les lignes numéroté, sauf les lignes vides)

head (affiche les 10 premières lignes d'un fichier)

head -n22 fichier (affiche les 22 premières lignes)

vi (l'éditeur en mode texte universel)

emacs (l'éditeur GNU Emacs multi fonction pour l'édition, les mails, les news, la programmation, la gestion des fichiers,...)

xemacs (l'éditeur GNU Emacs sous X)

diff (différence entre deux fichiers, utiles pour chercher les modifications)

diff fichier1 fichier2

Les commandes d'impression et de conversion

lp (la commande d'impression sur les systèmes Unix Système V)

lpr (la commande d'impression sur les systèmes BSD et Linux)

lpr fichier

echo \$PRINTER

lpc status (affiche l'état de la file d'attente)

lpq (affiche les travaux d'impression et leur numéro)

lprm (supprime un travail d'impression avec son numéro comme argument)

gv ("ghostview" permet de visualiser des fichiers POST SCRIPT)

gv fichier.ps

a2ps (convertit les fichiers ASCII en POST SCRIPT)

a2ps -4 fichier -P fichier-post-script

Les commandes de compilation et d'exécution

f77 compile un programme en fortran 77

f77 program.f (la terminaison .f indique que le fichier program.f est écrit en f77)

./a.out exécution

Les autres commandes

cal (calendar)

cal 2002

date (affiche la date, le mois, l'heure et l'année du jour. Les messages d'erreur et les e-mails sont toujours datés avec la date système)

date -s

wc ("word & count", affiche le nombre de lignes + mots + caractères)

who | wc -l (affiche uniquement le nombre de lignes)

spell (programme de correction orthographique)

cat rapport.txt | spell > faute.txt

read (lit dans un script shell la ligne saisie à partir de l'entrée par défaut, le clavier)

L'Éditeur vi

vi est un éditeur de fichiers qui contiennent des lignes de texte. Il fonctionne en mode écran; le nom **vi** provient du mot *visual*. I

1 Quelques commandes essentielles

- Démarrer l'éditeur : **vi nom_du_fichier_à_éditer** (vous êtes en mode commande)
- Sauvegarder un fichier : **:w nom_du_fichier**
- Quitter l'éditeur en sauvegardant le fichier: **:x**
- Quitter sans sauvegarder : **:q!**

1 Commandes de base

- **Pour entrer du texte :**

Ces commandes vous amènent en mode insertion.

- **a** ajoute du texte à la droite du curseur
- **i** insère du texte à la gauche du curseur
- **o** intercale une ligne vide au-dessous du curseur
- **O** intercale une ligne vide au-dessus du curseur

En mode insertion :

retour insère une fin de ligne

Lorsque vous êtes en mode insertion taper ECHAP (ou ESC) pour revenir au mode commande

- **Pour remplacer du texte :**

- **r** le caractère tapé remplace le caractère pointé par le curseur
- **R** remplace plusieurs caractères [taper **ECHAP** (ou **ESC**) pour revenir au mode **commande**]

- **Pour déplacer le curseur dans le texte :**

flèches pour se déplacer d'un caractère vers la gauche ou la droite, ou d'une ligne vers le haut ou le bas [ou utiliser les touches **h** (gauche), **j** (bas), **k** (haut), **l** (droite)]

Par ligne :

- **0** se positionne au début de la ligne
- **\$** se positionne à la fin de la ligne
- **retour** se positionne au premier mot de la ligne suivante

D'un écran :

- **^f** (peser simultanément sur les touches **CTRL** et **f**) avance d'un écran
- **^b** (peser simultanément sur les touches **CTRL** et **b**) recule d'un écran

Pour aller à une ligne en particulier :

- **#G** positionne le curseur à la ligne **#**
- **1G** positionne le curseur au début du fichier
- **G** positionne le curseur à la dernière ligne du fichier
- **^g** (peser simultanément sur les touches **CTRL** et **g**) révèle le numéro de la ligne courante

- **:set nu** affiche les numéros de lignes
- **Pour enlever, remplacer ou copier une partie du texte :**
 - x** détruit le caractère pointé par le curseur et place dans le tampon
 - #x** détruit # caractères et place dans le tampon
 - dd** détruit la ligne courante et place dans le tampon
- **#dd** détruit # lignes à partir de la ligne courante et place dans le tampon
 - **yy** copie la ligne courante dans le tampon
 - **#yy** copie # lignes consécutives dans le tampon
 - **p** insère le contenu du tampon à la droite du curseur (si 1 ou quelques caractères dans le tampon)
 - **P** insère le contenu du tampon à la ligne suivante (si 1 ou quelques lignes dans le tampon)
 - **P** insère le contenu du tampon à la ligne précédente

1 Commandes plus complexes

- **Recherche d'une chaîne de caractères particulière :**
 - **/chaîne** cherche chaîne en avançant vers la fin du fichier;
 - **n trouve** la prochaine occurrence de la dernière chaîne recherchée.
- **Substitution :**
 - **:s/ceci/cela/options** substitue la première occurrence de **ceci** par **cela** dans la ligne courante ; l'option **g** substitue toutes les occurrences dans la ligne courante ; l'option **c** demande de confirmer la substitution
 - **:3,9s/ceci/cela** remplace aux lignes **3 à 9** la première occurrence de **ceci** par **cela**.
 - **:%s/ceci/cela** remplace dans tout le fichier la première occurrence de ceci par cela
 - **:%s/ceci/cela/g** remplace dans tout le fichier toutes les occurrences de **ceci** par **cela**

1 Et d'autres commandes encore !

- **~** change la casse (majuscules/minuscules) d'une lettre
 - **J** joint la ligne courante à la suivante
 - **.** repète la dernière commande
 - **#commande** exécute une commande # fois
 - **u** annule la dernière commande
 - **U** annule les commandes affectant la ligne courante
 - **:3,9d** élimine les lignes **3 à 9**
-

notes de cours Fortran 77

notes de cours Fortran 77 : table des matières

- 1) Introduction
- 2) La gestion de l'ordinateur
- 3) Bases du fortran 77
- 4) Le séquençement des instructions
 - 4.1) type logical
 - 4.2) l'instruction conditionnelle (if)
 - 4.3) les boucles
 - 4.4) le saut (goto)
- 5) Les sous-programmes et fonctions
- 6) Les tableaux
- 7) Déclarations particulières : constantes, initialisation
- 8) Les chaînes de caractères
- 9) Les changements de type et fonctions intrinsèques
 - 9.1) mélanges de types
 - 9.2) conversion explicite
 - 9.3) fonctions intrinsèques
- 10) Variables communes
- 11) Entrées, sorties et formats
 - 11.1) ouverture et fermeture du flux
 - 11.2) lecture - écriture
 - 11.3) déclaration du format
- 12) Que n'ai-je pas dit ?

Les TP et TD 2004/2005 (sujet et correction) sont [ici](#)

1) Introduction

voir </pat/internet/techinfo>

- structure de l'ordinateur (CPU, mémoire, interfaces, bus)
- les caractéristiques du CPU, les différentes vitesses dans un PC, les caches.
- les périphériques
- le langage machine (petit exemple), les 5 générations de langages

2) La gestion de l'ordinateur

- BIOS, OS, DOS
- les disques et leur gestion (FAT, répertoires, adressage absolu et relatif)
- les Files Systems (FAT12, 16, 32, Unix, NFS...)
- sous Unix : arborescence classique, montage/démontage, *mkdir*, *rmdir*, *cd*, *ls*, *rm*
- la compilation, l'édition de liens. En pratique (*kwrite prog.f* & / *gcc prog.f -o prog / prog*). Si vous cherchez un compilateur, je vous conseille le meilleur : GNU G77 (www.gnu.org). Il est disponible dans toutes les distributions Linux, pour Windows j'en ai fait une copie [ici](#).

3) Bases du fortran 77

- le format de la ligne (C2345+789.....72CCCC) : en colonne 1, un C (ou *) indique que la ligne est un *commentaire* (ne sera pas prise en compte par le compilateur). Les colonnes 1 à 5 peuvent contenir un label (numéro de ligne par exemple). S'il y a un caractère en colonne 6 (un + par exemple), cette ligne est la continuation de la suivante. Les colonnes 7 à 72 contiennent les instructions. A partir de la colonne 73, plus rien n'est lu.
Les compilateurs actuels (dont g77) acceptent un commentaire en fin de ligne : il commence par ! (et va jusqu'à la fin de la ligne). C'est même conseillé en premier caractère d'une ligne de commentaire (au lieu de C) pour la compatibilité avec F90 et plus.
- les *identificateurs* (noms des objets) : utilisez les caractères (A-Z,0-9), ni espace ni accent ni - (réservé à la soustraction). Le premier doit être une lettre, pas un chiffre. Le compilateur transforme toutes les minuscules en majuscules (sauf si entre apostrophes '). 6 caractères maxi (31 maxi en général, mais il vaut mieux qu'il n'y en ait pas deux avec les mêmes 6 premiers). Les identificateurs sont séparés par un ou plusieurs espaces, que l'on peut omettre si le caractère suivant n'est pas (A-Z,0-9).
- les types de variables de base : *integer* (en général jusqu'à +ou- 10⁹), *real* (en général avec 7 chiffres significatifs), *double precision* (en général avec 15 chiffres significatifs). Remarque : on peut (quand on sait exactement ce qu'on fait) imposer le nombre d'octets des variables : integer *2 et *4, real *4 et *8). Il existe aussi les *character*, *logical* et *complex* (voir plus loin)
- les *opérateurs arithmétiques*, par priorité décroissante : (1) - unaire, (2) ** puissance, (3) * et /, (4) + et -. En cas de calcul entre deux entiers, le résultat est entier. Dès qu'il y a au moins un réel, le résultat est réel. Dès qu'il y a un double précision, le résultat est double précision.
- les constantes : 1 (entier), 100.0 ou 1E2 (réel), 1D0 (double précision)
- petit programme ([premprog.f](#))

```
program calcul                                entête
implicit none                                pour plus de sécurité, nous refusons
integer nb                                    les déclarations implicites
real pu,pt
print *, 'combien de pièces ?'                écrire à l'écran
read *,nb                                     lire au clavier
print *, 'prix unitaire ?'
read *,pu
pt=pu*nb                                       affectation: le calcul est effectué puis
print *, 'cela fera ',pt,' en tout'           le résultat est stocké dans pt
stop
end program calcul                            clôture
```

le résultat est (en rouge, tapé par l'utilisateur du programme):

```
combien de pièces ?
5
prix unitaire ?
1.25
cela fera  6.25 en tout
```

structure du programme :

- entête (*program* nom au choix)
- déclarations (implicites et explicites) : les variables non déclarées commençant par (I-N) sont considérées entières, les autres réelles. Accepter les déclarations implicites est souvent source d'erreurs difficiles à localiser (un erreur de frappe crée une autre variable au lieu d'un message d'erreur), je vous conseille de toujours commencer par "*implicit none*"
- instructions : *print* affiche à l'écran, *read* lit au clavier. L'affectation se note : variable = expression, ou contenant = contenu, ou où = combien. Le programme calcule d'abord combien vaut l'expression à droite du =, puis stocke le résultat dans la variable nommée à gauche du =. "*stop*" arrête le programme, il est optionnel en dernière instruction du programme.
- clôture du programme (*end* est suffisant, mais pour plus de clarté on le complète par le nom du programme).

4) Le séquençement des instructions

4.1) type logical

Abordons d'abord le type "*logical*", ses valeurs *.TRUE.* *.FALSE.*, les opérateurs associés (comparaisons *.LT.* , *.GT.*, *.LE.*, *.GE.*, *.EQ.*, *.NE.*; booléens *.AND.* *.OR.* *.NOT.*). Exemple : *test = (x.ge.3).and.(x.lt.10) : x dans [3,10[.*

4.2) l'instruction conditionnelle (if)

forme standard (archaïque)

IF (expression logique) instruction unique

si l'on veut rendre conditionnelle l'exécution de plusieurs lignes, *f77* permet :

IF (expression logique) THEN
 une ou plusieurs instructions
ENDIF

Si l'on a plusieurs condition EXCLUSIVES on peut même :

IF (expression1) THEN
 une ou plusieurs instructions
ELSEIF (expression2) THEN
 une ou plusieurs instructions
ELSEIF (expression3) THEN
 une ou plusieurs instructions
ELSE
 une ou plusieurs instructions
ENDIF

Le dernier ELSE (obligatoirement sans expression logique) est optionnel.

Encore plus complexe, parmi les instructions conditionnelles on peut utiliser des *if ... endif*. On nomme cela des *if imbriqués*. Un *endif* correspond toujours

au dernier *if* qui n'en était pas pourvu.

exemple (il manque l'entête, les déclarations, la saisie des coefficients et surtout un test) :

```
delta=b*b-4*a*c
if (delta.GT.0) then
  x1=(-b-sqrt(delta))/(2*a)
  x2=(-b+sqrt(delta))/(2*a)
  print *,'2 racines: ',x1,' et ',x2
elseif(delta.EQ.0) then
  x=-b/(2*a)
  print *,'1 racine double: ',x
else
  print *,'aucune racine réelle'
endif
```

(voir [seconddeg.f](#))

Remarque sur la division : il faut TOUJOURS vérifier que le dénominateur est non nul (le célèbre domaine de définition cher à tous vos profs de maths depuis le collège).

4.3) les boucles

forme standard (archaïque)

```
DO label indice=val.initiale, val.finale, incrément
  une ou plusieurs instructions
label CONTINUE
```

Le label est un entier dans]0,99999] (doit rentrer dans les 5 premiers caractères de la ligne). L'incrément est optionnel (1 par défaut). La boucle sera effectuée avec l'indice (que vous aurez déclaré avant en *integer*) allant de '*val.initiale*' à '*val.finale*' compris, par pas de '*incrément*'. Il ne faut JAMAIS tenter de modifier l'indice, *val.initiale*, *val.finale* ou incrément en cours de boucle. En sortie de boucle, l'indice ne vaut pas nécessairement la valeur finale, ça dépend des compilateurs.

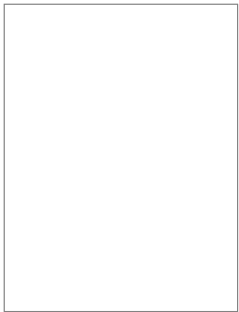
exemple :

```
do 10 i=1,10
  print *,i,'è coucou'
10  continue
```

qui donne

```
1è coucou
2è coucou
3è coucou
4è coucou
5è coucou
6è coucou
7è coucou
8è coucou
9è coucou
10è coucou
```

En fait, le *continue* ne peut fermer que la dernière boucle ouverte (mais on peut imbriquer des boucles). Le label n'apporte rien (que peut-être la clarté).

exemple : <pre> DO 10 ---- ---- do 20 ---- ---- 20 continue do 30 ---- ---- 30 continue ---- 10 continue </pre>	boucles imbriquées : OK 	chevauchement impossible : 
---	--	---

Fortran77 propose donc une écriture sans label :

```

DO indice=val.initiale, val.finale, incrément
  une ou plusieurs instructions
ENDDO

```

Mais ces boucles ne peuvent se faire que si leur nombre est fixé d'avance. C'est pourquoi la plupart des compilateurs acceptent d'autres boucles, mais elles ne sont pas standardisées dans fortran 77. Celle qui me semble la plus fréquente (et acceptée par les normes suivantes) est :

```

DO while (condition logique)
  une ou plusieurs instructions
ENDDO

```

exemple :

```

DO while(x.gt.1)
  x=x/2
ENDDO

```

si x est inférieur à 1 avant de commencer la boucle, aucune division par 2 n'est effectuée.

Deux instructions spécifiques aux boucles ont été rajoutées dans G77 : *EXIT* (sortir immédiatement de la boucle) et *CYCLE* (passer directement à la prochaine itération).

Autres écritures de boucles existantes, suivant les compilateurs (mais je ne vous les conseille pas) :

```

while (condition) do
  instructions
enddo

```

```

do
  instructions
while(condition)

```

```

do
  instructions

```

until (condition)

4.4) le saut (goto)

GOTO label

label CONTINUE

Le label doit être unique dans un programme, il peut se situer au dessus ou au dessous du GOTO. L'instruction en face du label n'est pas nécessairement un *continue*, ce peut être n'importe quelle autre instruction (qui sera exécutée lors du saut). Les goto sont créateurs de nombreux problèmes, (dans les programmes complexes ou buggés, uniquement) c'est pourquoi les programmeurs expérimentés essayent de s'en passer. Plusieurs GOTO différents peuvent sauter au même label. On peut sortir par GOTO de n'importe où, mais on ne peut pas pointer vers l'intérieur d'un bloc d'instructions (IF, DO, Subroutine)

exemple :

```
1    do 10 i=deb,fin
      ----
      GOTO 5: ici un goto 10 passe au prochain i, goto 1 recommence à deb.
      ----
5     continue
      ----
10    continue
      ----
      GOTO 5: d'ici, ce GOTO est impossible. Mais GOTO 1 le serait
```

Une écriture fréquente est le **IF (condition) GOTO label**, servant par exemple pour créer des boucles "tant que" qui soient compatibles avec tout compilateur.

5) Les sous-programmes et fonctions

Un sous-programme est le regroupement sous un même nom d'un groupe d'instructions (et de variables, dites locales). Le programme et les sous-programmes partagent des variables via ce que l'on appelle les paramètres (et à la rigueur le COMMON, voir plus loin)

Déclaration d'un sous-programme :

```
SUBROUTINE nom (liste d'arguments formels)
déclarations locales (y compris arguments formels)
instructions (dont RETURN pour quitter la subroutine, optionnel en
dernière instruction)
END SUBROUTINE nom
```

appel d'un sous-programme :

```
CALL nom (liste d'arguments réels)
```

exemple :

```
SUBROUTINE echange (x,y)
implicit none
```

```

real x,y,tampon
tampon=x
x=y
y=tampon
RETURN
END SUBROUTINE echange

```

autre part :

```

real a,b
a=10
b=15
CALL echange (a,b)

```

Les arguments réels doivent être exactement du même type que les arguments formels, aussi nombreux et dans le même ordre (même pas de transformation automatique d'entier en réel). Les arguments sont dits "passés par adresse", ce qui signifie que toute modification à l'argument formel est répercutée à l'argument réel (j'en parle parce que dans d'autres langages c'est différent).

Une fonction, quand à elle, a un résultat. Elle est appelée dans une expression

```

type FUNCTION nom (liste d'arguments formels)
déclarations locales (y compris arguments formels)
instructions (dont RETURN pour quitter la fonction, précédé d'une
    affectation au nom de la fonction)
END FUNCTION nom

```

exemple:

```

real function puiss(x,n)
implicit none
real x,p
integer i,n
p=x;
do i=2,n
    p=p*x
enddo
puiss=p
return
end function puiss

```

appel :

```

real z,puiss
z=2.3
print *, 'calcul: ', puiss(z,10)+1

```

Remarque : ici z reste à 2.3, car dans la fonction x est inchangé. Si l'on avait modifié x dans la fonction, on aurait obtenu ce que l'on appelle un "effet de bord".

Exemple de sous-programme à effet de bord :

```

real function factorielle(n)
integer res,n
res=1
do while (n.GT.1)
    res=res*n

```



```

        n=n-1
    enddo
    factorielle=res
    return
end function factorielle

```

Cette fonction calcule bien les factorielles, mais ces instructions :

```

integer nb,factorielle
nb=5
print *, factorielle(nb),' est la factorielle de ', nb

```

donneront ce résultat

```

120 est la factorielle de 1

```

C'est assez déplorable, nb ne vaut plus 5 ! Conclusion : ne modifiez un argument formel que si c'est nécessaire.

Dans le programme appelant la fonction, son type doit être déclaré. Dans l'exemple de la factorielle si vous l'oubliez (et avez oublié "*implicit none*"), il la considérera *real*, ce qui donnera des résultats aberrants !

6) Les tableaux

On peut regrouper sous un seul nom plusieurs variables simples, c'est un tableau unidimensionnel ou vecteur. On les déclare ainsi :

```

type nom (indice début: indice fin)

```

La plupart du temps, on omet l'indice début (qui sera 1), l'indice fin sera alors aussi le nombre de composantes. Il existe dans ce cas une seconde méthode pour déclarer le tableau :

```

type nom
dimension nom(nombre)

```

exemple :

```

real vecteur(3)

```

ou

```

real vecteur
dimension vecteur(3)

```

On utilise les composantes d'un tableau dans toute expression par

```

nom(indice)

```

exemple

```

x=(vecteur(i)-vecteur(1))/vecteur(i+1)

```

Attention, c'est au **programmeur** de vérifier qu'il utilise toujours des indices dans les limites déclarées ! La dimension de tous les tableaux doit être connue du compilateur, pour qu'il puisse gérer la mémoire utilisée par le programme,

avant même que la première instruction n'ait été exécutée . La dimension doit donc être une constante, elle ne peut pas être une variable que vous demanderiez à l'utilisateur par un *read*. Par contre rien ne vous empêche de déclarer une dimension assez importante, et n'utiliser qu'une partie du tableau, variable à chaque utilisation du programme, mais toujours inférieure à la dimension déclarée.

On ne peut, dans un programme, que traiter les composantes d'un tableau, pas son intégralité (par exemple, pour copier un tableau il faut, dans une boucle, copier toutes ses composantes). Par contre on peut passer un tableau en argument d'un sous-programme ou fonction.

exemple :

```
real x(5)
print *, 'moyenne: ', moy(x,5)
-----
real function moy(x,nb)
real x(*),som
integer nb,i
som=x(1)
do i=2,nb
  som=som+x(i)
enddo
moy=som/nb
end function moy
```

Dans la déclaration, la dimension du tableau n'a besoin d'être donnée que dans l'entité qui réserve la place nécessaire en mémoire, pas pour ceux qui reçoivent le tableau en argument, qui même si on leur donne la dimension ne s'en servent pas. C'est pourquoi il vaut mieux indiquer * (certains programmeurs dimensionnent à 1 les tableaux reçus en arguments, mais c'est peu clair). Une autre solution est de la passer en argument (ou même une dimension inférieure à la dimension réelle, mais pas plus grande), dans la fonction ci-dessus on déclarerait `real x(nb)`.

Vous pouvez voir ici un programme de base sur les tableaux ([tab-sp.f](#))

Pour représenter une matrice, on utilise un tableau bidimensionnel. La déclaration est :

```
type nom (deb lig:fin lig , deb col:fin col)
```

l'appel est

```
nom(indice ligne, indice colonne)
```

Dans la pratique, le compilateur range en mémoire toute la première ligne de la matrice, puis la seconde, la troisième... Mais tant que vous n'êtes pas spécialistes, il vaut mieux l'ignorer. Rappelez-vous qu'en général le premier indice est la ligne, le second la colonne (licol).

On peut étendre à des dimensions supérieures, mais en se limitant à la 7è dimension. Exemple :

```
integer rubiks(3,3,3)
```

Attention, en cas de passage en arguments de tableaux multidimensionnels, toutes les dimensions doivent être déclarées exactement, sauf la dernière qui peut être * (en général on les donne toutes). Mais la solution idéale est de passer les dimensions en arguments (obligatoirement la dimension exacte, sauf pour la dernière qui peut être inférieure si c'est vraiment utile).

exemple :

```
program matrice
integer maxlig,maxcol
parameter(maxlig=5,maxcol=3)
real m(maxlig,maxcol)
integer il,ic
do il=1,maxlig
do ic=1,maxcol
m(il,ic)=il+(ic/10.0)
enddo
enddo
call affmat(m,maxlig,maxcol)
end

subroutine affmat(mat,diml,dimc)
implicit none
integer diml,dimc
real mat(diml,dimc)
integer il,ic
do il=1,diml
print *,('M(',il,ic,')=',m(il,ic),ic=1,dimc)
enddo
end
```

7) Déclarations particulières : constantes, initialisation

On peut déclarer une constante, c'est à dire une valeur fixée par le programmeur, qui ne pourra pas changer lors de l'exécution du programme. On s'en sert principalement pour déclarer les tailles de tableau (ce qui permettra par la suite de la changer plus facilement)

exemples :

```
real tva
parameter (tva=19.6)
integer largeur,hauteur
parameter (largeur=5,hauteur=2)
real table(largeur,hauteur)
do i=1,largeur
----etc -----
call subroutine(table, largeur, hauteur)
----etc -----
```

Il est également possible d'initialiser des variables. C'est à dire fixer une valeur initiale à une variable, mais cette valeur pourra changer au cours du programme. On utilise pour cela la déclaration DATA :

[DATA liste de variables / liste de valeurs initiales /](#)

exemple :

```
real x,y  
data x,y / 0.0 , 0.5 /
```

on peut remplacer une suite de valeurs identiques par "nombre*valeur". On peut également initialiser des tableaux :

```
real t(2,5)  
data t / 5*0.0, 1.0, 4*0.0 /
```

Même si un *data* se trouve dans une subroutine, l'initialisation ne sera effectuée qu'une fois, au début de l'exécution du programme.

8) Les chaînes de caractères

On peut stocker des caractères dans une "chaîne", qui est en fait un tableau de caractères. On le déclare par CHARACTER*dim NOM. Comme tout tableau, la dimension est importante et doit être définie lors de l'écriture du programme. Si l'on n'utilise pas la totalité de la dimension déclarée, le compilateur complète avec des espaces (qu'il rajoute en fin de chaîne).

Une constante chaîne de caractères est délimitée par des apostrophes ('). Pour inclure une apostrophe dans une chaîne, il faut la doubler ('). // est l'opérateur de concaténation : il permet de "coller" deux chaînes pour n'en former qu'une. On peut extraire une partie d'une chaîne : nom(début:fin).

exemples :

```
character*10 x,y  
character*20 z  
x='bonjour'  
x(4:6)='soi'  
y='c'est moi'  
y(7:7)=y(5:5)  
z=x//y
```

ce qui donne dans z : "bonsoir c'est toi"

On peut créer des tableaux de chaînes. Par exemple : character*10 txt(20)

On peut aussi comparer des chaînes. Par .EQ. et .NE. mais aussi à l'aide des fonctions LGE, LGT, LLE ou LLT. Par exemple, LGE(c1,c2) est .TRUE. si a1 est supérieur ou égal à c2 (dans l'ordre alphabétique).

Il existe d'autres fonctions spécifiques aux chaînes de caractères : ichar(i) donne le caractère dont le code ASCII est l'entier i, char(c) donne le numéro de code du caractère c. Pour trouver la position de la sous-chaîne sc dans la chaîne plus longue lc : index(lc,sc).

9) Les changements de type et fonctions intrinsèques

9.1) mélanges de types

Toutes les variables sont stockées en mémoire sous forme d'une suite de 0 et de 1. Mais chaque type de variable est codé différemment. Pour info, les entiers sont en binaire signé, les réels sont en virgule flottante 32 bits et les double précision en 64 bits. Une même valeur sera donc codée par une suite de 0 et de 1 différente suivant qu'elle est dans une variable entière, réelle ou double précision. Le compilateur ne sait faire des opérations arithmétiques (+ - * / **) que sur des valeurs codées de la même manière. Si vous lui demandez un calcul entre deux variables de type différent, le compilateur doit avant tout en convertir une pour qu'elles soient de même type. Il sait convertir automatiquement un entier en réel et un réel en double précision (mais c'est tout). Donc, suivant le type des deux opérandes, le résultat de l'opération sera :

	integer	real	double précision
integer	integer	real	double précision
real	real	real	double précision
double précision	double précision	double précision	double précision

La conversion n'est faite que lorsque c'est nécessaire. Dans la plupart des cas, c'est la meilleure solution car la plus rapide. Il n'y a que dans le cas de la division que le résultat est différent suivant les cas. En effet, la division de deux entiers donne un entier (et un reste, mais il est ignoré). Par exemple, si x et $y=2$ sont réels, $i=5$ et $j=3$ sont des entiers, alors $x=y+i/j$ met 3 dans x : la division est prioritaire à l'addition, donc on l'effectue en premier. Les deux opérandes i et j sont de même type donc $5/3$ donne 1 (reste 2); y et 1 sont de type différent donc 1 est converti en 1.0, et additionné à y pour donner 3.0 qui est stocké dans x .

De même, l'affectation ($\text{variable}=\text{calcul}$) calcule d'abord l'expression à droite du signe $=$, ce n'est qu'après qu'il regarde le type de la variable devant recevoir le résultat pour faire, s'il le faut, une conversion. Vous pouvez donc écrire $x=i$ mais pas $i=x$.

exemple :

```
double precision x,y,z
x=5/3
y=5.0/3.0
z=5D0/3D0
print 10,x,y,z
```

10 *format (f18.15)*

affichera :

```
1.0000000000000000
1.666666626930237
1.666666666666667
```

ATTENTION : la conversion n'est automatique que dans ces cas. En particulier, elle n'est pas effectuée entre les arguments réels et formels d'un sous-programme ou fonction

9.2) conversion explicite

Vous pouvez par contre indiquer au compilateur quelles conversions il doit effectuer. Pour cela, on utilise les fonctions suivantes :

argument résultat	integer	real	double précision
integer (partie entière)		ifix (ou int)	idint
integer (entier le plus proche)		nint	idnint
real	float (ou real)		sngl
double précision	dfloat	dble	

Ici aussi, faites attention aux divisions : float(5/3) ne vaut pas float(5)/float(3)

9.3) fonctions intrinsèques

En plus des fonctions de conversion, Fortran propose diverses fonctions, qu'il est inutile de déclarer puisque déjà connues. Toutes les fonctions ci-dessous retournent une valeur de même type que leur argument.

	integer	real	double précision
racine carrée		sqrt	dsqrt
exponentielle		exp	dexp
log népérien		alog	dlog
log base 10		alog10	dlog10

valeur absolue	iabs	abs	dabs
minimum	min0	amin1	dmin1
maximum	max0	amax1	dmax1
sinus		sin	dsin
cosinus		cos	dcos
tangente		tan	dtan
arccos		acos	dacos
arcsin		asin	dasin
arctan		atab	datab

10) Variables communes

Pour que différents sous-programmes puissent échanger des données, la solution la plus évidente est de les passer en arguments. Mais quand un certain nombre de données doivent être utilisées dans toutes les parties d'un programme, on choisira plutôt le *common*. Il consiste à donner un nom à une liste (ordonnée) de variables à partager.

common/nom/variable1,variable2,...

Le *common* sera déclaré dans chaque entité de programme (sous-programme, fonction) qui doit accéder aux variables communes. Le nom définit le *common*. Les variables communes par contre sont définies par leur ordre, et pourraient avoir des noms différents dans toutes les entités (mais je vous le déconseille fortement), et doivent y être déclarées. Il peut y avoir plusieurs *common* différents, mais une variable ne peut appartenir qu'à un seul *common*.

Exemple : dans un programme d'analyse de l'évolution de la température, toutes les entités doivent connaître les différentes températures (et leur nombre)

```

    real maxi()
    common/temp/nb,t
    integer nb,i
    real t(100),m
    m=t(1)
    DO 100 i=2,nb
        if(t(i).gt.m)m=t(i)
100  continue
    maxi=m
    return
end
c
    program main
```

```

common/temp/nb,t
integer nb
real t(100)
---- etc ----
end

```

Les variables des différents common peuvent être initialisées (si nécessaire) dans une entité nommée bloc data qui ne peut comporter que des common, des déclarations et des data :

```

block data
common/temp/nb,t
integer nb
real t(100)
data nb /0/
end block data

```

Ne mettez qu'un seul block data dans un programme !

Pour éviter de recopier partout les mêmes common, on peut utiliser l'instruction

```
INCLUDE 'nomd1fichier'
```

Le fichier désigné est inclus à cet endroit. Attention, cette possibilité n'est pas disponible partout, mais sous g77 elle l'est (comme d'ailleurs toutes les directives # du C).

11) Entrées, sorties et formats

11.1) ouverture et fermeture du flux

Le programme peut lire (resp. écrire) des données depuis (resp. vers) l'extérieur par l'intermédiaire d'une "unité logique". Une unité logique (un flux) correspond à une interface de l'ordinateur qui transmet un flot de données entre la carte mère et un de ses périphériques. Ce sont principalement les fichiers sur disque, le clavier et l'écran, mais peut aussi être un lecteur de bandes, un port RS232, la carte son,... à condition que le système d'exploitation y donne accès (dans /dev sous Unix).

La première chose à faire avant d'utiliser un flux est de l'ouvrir. C'est une sorte de "demande de réservation" faite à l'OS, qui peut la refuser (si il est déjà ouvert par un autre programme, si il n'existe pas, si vous n'avez pas les bons droits d'accès,...). Chaque flux ouvert est associé à un "numéro d'unité logique" qui est un entier positif (en général entre 9 et 99), qui nous servira à désigner le flux dans la suite du programme. Dans le cas d'un fichier, l'ouverture se fait par l'instruction OPEN :

```
open(UNIT=numéro,FILE='nom du fichier',STATUS='état',ERR=label,Iostat=variable
entière)
```

- Si le numéro est donné en premier, il n'est pas nécessaire de le précéder par UNIT=. C'est le programmeur qui fixe le numéro, il doit être unique dans le programme et ne peut être variable (mais un parameter est

- possible).
- Le nom du fichier peut être donné directement entre apostrophes, ou être une variable chaîne (c'est également le cas pour d'autres arguments mais moins courant).
- Le statut est optionnel, il vaut soit NEW si le fichier doit obligatoirement ne pas exister avant (pour ne pas l'écraser), soit OLD s'il doit exister avant (pour la lecture par exemple), soit SCRATCH si le programme doit le créer le temps du programme et le détruire à la fin, soit UNKNOWN (ou ne rien mettre) s'il doit l'ouvrir, après l'avoir créé si nécessaire. APPEND permet de se placer à la fin du fichier (pour rajouter du texte à la suite).
- En cas de refus d'ouverture, la variable désignée par IOSTAT reçoit un code d'erreur (0 si ok), et le programme saute directement au label désigné par ERR (s'il sont omis, le programme s'arrête en cas d'erreur).
- On peut aussi utiliser le fichier en accès direct (ACCESS='DIRECT') en imposant la taille des enregistrements (RECL=entier), qui seront soit directement écrits en binaire dans le fichier (FORM='UNFORMATED') soit traduits en ASCII (FORM='FORMATED').

Il y a deux unités qui sont automatiquement ouvertes au début du programme : 5 pour le clavier et 6 pour l'écran.

Lorsque vous avez fini d'utiliser un flux, il faut le refermer, en particulier pour en donner accès aux autres. Tout flux non encore fermé le sera automatiquement à la fin du programme. Pour cela, on utilise l'instruction :

```
close(unit=numéro,IOSTAT=variable, ERR=label)
```

exemple :

```
open(10,FILE='resultats.txt')
---- etc ----
close(10)
```

11.2) lecture - écriture

Pendant qu'il est ouvert, on écrit dans un flux par

```
write (unité,format) liste de variables
```

et l'on lit dans un flux par

```
read (unité,format) liste de variables
```

Ces deux écritures acceptent un format plus complet :

```
(UNIT=unité, FMT=format, IOSTAT=variable, ERR=label, END=label)
```

Les options sont identiques à l'open, plus END qui donne le label où doit sauter le programme quand on est arrivé à la fin du fichier (pratique quand on n'en connaît pas la longueur).

En cas d'écriture à l'écran, l'unité est 6, mais on peut l'appeler *. On peut même utiliser

```
print format, liste de variables
```

Pour la lecture au clavier, l'unité est 5, mais on peut l'appeler *. On peut également utiliser :

`read format, liste de variables`

La liste de variables contient des variables ou expressions séparées par des virgules, mais on peut également utiliser une "boucle implicite" :

`(expression contenant un indice, indice=début,fin,pas)`

Par défaut, le pas vaut 1.

exemples :

```
read *,a,b
write (*,*) 'a vaut',a,'et b vaut',b
write (6,100) a,b,( t(i),i=1,10),c
print *, ( (i+3*j, i=1,3), ' ',j=0,4)
```

la dernière ligne affichera :

`1 2 3 - 4 5 6 - 7 8 9 - 10 11 12 - 13 14 15 -`

11.3) déclaration du format

Le format peut-être soit * (format libre : le compilateur choisit le format d'affichage qui lui paraît le plus adapté), soit un label (format imposé : au label donné, le programmeur déclare le format d'affichage), soit une variable chaîne de caractères (format calculé : le format est créé par le programme, et peut donc varier suivant les calculs précédents).

La déclaration du format imposé (qui doit être dans la même entité de programme) est de la forme :

`label FORMAT(liste de composants)`

Les composants du format sont :

- 'texte entre apostrophes' : le texte est écrit tel quel
- In : entier sur n caractères (avec espaces devant si le nombre est plus petit)
- An : chaîne sur n caractères (avec rajout d'espaces en fin si nécessaire)
- Fn.p : réel sur f caractères dont p après la virgule (F5.2 donne 12.45)
- En.p : réel en notation scientifique dont p pour la partie décimale (l'exposant en prend 4, ne pas oublier le . et peut-être le signe -) (E9.2 donne -2.45E+09)
- Gn.p : réel en Fn.p si possible, En.p s'il est trop grand
- Dn.p : idem En.p mais pour un réel double précision
- X : un espace (ou ignorer un caractère, dans le cas d'un read)
- / : un retour à la ligne (ou ignorer la fin de la ligne dans un read)

Un composant précédé d'un entier n est répété n fois. Pour répéter plusieurs composants, il faut les mettre entre parenthèses : 3(5X,2I1) : 3 fois (5 espaces et 2 entiers d'un chiffre).

Exemples :

```

write (*,10) nb,moy
10  format ('la moyenne des ',l2,'températures est de ',F5.1,' degrés')
write (*,11) ((mat(i,j),i=1,4),j=1,4)
11  format ('la matrice est',4(/,'|', 4F5.2,'|')

```

Chaque read ou write correspond à une ligne. Si l'on donne une liste de plusieurs variables dans un read, il faudra absolument entrer toutes les valeurs (entre espaces ou virgules) avant la touche Entrée. Mais certains compilateurs (G77 par exemple) considèrent qu'un caractère '\$' en fin de format demande de ne pas retourner à la ligne.

remarque : on peut également écrire ou lire dans une variable chaîne de caractères, il suffit de remplacer le numéro d'unité par le nom de la chaîne. De même, le format peut être une chaîne.

```

character*40 fmt
integer i,nbchif
data i /123/ , nbchif /4/
write(fmt,10) nbchif
10  format(' '*','|',l1,' '*')
write (*,*) 'pour vérification FMT vaut:',fmt
write (*,fmt) i

```

ceci affichera :

```

pour vérification FMT vaut:( '*|l4,')
* 123*

```

12) Que n'ai-je pas dit ?

Plein de choses. En particulier, il existe un type et des fonctions intrinsèques spécialement conçus pour le traitement des nombres complexes : le type complex.
